Python/CREXX

A Python 3 subset interpreter in strict C89 and in CREXX

Language Reference and User's Guide

Release 1 • Build 10 • 12 October 2026
for CREXX 1.0.0 beta 3 and later (CREXX interpreter) and any C89 compiler (C interpreter)

Publication PYC-0001-0
File No. PYTHON-01

First Edition, revised (October 2026)

This edition applies to Python/CREXX Release 1 (build 9, 12 October 2026), a pair of interpreters for a subset of the Python 3 language, one written in strict C89 and one written in CREXX Level B, and to all subsequent builds until otherwise indicated in new editions.

Python/CREXX is a hobbyist project. It is not a product of, and this manual is not a publication of, the Python Software Foundation or of the authors or owners of any Python implementation or library. Python, NumPy, pandas and SciPy are names of languages and products, used here only to identify the language and the libraries with which this software is compatible and the implementations to which it is compared.

CREXX is an open-source REXX implementation and is maintained by its own community. The CREXX interpreter of Python/CREXX needs it to run, and starts about five times faster on a CREXX virtual machine that has the small patch supplied with the repository (Chapter 2); the C interpreter needs only a C compiler.

The project repository holds the source of both interpreters, the shared library, the regression tests and the benchmark programs from which every statement in this manual can be checked. Comments on this publication may be addressed to the project.

Contents

Preface ... 1

Chapter 1. Introduction ... 2
- The two implementations ... 2
- Status ... 2
- What is in the repository ... 3

Chapter 2. Installing and Running ... 4
- What you need ... 4
- Building the C interpreter ... 4
- Finding the library ... 5
- Running the CREXX interpreter ... 5
- Start-up time and the patched virtual machine ... 5
- The command line ... 6
- Exit codes and tracebacks ... 6
- Portability notes ... 7
- The regression tests ... 7

Chapter 3. The Python Language ... 9
- Source text ... 9
- Names, assignment and unpacking ... 9
- Numbers and the arithmetic operators ... 10
- Strings ... 93
- String formatting ... 12
- Control flow ... 13
- Functions ... 14
- Classes ... 17
- Subclassing the built-in types and __new__ ... 20
- Exceptions ... 23
- Iterators and generators ... 25
- Comprehensions ... 27
- Modules and import ... 27
- The with statement ... 28
- Other statements ... 29

Chapter 4. Built-in Functions and Types ... 30
- Built-in functions ... 30
- Methods of str ... 31
- Methods of list and tuple ... 32
- Methods of dict and set ... 33
- Methods of int, float and bool ... 34
- complex, bytes and bytearray ... 35

Chapter 5. The Standard Library ... 37
- math ... 37
- random ... 38
- sys and os ... 39
- time ... 40
- json ... 40
- re ... 41
- cmath ... 42
- struct ... 42
- socket and select ... 43
- datetime ... 44
- collections ... 45
- itertools and functools ... 45
- statistics, string, copy, operator, textwrap ... 46
- csv, heapq, bisect ... 47
- fractions and decimal ... 48
- enum, dataclasses, abc, contextlib, typing ... 49
- pprint, io, warnings, types ... 50

Chapter 6. NumPy ... 52
- Creating arrays ... 52
- Indexing and slicing ... 52
- Arithmetic and broadcasting ... 53
- Reductions, sorting and searching ... 55
- Linear algebra ... 55
- Random numbers ... 58

Printing ... 74
Data types and scalar types ... 59
Complex arrays ... 60
Fourier transforms: numpy.fft ... 61
Function reference ... 62
Differences from real NumPy ... 64

Chapter 7. pandas ... 65
Series ... 65
DataFrame ... 66
Missing data ... 69
Grouping and aggregation ... 69
Combining frames ... 71
Strings, dates and windows ... 72
Dates and times: Timestamp, Timedelta, DatetimeIndex ... 72
Reading and writing ... 74
Printing ... 74
Differences from real pandas ... 75

Chapter 8. SciPy ... 76
scipy.stats ... 76
scipy.special ... 77
scipy.optimize ... 78
scipy.integrate and scipy.interpolate ... 79
scipy.linalg and scipy.sparse ... 80
scipy.fft ... 81
scipy.spatial, scipy.signal and scipy.constants ... 82
Differences from real SciPy ... 83

Chapter 9. Errors and Exceptions ... 84
Uncaught exceptions and exit status ... 84
The exception hierarchy ... 85
Exception objects ... 86
Messages of the common errors ... 87
Warnings and the standard error stream ... 90

Chapter 10. How Python/CREXX Works ... 91
Overview ... 91
The lexer ... 91
The parser ... 91
Scope resolution ... 91
The evaluator ... 92
Generators ... 92
Integers and floats ... 92
Memory management in the C interpreter ... 92
Strings ... 93
The pure-Python library layer ... 93
Native regular expressions and JSON in the C interpreter ... 93
Complex numbers, bytes and subclasses of built-in types ... 93
Amalgamation ... 94
The CREXX interpreter ... 97
Memory management in the CREXX interpreter ... 95

Chapter 11. Performance ... 96
The C interpreter against CPython ... 96
Native re and json ... 96
Start-up and imports ... 97
The CREXX start-up and the patched virtual machine ... 97
The CREXX interpreter ... 97
Reproducing the measurements ... 98

Chapter 12. Restrictions and Known Problems ... 99
Syntax that is not accepted ... 99
Strings and bytes ... 99
Numbers ... 100
Containers ... 101
Classes and objects ... 102
Namespaces, eval and exec ... 102
Input, output and the operating system ... 102
The libraries ... 103
The CREXX interpreter in particular ... 104
Known defects ... 104

Chapter 13. Porting to VM/370 106
The rules 106
The host adapter 106
Embedded libraries 107
The deck step 108
Building and checking 109
The 32-bit long hunt (Build 4) 109
The same hunt in the CREXX interpreter (Build 5) 110
Assumptions that remain 111
What remains for the real port 111
Appendix A. Command Summary 113
Appendix B. Contents of the Repository 114
Appendix C. Index of Modules 116
Appendix D. Sample Programs and Tests 117
Appendix E. Differences between the C and CREXX Interpreters 119

Preface

This manual introduces Python/CREXX, a pair of interpreters for a large subset of Python 3: one written in strict C89, one written in CREXX. It tells you how to build and run them and describes the language they implement, the built-in functions and types, the standard library modules, and the NumPy, pandas and SciPy subsets that are built into them. It also explains how the interpreters work and lists what does not work yet.

It is written for the programmer who already knows Python and wants to know what this implementation will and will not run, and for the programmer who wants to read, port or extend the interpreters. Where Python/CREXX behaves as CPython 3 does, this manual says little; where it differs, it says how. Every example program in the manual was run, and the output shown is the output that was printed.

How this manual is organized

- **Chapter 1, Introduction**, says what Python/CREXX is, how the two implementations relate and how complete they are.
- **Chapter 2, Installing and Running**, covers building the C interpreter, the CREXX launcher, the command line, the REPL, exit codes and the test runner.
- **Chapter 3, The Python Language**, describes every statement and expression form with examples.
- **Chapter 4, Built-in Functions and Types**, is the reference for the built-in functions and for the methods of str, list, tuple, dict, set, int and float.
- **Chapter 5, The Standard Library**, describes the modules math, random, sys, os, time, json, re, cmath, struct, socket, select, datetime, collections, itertools, functools and the others.
- **Chapter 6, NumPy**, **Chapter 7, pandas** and **Chapter 8, SciPy**, describe the scientific subsets and how they differ from the real libraries.
- **Chapter 9, Errors and Exceptions**, lists the exception classes and the messages of the common errors.
- **Chapter 10, How Python/CREXX Works**, describes the lexer, parser, scope resolution, evaluator, generators, integers, memory management, the library layer and the start-up of the CREXX interpreter.
- **Chapter 11, Performance**, gives measured timings against CPython 3.
- **Chapter 12, Restrictions and Known Problems**, lists what does not work yet. **Chapter 13, Porting to VM/370**, describes the deck step, the embedded library, the host adapter and what remains to be done for the real port.
- **Appendix A** summarizes the command line, **Appendix B** lists the files of the repository, **Appendix C** is an index of the modules, **Appendix D** lists the sample programs and tests, and **Appendix E** lists the differences between the C and the CREXX interpreters.

Conventions

Program text is shown in monospaced type. In a figure that shows a run, the program comes first, then a line that starts with a dollar sign and shows the command, then the output of the interpreter. The command py stands for ./c/py or ./crexx/py. In tables, an entry marked C exists only in the C interpreter and an entry marked X only in the CREXX interpreter. The name of the manual's subject is Python/CREXX; the interpreters themselves are called the C interpreter and the CREXX interpreter.

Python 3 means the language as implemented by CPython 3.12 and 3.13, with which the output of the interpreters was compared. The names Python, NumPy, pandas and SciPy refer to the languages and libraries that this software is compatible with and are used to identify them.

Chapter 1. Introduction

Python/CREXX is a Python 3 interpreter, written twice. The first version is a strict C89 program in one amalgamated source file, meant to be portable to unusual systems such as VM/370; the second is a program in CREXX, the open-source REXX implementation, which gives the language a home on the CREXX virtual machine. Both read ordinary Python 3 source and aim to print exactly what CPython 3 prints. Neither is CPython and neither uses it: the lexer, parser, evaluator, big-integer arithmetic, floating-point printing, garbage collector (in C), regular-expression engine and array library were written from scratch.

The two interpreters share three things:

- **One language specification** (SPEC.md): the language, the built-in types, the error messages and the command line.
- **One test corpus** (tests/*.py with the expected output in tests/*.out). The expected output was generated by CPython 3 itself; a test passes only if the interpreter prints exactly the same text. There are 33 tests, and both interpreters pass all of them.
- **One library directory** (pylib/): most of the standard library modules, pandas, SciPy, the FFT and the less common parts of NumPy are written in Python and are loaded by the interpreters like any user module. NumPy's array engine, math, random, sys, os, time, complex numbers, bytes and struct are native in each interpreter, and the C interpreter also has native re, json, cmath, socket and select (Chapter 5, Appendix C).

The two implementations

	C interpreter (c/py)	CREXX interpreter (crexx/py)
Written in	strict C89, 27,800 lines in 23 source files, amalgamated to c/py_all.c (and a generated 57,500-line table of the embedded library)	CREXX Level B, 21,150 lines in 23 source files, assembled into crexx/py.crexx (537 KB)
Needs	a C compiler and the C math library	the CREXX toolchain (crexx driver, compiler, assembler, VM)
Start-up	about 2 ms	about 0.4 seconds with the patched CREXX virtual machine, about 2 to 3 seconds with the stock one (loading 3.1 MB of bytecode)
Speed	about the same as CPython 3.13 on the benchmark programs	10 to 100 times slower than CPython; a teaching and scripting tool
Strings	sequences of bytes; UTF-8 passes through	sequences of Unicode code points
Native modules	math random sys os time numpy re json cmath struct socket select	math random sys os time numpy and a few more; re, json, cmath, struct are Python from pylib; socket and select use the rxsocket library
Memory	mark-and-sweep garbage collector	mark-and-sweep garbage collector at safe points (options --memstat, --nogc, --gcfix, --gcstress); frames of generators that define closures are not yet freed (Chapter 10)
REPL	yes (py with no argument, or py -i)	no (py with no argument runs a program read from standard input)
Portability	any hosted C89 system; written to run on VM/370 CMS (Chapter 13)	any system that runs CREXX

Figure 1. The two interpreters

Status

The figure below shows the major areas of Python and their status in this release. A blank in the last two columns means complete.

Area	Content	C	CREXX
Statements	if, while, for, break, continue, pass, else on loops, def, class, return, global, nonlocal, del, assert, try/except/else/finally, raise, with, import		
Expressions	all operators, chained comparisons, conditional expressions, lambda, comprehensions, generator expressions, slicing, star-unpacking, f-strings		

Area	Content	C	CREXX
Assignment expressions	the walrus operator :=		
Pattern matching	the match statement with literal, capture, wildcard, value, sequence, mapping, class, OR and as patterns and guards		
Numbers	int of any size, float, bool; bit operations on ints of any size (two's complement); CPython-compatible hash() of numbers	bit operations only up to 18 digits; hash(int) is the integer for small values	bit operations only up to 18 digits
Complex numbers	1j, complex(), cmath; numpy complex128		
Strings	str, all common methods, % formatting, format(), f-strings	bytes, not code points	
bytes	b"..." literals and methods, bytearray, struct, int.to_bytes		memoryview and the array module are absent in both
Sockets	socket and select: IPv4 TCP	POSIX only (-DPY_POSIX)	through rxsocket
Containers	list, tuple, dict (ordered), set, frozenset, range, slice	sets iterate in CPython order for int-like keys, else insertion order	sets keep insertion order
Functions	defaults, *args, **kwargs, keyword-only, closures, decorators, generators, send/throw/close, yield from	lambda cannot contain yield	
Classes	single and multiple inheritance (C3), super(), properties, class and static methods, special methods, __new__, subclasses of the built-in types	metaclasses ignored	metaclasses ignored; super() in classmethods fails
Exceptions	hierarchy, messages, finally, chaining syntax, ExceptionGroup and except*	no __cause__	no __cause__
Modules	import, from, as, packages, relative imports, sys.modules, circular imports		
Not supported	async/await, type-parameter syntax, exec of bytes, input() prompts to a terminal other than standard input		
Standard library	about 40 modules (Chapter 5, Appendix C)		
NumPy	ndarray subset with complex128, dtype objects and np.fft (Chapter 6)		integer overflow gives float
pandas	Series, DataFrame, groupby, merge, IO, datetime (Chapter 7)		
SciPy	stats, special, optimize, integrate, interpolate, linalg, spatial, signal, sparse, fft, constants (Chapter 8)		

Figure 2. Python/CREXX language areas

What is in the repository

The repository holds, besides the two interpreters, the shared library (pylib), the tests (tests), a specification (SPEC.md), a benchmark set (c/bench), the porting notes for VM/370 (c/PORTING.md), the start-up notes and the patch for the CREXX virtual machine (crexx/STARTUP.md, crexx/vm-startup.patch) and this manual (docs). Appendix B lists the files. The interpreters need no installation: they run from the directory in which they were built, and find pylib by looking next to the executable or by the PYLIB environment variable.

Chapter 2. Installing and Running

What you need

For the C interpreter, any C compiler that accepts C89 and the C math library: gcc and clang with -std=c89 -pedantic -Wall -Wextra produce no warnings. For the CREXX interpreter you need CREXX 1.0.0 beta 3 or later: the crexx driver together with the compiler, assembler and virtual machine that it calls. The build script crexx/build.py needs Python 3, but only if you change the CREXX sources. Python 3 is also needed to regenerate the expected output of the tests, never to run the interpreters.

Building the C interpreter

```
$ make -C c posix
make: Entering directory '/home/claude/python/c'
cc -std=c89 -pedantic -Wall -Wextra -O2 -DPY_POSIX
    -DPY_LIBDIR='"/home/claude/python/pylib"' -o py py_all.c -lm
make: Leaving directory '/home/claude/python/c'
$ ./c/py -c "import sys; print(sys.version)"
3.12.0 (C89 subset interpreter)
```

Figure 3. Building and starting the C interpreter

The Makefile has these targets:

Command	Effect
make	cc -std=c89 -pedantic -Wall -Wextra -O2 -DPY_LIBDIR=... -o py py_all.c -lm; the portable build
make posix	the same with -DPY_POSIX: a real-time clock for time.time(), a sleep() that sleeps, os.getcwd(), the environment in os.environ, and a prompt in the REPL when standard input is a terminal
make LIBDIR=/path/to/pylib	sets the compile-time fall-back directory for the library (default: the absolute path of ../pylib)
make test	builds the posix build (as make posix, with -DPY_POSIX, since test 24_sockets needs sockets) and runs ../tests/run_tests.sh ./py
make embed	./py_embed: the pure-Python library pylib/ compiled into the binary (-DPY_EMBED_LIBS), so that no library directory is needed
make deck make deck-embed	generate deck/py_deck.c and deck/py_deck_embed.c: the amalgamation folded to 71 columns for the VM/370 deck step (Chapter 13)
make deckcheck	fails if a line of a deck is longer than 71 columns or contains a tab, a non-ASCII character or a trigraph
make deck-test embed-test deck-embed-test	compile the deck, the embedded build or the embedded deck with strict flags and run the 33 tests (the embedded builds from an empty directory, with no PYLIB and a bogus PY_LIBDIR)
make strict	conformance matrix: gcc and clang with -std=c89 -pedantic-errors and many warnings, no C99 library function referenced, the whole suite under AddressSanitizer and UBSan
make linecheck	fails if a development source line is longer than 100 columns
make bench	runs bench/run.sh, which times the benchmark programs against python3 (and the native re and json against the pure-Python library)
make clean	removes the executables
sh tools/amalg.sh > py_all.c	regenerates the amalgamated source from src/*.c (py_all.c is checked in and is what make compiles)

Figure 4. Makefile targets of c/

Use make posix on any POSIX system. The plain make builds a strictly portable program that uses only the C library: time() has a resolution of one second, sleep() busy-waits, os.getcwd() returns ".", os.environ is empty and the socket module is present but cannot create a socket (OSError: sockets not available). The flags -DPY_GC_STRESS (collect at every safe point, for testing the collector) and -DPY_LIBDIR="..." (the library directory) can be added to CFLAGS or the command line.

Finding the library

The standard library modules, pandas and SciPy are Python files in pylib. The C interpreter looks for a module in these places, in this order: the directory of the script (the empty string, for the current directory, in the REPL and with -c); in a build made with make embed, the library compiled into the program, which appears in sys.path as the pseudo directory <pylib>; each directory named in the environment variable PYLIB (separated by colons); pylib next to the executable; ../pylib relative to the executable (the arrangement of the repository); and the compile-time directory PY_LIBDIR. The result is the list sys.path. Native modules (math, random, sys, os, time, numpy, re, json, cmath, struct, socket, select) win over files of the same name; the Appendix C index shows which module is native in which interpreter.

```
$ ./c/py -c "import sys; print(sys.path)"
['', './c/pylib', './c/../pylib', '/home/claude/python/pylib']
$ PYLIB=/usr/local/lib/mypy ./c/py -c "import sys; print(sys.path)"
['', '/usr/local/lib/mypy', './c/pylib', './c/../pylib', '/home/claude/python/pylib']
$ ./c/py_embed -c "import sys; print(sys.path)"
['', '<pylib>', './c/pylib', './c/../pylib', '/nonexistent/pylib']
```

Figure 5. sys.path in the C interpreter

Running the CREXX interpreter

The launcher crexx/py is a POSIX shell script. It finds the crexx driver in the environment variable CREXX_VM if that is set (a driver built from a patched CREXX, see below), else in CREXX, else as crexx on the PATH. It compiles crexx/py.crexx to bytecode on the first run (and again whenever py.crexx is newer than py.rxbin), and then runs the bytecode. The compilation takes several minutes the first time (it writes py.rxas, assembler text, and py.rxbin, 3.1 MB of bytecode). Later runs start in about 0.4 seconds with the patched virtual machine and in 2 to 3 seconds with a stock one.

Start-up time and the patched virtual machine

Before the interpreter runs the first line of a program, the CREXX virtual machine must load the 3.1 MB of bytecode and about 280 library modules and link them. Profiling the stock virtual machine (callgrind on rxvme, 1.6 billion instructions for print(1)) showed that 58 per cent of the time went into rebuilding the interface-method and factory registries during the link step, mostly in two linear scans of the list of every module's meta entries (py.rxas has about 13,000 of them); only 5 per cent was spent in the program itself. The file crexx/vm-startup.patch (which touches only interpreter/rxvmintp.c of CREXX; crexx/STARTUP.md describes it in full) builds, once per registry rebuild, a hash index of the meta lists: procedures by lower-cased name and by pointer, members by owner, and implements entries by class and interface. The first match still wins and the registry contents are unchanged. Outside a rebuild the old linear code is used.

```
$ CREXX_VM=/path/to/patched/bin/crexx ./crexx/py -c "print(6 * 7)"
42
```

Figure 6. Choosing the virtual machine with CREXX_VM

Virtual machine	Start-up	Test suite (33 tests)
stock CREXX 77ba820c	3.4 s (1.6 to 3.4 s in repeated measurements)	160 s
patched (vm-startup.patch)	0.39 s	74 s

Figure 7. Start-up of the CREXX interpreter, print(1) (this machine, best of three)

To use the patch, apply it to a CREXX source tree (tested against commit 77ba820c), build rxvme and the driver as usual, and point CREXX_VM at the patched crexx driver: git apply crexx/vm-startup.patch. The patched bin directory must be a complete build tree, since the driver finds rxvme and its plug-ins next to itself. Nothing in the Python/CREXX sources depends on the patch, and without it everything works as before, only slower.

The patch was checked against the CREXX project's own test suite. Both a stock and a patched virtual machine were built from clean copies of the same commit (Release configuration) and the CTest labels essential, smoke and comprehensive were run on each: 2289 of 2289 tests pass on both, with identical per-

test results; the 12 qualification and stress tests also pass on both. The 186 timing-only performance-measurement tests were not run. The interface-registry tests (interface_*, dynamic_interface_load_rxvm and dynamic_interface_load_driver), which exercise the changed code, are among those that pass.

```
$ export CREXX_VM=/path/to/patched/bin/crexx   # or CREXX=/path/to/crexx
$ ./crexx/py hello.py
py: compiling interpreter (first run only)...
Hello from CREXX
$ ./crexx/py hello.py
Hello from CREXX
```

Figure 8. Running the CREXX interpreter

The launcher passes the directories crexx/pylib (the CREXX-only modules socket and select) and ../pylib (relative to the launcher) to the interpreter with --pypath=. The interpreter's search path is: the script directory, the directories of the PYLIB environment variable, the --pypath directories, and finally the directory /home/claude/python/pylib, whose name is compiled into the sources (it is the last item in the list; change it in crexx/src/h_imp.crexx if you keep the repository elsewhere). The CREXX driver serialises concurrent runs with a lock file next to py.crexx; a second interpreter started while the first is running waits for it. If a run is killed, an orphaned rxvme process can keep the lock: kill it.

The command line

Form	Meaning
py file.py [arg ...]	run the script; sys.argv is ['file.py', arg, ...]
py -c "code" [arg ...]	run the code in the string; sys.argv is ['-c', arg, ...]
py	C: start the REPL on standard input (a prompt >>> is printed when standard input is a terminal in a posix build). CREXX: read a program from standard input and run it
py -i	C: force the REPL prompts even when input is not a terminal
py -h	CREXX: print a usage line (the C interpreter treats -h as a file name)
py --memstat --nogc --gcfix=N --gcstress=N file.py	CREXX: garbage-collector options, written before the script name (Chapter 10)

Figure 9. Command line

```
$ py -c "import sys; print(sys.argv)" a b
['-c', 'a', 'b']
$ echo "print(6 * 7)" | py
42
$ printf "x = 5\nx * 2\ndef f(a):\n    return a * 3\n\nf(4)\n" | py -i
>>> >>> 10
>>> ... ... >>> 12
>>>
```

Figure 10. The command line

In the REPL the value of an expression statement is printed with repr() unless it is None. A line that ends with a colon (or a decorator) starts a block that is ended by an empty line; unclosed brackets continue on the next line, and the prompt changes from >>> to ... A traceback does not end the session.

Exit codes and tracebacks

Status	Meaning
0	the program ended normally (or sys.exit(0), or SystemExit with None)
1	an uncaught exception (a traceback was written to standard error); also sys.exit() with a non-integer argument, which is printed
n	sys.exit(n); in the CREXX interpreter the status is taken modulo 256
2	the script file could not be opened (C: python: can't open file 'name'; CREXX: py: can't open file 'name': [Errno 2] No such file or directory); also, for the CREXX launcher, a failure to compile the interpreter

Figure 11. Exit status

An uncaught exception prints Traceback (most recent call last):, a line File "name", line N and the last line Class: message, all on standard error. Chapter 9 shows examples. Nothing is ever written to standard output by the error machinery, so redirecting standard output captures only the program's output.

Portability notes

The C source follows strict rules so that it can be compiled on small and old systems, in particular VM/370 with its C compiler and EBCDIC character set.

- Strict C89: no // comments, no declarations after statements, no long long, no stdint.h, no variable-length arrays, no inline. All functions are static; the only external symbol is main. Lines are at most about 100 columns.
- Character classes are tested with literal sets (for example "abc...xyz"), never with code ranges such as c >= 'a' && c <= 'z', so the code works unchanged in EBCDIC.
- A small integer is a long of at most 18 decimal digits where long has 64 bits and 9 digits where long has 32 bits; larger values are big integers. The 32-bit range is what a 32-bit system such as VM/370 gets.
- Operating-system services (the clock, sleep, getcwd, the environment, isatty) are compiled only with -DPY_POSIX. Without it the interpreter uses only the standard C library: stdio fopen for files, time(), clock() and the math library.
- All the tables that grow (global names, the value stack, frames) are plain arrays grown with realloc. The program is compiled as one file (py_all.c) for systems whose build tools want that.
- The test suite was run with gcc and clang and under AddressSanitizer and UBSan: all 33 tests are clean. Chapter 13 describes the checks (make strict) and the deck step.

The C interpreter has been built and tested only on a modern 64-bit Linux system for this manual (and on a 64-bit host that simulates a 32-bit long, see Chapter 13). The commit history records one run of the test corpus on VM/370plus CMS (GCC 13, -m31, a 32-bit long), which found a float-to-integer conversion that saturated at 2,147,483,647 (fixed in commit 6e4a7a5, see Chapter 13); that run was not repeated for this manual, so the other claims about that system are design intentions, not test results. The floating-point format of that system (IBM hexadecimal floating point) means that the printing of floating-point numbers may differ there.

The regression tests

tests/run_tests.sh runs every tests/NN_name.py with the interpreter given as its arguments and compares the standard output with tests/NN_name.out, which was generated by CPython 3. It prints ok or FAIL for each test and a summary, and its exit status is 0 only if all tests pass. VERBOSE=1 shows the first lines of the difference for each failure (VERBOSE_LINES=n changes the number of lines); SKIP="15_import 17_language2" skips the named tests.

```
$ tests/run_tests.sh ./c/py
ok   01_basics
ok   02_strings
ok   03_control
ok   04_functions
ok   05_collections
ok   06_classes
ok   07_exceptions
ok   08_iter_gen
ok   09_algos
ok   10_misc
ok   11_numpy
ok   12_stdlib
ok   13_pandas
ok   14_scipy
ok   15_import
ok   16_linalg_random
ok   17_language2
ok   18_re
ok   19_modules
ok   20_pandas2
ok   21_scipy2
ok   22_complex
ok   23_new_subclass
ok   24_sockets
ok   25_fft
ok   26_pandas_dt
ok   27_pandas_dt2
ok   28_fft_more
ok   29_np_dtype_pkg
passed 29, failed 0, skipped 0
```

Figure 12. Running the test suite with the C interpreter

Both interpreters pass all 33 tests: ./tests/run_tests.sh ./c/py takes less than a second, ./tests/run_tests.sh ./crexx/py takes about a minute and a quarter with the patched virtual machine (74 seconds with 33 tests, 77 seconds with 32 tests; 59 seconds for the first 29 tests before the collector was added) and about two and two thirds minutes with a stock one (160 seconds, measured with 31 tests), because every test pays the start-up time of the CREXX interpreter and the larger tests run the pandas, SciPy and FFT libraries. Appendix D lists the tests.

Chapter 3. The Python Language

Py runs ordinary Python 3 programs of the kind found in textbooks, scripts and small applications. This chapter describes every construct of the language that the interpreters support. Each example was run on the C interpreter and its output is shown as printed; the CREXX interpreter produces the same output except where an example is marked (Appendix E lists the places where the two differ).

Source text

A program is a sequence of logical lines. Indentation (spaces or tabs) groups statements into blocks: a compound statement ends with a colon and its body is a block indented more deeply than the statement. A line inside parentheses, brackets or braces may continue on the next line, and a backslash at the end of a line joins it to the next. Comments run from # to the end of the line. A string that stands alone as the first statement of a module, class or function (a docstring) is accepted and ignored.

```
# a comment
total = 1 + 2 + \
        3
names = ["ann",
         "bob"]          # brackets join lines
if total > 5:
    print("big", total)
    for n in names:
        print(" ", n)
else:
    print("small")

$ py prog.py
big 6
  ann
  bob
```

Figure 13. Indentation, comments and line joining

Names, assignment and unpacking

A name is bound by assignment. The right-hand side is evaluated first. Tuples and lists on the left unpack the value; one target may be starred and collects the remainder as a list. Assignment may be chained, augmented (+= -= = /= //= %= *= &= |= ^= <<= >>=) and may target subscripts and attributes. The walrus operator := binds a name inside an expression, in conditions, call arguments, subscripts and comprehensions (in a comprehension the name belongs to the enclosing function or module, not to the comprehension; the CREXX interpreter has it since Build 9).

```
a, b = 1, 2
a, b = b, a                    # swap
first, *rest = [10, 20, 30, 40]
*init, last = "abc"
x = y = 0
x += 5
(p, q), r = (1, 2), 3
d = {}
d["k"] = [1, 2]
d["k"][0] += 10
print(a, b, first, rest, init, last, x, y, p, q, r, d)

$ py prog.py
2 1 10 [20, 30, 40] ['a', 'b'] c 5 0 1 2 3 {'k': [11, 2]}
```

Figure 14. Assignment forms

```
if (n := len("hello")) > 3:
    print("long", n)
squares = [y for x in range(5) if (y := x * x) > 3]
print(squares, [z := 5, z + 1])

$ py prog.py
long 5
[4, 9, 16] [5, 6]
```

Figure 15. The walrus operator

Numbers and the arithmetic operators

int is unbounded: values that do not fit a machine word become big integers automatically. **float** is an IEEE double and prints with the shortest representation that reads back to the same value. **bool** is a subtype of int. Integer literals may be decimal, 0x hexadecimal, 0o octal or 0b binary and may contain underscores; float literals may have an exponent. **complex** is a pair of doubles: 3j and 1.5e3j are imaginary literals, 3 + 4j builds a complex number, and complex(real, imag) and complex("1+2j") convert.

```
print(2 ** 100)
print(7 // 2, -7 // 2, 7 % 3, -7 % 3, divmod(-7, 2))
print(7 / 2, 6 / 3, 1e22, 1e-5, 0.1 + 0.2)
print(0xff, 0o17, 0b101, 1_000_000)
print(10 ** 20 // 7, (10 ** 20) % 7)
print(abs(-3), round(2.5), round(3.5), round(2.675, 2), round(1234, -2))
print(True + True, int("42"), float("1.5"), int(3.9), int(-3.9))
print(5 & 3, 5 | 3, 5 ^ 3, ~5, 1 << 10, 1024 >> 3)
import math
print(math.factorial(25), math.gcd(12, 18), math.isqrt(10**20))

$ py prog.py
1267650600228229401496703205376
3 -4 1 2 (-4, 1)
3.5 2.0 1e+22 1e-05 0.30000000000000004
255 15 5 1000000
142857142857142857 2
3 2 4 2.67 1200
2 42 1.5 3 -3
1 7 6 -6 1024 128
15511210043330985984000000 6 10000000000
```

Figure 16. Integers, floats and division

Complex numbers support + - * / and unary minus with ints, floats and other complex numbers, use CPython's algorithms for division and powers (so results agree to the last digit), compare only with == and !=, hash like the equal int or float, and print as CPython prints them. Ordering comparisons raise TypeError, as in CPython; // % divmod and round are not defined for them either. The module cmath (Chapter 5) provides the functions.

```
z = 3 + 4j
w = complex(1, -2)
print(z + w, z * w, z / w, abs(z), z.conjugate(), z.real, z.imag)
print(z ** 2, 1j ** 2, (1 + 1j) ** 0.5, complex("2-3j"), 2j * 2j)
print(z == 3 + 4j, z != w, bool(0j), repr(1j), str(-1j), f"{z:.2f}", format(w, ".1e"))
print(sum([1j, 2, 3 + 1j]), {z: "a"}[3 + 4j], hash(3 + 0j) == hash(3))
try:
    z < w
except TypeError as e:
    print("TypeError:", e)
try:
    1 / 0j
except ZeroDivisionError as e:
    print("ZeroDivisionError:", e)
print((1 + 2j) * (1 - 2j), isinstance(z, complex), type(z).__name__)

$ py prog.py
(4+2j) (11-2j) (-1+2j) 5.0 (3-4j) 3.0 4.0
(-7+24j) (-1+0j) (1.0986841134678098+0.45508986056222733j) (2-3j) (-4+0j)
True True False 1j (-0-1j) 3.00+4.00j 1.0e+00-2.0e+00j
(5+2j) a True
TypeError: '<' not supported between instances of 'complex' and 'complex'
ZeroDivisionError: complex division by zero
(5+0j) True complex
```

Figure 17. Complex numbers

The figure below lists the operators from the loosest binding to the tightest.

Operators	Meaning
lambda	lambda expression
x if c else y	conditional expression
:=	assignment expression (walrus)
or	boolean or (returns an operand)
and	boolean and (returns an operand)
not x	boolean not
in, not in, is, is not, <, <=, >, >=, !=, ==	comparisons and membership; they chain: a < b < c
|	bitwise or, set union, dict merge
^	bitwise xor, set symmetric difference
&	bitwise and, set intersection
<<, >>	shifts
+, -	addition, subtraction, concatenation, set difference
*, /, //, %, @	multiplication, division, floor division, remainder, matrix product
+x, -x, ~x	unary operators
**	power (right associative)
x[i], x[i:j], x(args), x.attr	subscription, slicing, call, attribute reference

Figure 18. Operator precedence (loosest first)

```
x = 5
print(1 < x < 10, 1 < x < 3, x is None, x is not None)
print(0 or "default", "a" and "b", not [], [] or {})
print("yes" if x > 3 else "no")
print(2 ** 3 ** 2, -2 ** 2, (1, 2) < (1, 3), [1, 2] == [1, 2], "a" < "b")
print(3 in [1, 2, 3], "ell" in "hello", 4 not in (1, 2), "k" in {"k": 1})
print({1, 2} | {2, 3}, {1, 2} & {2, 3}, {1, 2} - {2, 3}, {1, 2} ^ {2, 3})
print({"a": 1} | {"b": 2})

$ py prog.py
True False False True
default b True {}
yes
512 -4 True True True
True True True True
{1, 2, 3} {2} {1} {1, 3}
{'a': 1, 'b': 2}
```

Figure 19. Comparisons, boolean operators and conditional expressions

Strings

A **str** is written with single, double or triple quotes. The escapes \n \t \\ \' \" \r \0 \xHH \uHHHH are recognised, a prefix r gives a raw string, and adjacent literals are concatenated. Strings are immutable. In the C interpreter a string is a sequence of bytes (UTF-8 text passes through unchanged but len() counts bytes); in the CREXX interpreter it is a sequence of Unicode code points. See Chapter 12. Binary data has its own types, bytes and bytearray (Chapter 4), written b"...".

```
s = "Hello, World"
print(s[0], s[-1], s[7:], s[:5], s[::2], s[::-1], len(s))
print('it\'s', "tab\there", r"raw\n", """multi
line""")
print("ab" "cd", "-" * 10, "a" + "b", "x" "y" * 2)
print(s.upper(), s.lower(), s.replace("l", "L", 2), s.find("o"), s.count("l"))
print(s.split(", "), "-".join(["a", "b", "c"]), "  pad ".strip() + "|")
print("abc".center(9, "*"), "7".zfill(3), "ab".ljust(4) + "|", "ab".rjust(4))
print("Hello" == "hello", "Hello".startswith("He"), "x=1".partition("="))
print(ord("A"), chr(97), "a,b;c".replace(";", ",").split(","))

$ py prog.py
H d World Hello Hlo ol dlroW ,olleH 12
it's tab     here raw\n multi
line
abcd ---------- ab xyxy
HELLO, WORLD hello, world HeLLo, World 4 3
['Hello', 'World'] a-b-c pad|
***abc*** 007 ab  |   ab
False True ('x', '=', '1')
65 a ['a', 'b', 'c']
```

Figure 20. String literals, indexing and slicing

String formatting

Three mechanisms are available and all follow CPython: f-strings, the format() method (and the format() built-in) and the % operator. Replacement fields take a conversion (!r !s !a) and a format specification `[[fill]align][sign][#][0][width][,][.precision][type]` where the type is one of d f F e E g G s x X o b c n or %. An f-string may contain nested quotes of the same kind and `{x=}` prints the expression as well as the value.

```
name, n, pi = "Ann", 42, 3.14159265
print(f"{name} has {n} items; pi={pi:.3f}")
print(f"[{name:>8}] [{name:<8}] [{name:^8}] [{name:*^9}]")
print(f"{n:5d}|{n:<5d}|{n:05d}|{n:+d}|{n:x}|{n:#x}|{n:b}|{n:o}|{n:#b}")
print(f"{1234567.891:,.2f} {0.256:.1%} {12345.6789:e} {1e10:g} {0.00001234:g}")
print(f"{name!r} {name!s} {'q'!r}")
print(f"{n * 2} {n > 40} {'yes' if n else 'no'} {[x * x for x in range(4)]}")
width, prec = 10, 3
print(f"[{pi:{width}.{prec}f}]")
print(f"{n=} {name=}")
print(f"{{literal}} {n}")

$ py prog.py
Ann has 42 items; pi=3.142
[     Ann] [Ann     ] [  Ann   ] [***Ann***]
   42|42   |00042|+42|2a|0x2a|101010|52|0b101010
1,234,567.89 25.6% 1.234568e+04 1e+10 1.234e-05
'Ann' Ann 'q'
84 True yes [0, 1, 4, 9]
[     3.142]
n=42 name='Ann'
{literal} 42
```

Figure 21. f-strings

```
print("{} is {}".format("sky", "blue"))
print("{1} {0} {1}".format("a", "b"))
print("{name:>6}|{val:08.3f}".format(name="x", val=3.14159))
print("{0[0]} {0[1]}".format([7, 8]))
print(format(255, "x"), format(3.5, "08.2f"), format("s", ">4"))
print("%d items, %s, %5.2f, %x, %5s|%-5s|" % (3, "ok", 2.5, 255, "ab", "ab"))
print("%(a)s-%(b)03d" % {"a": "x", "b": 7})
print("%c%c %% %r" % (72, "i", "q"))
print("%e %g %10.4f" % (12345.678, 0.0001, 3.14159265))

$ py prog.py
sky is blue
b a b
     x|0003.142
7 8
ff 00003.50    s
3 items, ok,  2.50, ff,    ab|ab   |
x-007
Hi % 'q'
1.234568e+04 0.0001     3.1416
```

Figure 22. The format() method and % formatting

Control flow

The compound statements are if/elif/else, while, for and try. Loops may have an else clause that runs when the loop was not left by break; break, continue and pass are available. The match statement (since Build 9) takes patterns that are literals, captures, the wildcard _, dotted names (compared with ==), sequences with one starred name, mappings with a **rest* name, class patterns with positional sub-patterns (through *__match_args__*, which dataclasses sets) and keyword sub-patterns, | alternatives and as bindings, each case with an optional guard: case ["go", ("n" | "s") as d] if ok: ... . The try statement accepts *except* handlers (see Chapter 9).

```
for i in range(1, 16):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 5 == 0:
        print("Buzz")
    elif i % 3 == 0:
        print("Fizz")
    else:
        print(i, end=" ")
print()

n = 27
steps = 0
while n != 1:
    n = n // 2 if n % 2 == 0 else 3 * n + 1
    steps += 1
print("collatz steps:", steps)

for p in range(2, 30):
    for d in range(2, p):
        if p % d == 0:
            break
    else:
        print(p, end=" ")
print()
for i, (a, b) in enumerate(zip("abc", [10, 20, 30]), start=1):
    print(i, a, b)

$ py prog.py
1 2 Fizz
4 Buzz
Fizz
7 8 Fizz
Buzz
11 Fizz
13 14 FizzBuzz

collatz steps: 111
2 3 5 7 11 13 17 19 23 29
1 a 10
2 b 20
3 c 30
```

Figure 23. if, while and for

Functions

A function is defined with def. Parameters may have defaults (evaluated once, when the def is executed), may collect extra positional arguments with *args and extra keywords with kwargs, and may be keyword-only when they follow a bare* or *args, and positional-only when they precede a bare /. Calls may unpack sequences with* and mappings with **. Functions are objects: they can be passed around, nested and given attributes. A lambda is an anonymous function with a single expression.

```
def greet(name, greeting="Hello", *, punct="!"):
    return greeting + ", " + name + punct

def total(*nums, **opts):
    s = sum(nums)
    return s * opts.get("scale", 1)

def show(a, b=2, *rest, c=3, **kw):
    print(a, b, rest, c, kw)

print(greet("Ann"), greet("Bob", "Hi", punct="?"))
print(total(1, 2, 3), total(1, 2, 3, scale=10))
show(1)
show(1, 2, 3, 4, c=5, d=6)
args = (10, 20)
kw = {"c": 99, "z": 0}
show(*args, **kw)
square = lambda x: x * x
print(list(map(square, range(5))), sorted(["bb", "a", "ccc"], key=len, reverse=True))
def f(): pass
f.calls = 0
f.calls += 1
print(f.calls)

$ py prog.py
Hello, Ann! Hi, Bob?
6 60
1 2 () 3 {}
1 2 (3, 4) 5 {'d': 6}
10 20 () 99 {'z': 0}
[0, 1, 4, 9, 16] ['ccc', 'bb', 'a']
1
```

Figure 24. Parameters and arguments

```
def counter(start=0):
    count = start
    def inc(step=1):
        nonlocal count
        count += step
        return count
    return inc

c = counter(10)
print(c(), c(), c(5))

total = 0
def add(n):
    global total
    total += n
add(4); add(5)
print(total)

adders = [(lambda x, i=i: x + i) for i in range(3)]
print([f(10) for f in adders])

def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print([fib(i) for i in range(15)])
def fact(n):
    return 1 if n <= 1 else n * fact(n - 1)
print(fact(30))

$ py prog.py
11 12 17
9
[10, 11, 12]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
265252859812191058636308480000000
```

Figure 25. Closures, nonlocal, global and recursion

```
import functools

def logged(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kw):
        print("calling", fn.__name__, args)
        return fn(*args, **kw)
    return wrapper

def repeat(times):
    def deco(fn):
        def wrapper(*a):
            return [fn(*a) for _ in range(times)]
        return wrapper
    return deco

@logged
def add(a, b):
    return a + b

@repeat(3)
def hi(name):
    return "hi " + name

print(add(2, 3))
print(hi("x"))

@functools.lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print(fib(80))

$ py prog.py
calling add (2, 3)
5
['hi x', 'hi x', 'hi x']
23416728348467685
```

Figure 26. Decorators

Classes

A class is defined with class and may inherit from one or more base classes. Methods take self as their first parameter; __init__ initialises new instances. Class attributes are shared, instance attributes live in the instance. super() follows the C3 method resolution order. The decorators @property (with .setter and .deleter), @staticmethod and @classmethod are supported. Special methods give objects the behaviour of the built-in types; the second table below lists them.

```
class Shape:
    count = 0
    def __init__(self, name):
        self.name = name
        Shape.count += 1
    def area(self):
        raise NotImplementedError
    def __str__(self):
        return f"{self.name} with area {self.area():.2f}"
    def __repr__(self):
        return f"Shape({self.name!r})"

class Circle(Shape):
    def __init__(self, r):
        super().__init__("circle")
        self.r = r
    def area(self):
        return 3.14159265 * self.r ** 2

class Rect(Shape):
    def __init__(self, w, h):
        super().__init__("rect")
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h
    @property
    def diag(self):
        return (self.w ** 2 + self.h ** 2) ** 0.5
    @staticmethod
    def unit():
        return Rect(1, 1)
    @classmethod
    def square(cls, s):
        return cls(s, s)
    def __eq__(self, o):
        return (self.w, self.h) == (o.w, o.h)
    def __lt__(self, o):
        return self.area() < o.area()

shapes = [Circle(1), Rect(2, 3), Rect.square(2), Rect.unit()]
for s in shapes:
    print(s)
print(repr(shapes[0]), Shape.count, Rect(3, 4).diag)
print(sorted(shapes[1:])[0], Rect(1, 1) == Rect.unit(), isinstance(shapes[1], Shape))
print(issubclass(Circle, Shape), type(shapes[0]).__name__, hasattr(shapes[1], "w"))

$ py prog.py
circle with area 3.14
rect with area 6.00
rect with area 4.00
rect with area 1.00
Shape('circle') 4 5.0
rect with area 1.00 True True
True Circle True
```

Figure 27. A class with inheritance, properties and special methods

```
class Vec:
    def __init__(self, *c):
        self.c = list(c)
    def __add__(self, o):
        return Vec(*[a + b for a, b in zip(self.c, o.c)])
    def __sub__(self, o):
        return Vec(*[a - b for a, b in zip(self.c, o.c)])
    def __mul__(self, k):
        return Vec(*[a * k for a in self.c])
    __rmul__ = __mul__
    def __neg__(self):
        return Vec(*[-a for a in self.c])
    def __len__(self):
        return len(self.c)
    def __getitem__(self, i):
        return self.c[i]
    def __setitem__(self, i, v):
        self.c[i] = v
    def __iter__(self):
        return iter(self.c)
    def __contains__(self, v):
        return v in self.c
    def __bool__(self):
        return any(self.c)
    def __eq__(self, o):
        return self.c == o.c
    def __hash__(self):
        return hash(tuple(self.c))
    def __call__(self, k):
        return self * k
    def __repr__(self):
        return "Vec(" + ", ".join(map(str, self.c)) + ")"

v, w = Vec(1, 2, 3), Vec(4, 5, 6)
print(v + w, v - w, v * 2, 3 * v, -v, len(v), v[1], 2 in v)
v[0] = 9
print(v, list(v), bool(Vec(0, 0)), v == Vec(9, 2, 3), v(2), {v: 1}[Vec(9, 2, 3)])

$ py prog.py
Vec(5, 7, 9) Vec(-3, -3, -3) Vec(2, 4, 6) Vec(3, 6, 9) Vec(-1, -2, -3) 3 2 True
Vec(9, 2, 3) [9, 2, 3] False True Vec(18, 4, 6) 1
```

Figure 28. Special methods: a vector class

```
class A:
    def who(self): return "A"
class B(A):
    def who(self): return "B>" + super().who()
class C(A):
    def who(self): return "C>" + super().who()
class D(B, C):
    def who(self): return "D>" + super().who()
print(D().who())
print([k.__name__ for k in D.__mro__])

class Temp:
    def __init__(self): self._c = 0
    @property
    def c(self): return self._c
    @c.setter
    def c(self, v):
        if v < -273: raise ValueError("below absolute zero")
        self._c = v
    @property
    def f(self): return self._c * 9 / 5 + 32
t = Temp(); t.c = 100
print(t.c, t.f)
try:
    t.c = -300
except ValueError as e:
    print("error:", e)

$ py prog.py
D>B>C>A
['D', 'B', 'C', 'A', 'object']
100 212.0
error: below absolute zero
```

Figure 29. Multiple inheritance and the method resolution order

Subclassing the built-in types and __new__

A class may derive from tuple, list, dict, set, str, int, float, bytes, bytearray, complex or any exception class, and may define __new__, which is called with the class as its first argument before __init__ and decides what object is created. Inside __new__ the base type's constructor is reached with super().__new__(cls, ...) or explicitly with tuple.__new__(cls, items), float.__new__(cls, v), int.__new__(cls, v), str.__new__(cls, s) and object.__new__(cls). An instance of such a class is a real list, dict, etc. that also has an instance dictionary and the methods of the subclass. The operations of the interpreter honour the overrides: len(), iteration, in, x[i] with an overridden __getitem__, repr(), str(), hash() and the comparison and arithmetic operators look for the method of the subclass first; calling the base implementation explicitly (super().append(v), list.append(self, v), dict.__setitem__(self, k, v)) runs the built-in code. For a dict subclass, d[k] calls __missing__ when the key is absent. collections.namedtuple creates real subclasses of tuple, so isinstance(p, tuple) is true.

```
class Point(tuple):
    def __new__(cls, x, y):
        return super().__new__(cls, (x, y))
    @property
    def x(self): return self[0]
    @property
    def y(self): return self[1]
    def norm(self): return (self.x ** 2 + self.y ** 2) ** 0.5

p = Point(3, 4)
print(p, p.norm(), isinstance(p, tuple), len(p), p == (3, 4), tuple(p), p + (5,))

class Stack(list):
    def push(self, v):
        self.append(v)
    def append(self, v):
        print("append", v)
        super().append(v)
    def peek(self):
        return self[-1]

s = Stack([1, 2])
s.push(3)
print(s, s.peek(), len(s), isinstance(s, list), sorted(s, reverse=True),
    type(s).__name__)

class Counter(dict):
    def __missing__(self, key):
        return 0
    def __setitem__(self, key, value):
        print("set", key, value)
        dict.__setitem__(self, key, value)

c = Counter()
c["a"] += 1
c["a"] += 1
print(c, c["zz"], len(c), isinstance(c, dict))

class Celsius(float):
    def __new__(cls, v):
        return float.__new__(cls, v)
    def __repr__(self):
        return "Celsius(%s)" % float.__repr__(self)
    def fahrenheit(self):
        return self * 9 / 5 + 32

t = Celsius(100)
print(repr(t), t + 1, t.fahrenheit(), isinstance(t, float))

class Name(str):
    def shout(self):
        return self.upper() + "!"

n = Name("bob")
print(n, n.shout(), n + "x", len(n), n == "bob", {n: 1}["bob"])

class Even(int):
    def __new__(cls, v):
        if v % 2:
            raise ValueError("%d is odd" % v)
```

```
        return int.__new__(cls, v)

print(Even(4) + 1, isinstance(Even(2), int))
try:
    Even(3)
except ValueError as e:
    print("ValueError:", e)

$ py prog.py
(3, 4) 5.0 True 2 True (3, 4) (3, 4, 5)
append 3
[1, 2, 3] 3 3 True [3, 2, 1] Stack
set a 1
set a 2
{'a': 2} 0 1 True
Celsius(100.0) 101.0 212.0 True
bob BOB! bobx 3 True 1
5 True
ValueError: 3 is odd
```

Figure 30. Subclasses of built-in types and __new__

```
class Singleton:
    _inst = None
    def __new__(cls, *args):
        if cls._inst is None:
            cls._inst = super().__new__(cls)
        return cls._inst
    def __init__(self, v=0):
        self.v = v

a, b = Singleton(1), Singleton(2)
print(a is b, a.v)

class AppError(Exception):
    def __init__(self, msg, code):
        super().__init__(msg)
        self.code = code

try:
    raise AppError("disk full", 28)
except Exception as e:
    print(type(e).__name__, e, e.code, e.args)

import collections
Pt = collections.namedtuple("Pt", "a b")
q = Pt(1, 2)
print(q, isinstance(q, tuple), q._asdict(), q._replace(a=9), Pt._fields)

$ py prog.py
True 2
AppError disk full 28 ('disk full',)
Pt(a=1, b=2) True {'a': 1, 'b': 2} Pt(a=9, b=2) ('a', 'b')
```

Figure 31. __new__ without a built-in base: a singleton

Limits: in the CREXX interpreter metaclass __call__ is ignored; __slots__ is accepted and has no effect; OSError(2, "x") does not turn into FileNotFoundError (Chapter 12).

Method	Used for
__init__, __new__	creating instances; calling a class runs __new__ and then __init__ when __new__ returned an instance of the class (see Subclassing the built-in types, below)

Method	Used for
__str__, __repr__, __format__	str(), print, repr(), format() and f-strings
__eq__, __ne__, __lt__, __le__, __gt__, __ge__	comparisons (NotImplemented falls through to the other operand)
__hash__	hash() and use as dict key or set member
__bool__, __len__	truth value
__add__, __sub__, __mul__, __truediv__, __floordiv__, __mod__, __pow__, __matmul__, __and__, __or__, __xor__, __lshift__, __rshift__	binary operators; the reflected forms __radd__ ... and the in-place forms __iadd__ ... are also used
__neg__, __pos__, __invert__, __abs__, __round__	unary operators and abs(), round()
__getitem__, __setitem__, __delitem__	subscription, including slice objects
__contains__, __iter__, __next__	in, for, iter(), next()
__call__	calling an instance
__enter__, __exit__	the with statement
__getattr__, __setattr__, __delattr__	attribute access hooks
__int__, __float__, __index__	int(), float(), indexing and range()
__missing__	d[k] on a dict subclass when k is absent
__class_getitem__, __init_subclass__, __set_name__	accepted by the library code that needs them (see Chapter 12 for limits)

Figure 32. Special methods supported

Exceptions

An exception is raised with raise and handled with try. A try statement may have several except clauses (each naming a class, a tuple of classes or nothing), an else clause that runs when no exception occurred, and a finally clause that always runs. Exceptions are instances of classes derived from BaseException; programs define their own by subclassing Exception. raise ... from ... is accepted (the cause is recorded but not printed). The hierarchy is described in Chapter 9.

```
class InsufficientFunds(Exception):
    def __init__(self, need, have):
        super().__init__(f"need {need}, have {have}")
        self.need, self.have = need, have

def withdraw(bal, amt):
    if amt > bal:
        raise InsufficientFunds(amt, bal)
    return bal - amt

for amt in (30, 500):
    try:
        print("left", withdraw(100, amt))
    except InsufficientFunds as e:
        print("failed:", e, e.need - e.have)
    else:
        print("ok")
    finally:
        print("done", amt)

for bad in ("12", "x", None, [1]):
    try:
        print(int(bad))
    except (ValueError, TypeError) as e:
        print(type(e).__name__, "-", e)

try:
    {}["missing"]
except KeyError as e:
    print("KeyError", e)
try:
    [1, 2][5]
except LookupError as e:
    print("LookupError", e)
try:
    1 / 0
except ArithmeticError as e:
    print(repr(e))
def f():
    try:
        return "try"
    finally:
        print("cleanup")
print(f())
try:
    try:
        raise ValueError("inner")
    except ValueError as e:
        raise RuntimeError("outer") from e
except RuntimeError as e:
    print(e)
assert 1 + 1 == 2, "math works"
try:
    assert 1 > 2, "one is not greater"
except AssertionError as e:
    print("AssertionError:", e)

$ py prog.py
left 70
ok
```

```
done 30
failed: need 500, have 100 400
done 500
12
ValueError - invalid literal for int() with base 10: 'x'
TypeError - int() argument must be a string, a bytes-like object or a real number, not
    'NoneType'
TypeError - int() argument must be a string, a bytes-like object or a real number, not
    'list'
KeyError 'missing'
LookupError list index out of range
ZeroDivisionError('division by zero')
cleanup
try
outer
AssertionError: one is not greater
```

Figure 33. try, except, else, finally

Iterators and generators

Any object with __iter__ (returning an object with __next__) is an iterator source for for loops, unpacking, sum(), list() and so on. A function containing yield is a generator function: calling it returns a generator that runs up to each yield on demand. Generators support next(), send(), throw(), close(), yield from and return values (StopIteration.value). Generator expressions are lazy.

```
def count_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

def take(n, it):
    for _ in range(n):
        yield next(it)

def chain(*its):
    for it in its:
        yield from it

def accumulator():
    total = 0
    while True:
        x = yield total
        if x is None:
            return
        total += x

print(list(count_to(5)), list(take(10, fib())))
print(list(chain([1, 2], "ab", range(3))))
g = accumulator()
next(g)
print(g.send(10), g.send(5), g.send(100))
squares = (x * x for x in range(10))
print(sum(squares), next(iter(range(3))))
it = iter([10, 20])
print(next(it), next(it), next(it, "end"))

class Countdown:
    def __init__(self, n): self.n = n
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1
print(list(Countdown(4)), max(Countdown(3)))

$ py prog.py
[1, 2, 3, 4, 5] [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[1, 2, 'a', 'b', 0, 1, 2]
10 15 115
285 0
10 20 end
[4, 3, 2, 1] 3
```

Figure 34. Generators

Comprehensions

```
print([x * x for x in range(10) if x % 2 == 0])
print([(x, y) for x in range(3) for y in range(3) if x < y])
print({w: len(w) for w in ["apple", "fig", "kiwi"]})
print({c for c in "mississippi"} == set("misp"), sorted({x % 3 for x in range(10)}))
matrix = [[1, 2, 3], [4, 5, 6]]
print([[row[i] for row in matrix] for i in range(3)])
print([n for n in range(50) if all(n % d for d in range(2, n)) and n > 1])
print(sum(x for x in range(101)), any(c.isdigit() for c in "ab3"), all([]))
print(dict(zip("abc", range(3))), list(map(lambda p: p[0] + p[1], zip([1, 2], [3, 4]))))
print([y for x in [[1, 2], [3]] for y in x])

$ py prog.py
[0, 4, 16, 36, 64]
[(0, 1), (0, 2), (1, 2)]
{'apple': 5, 'fig': 3, 'kiwi': 4}
True [0, 1, 2]
[[1, 4], [2, 5], [3, 6]]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
5050 True True
{'a': 0, 'b': 1, 'c': 2} [4, 6]
[1, 2, 3]
```

Figure 35. List, dict and set comprehensions

Modules and import

import runs a module once and binds its namespace. The forms import m, import m as n, import a.b.c, from m import x, y as z and from m import * (which honours __all__) are supported, as are packages (a directory with __init__.py) and relative imports (from . import m, from .m import f, from .. import n). Imported modules are cached in sys.modules, __name__ is '__main__' in the script and the module name elsewhere, and circular imports are tolerated as in CPython. Modules are searched for in the directory of the script, in the directories named by the environment variable PYLIB (separated by colons) and in the library directory pylib. Native modules take precedence over files of the same name.

```
# main.py
import shapes
from shapes import circle
from shapes.square import area as sq_area
print(shapes.__name__, shapes.VERSION)
print(circle.area(2), sq_area(3))
print(circle.__name__, __name__)

# shapes/__init__.py
VERSION = "1.0"
__all__ = ["circle", "square"]

# shapes/circle.py
from . import VERSION
def area(r):
    return round(3.14159 * r * r, 2)

# shapes/square.py
from .circle import area as _unused
def area(s):
    return s * s

$ py main.py
shapes 1.0
12.57 9
shapes.circle __main__
```

Figure 36. A package with relative imports

The with statement

with calls __enter__ on the context manager and guarantees that __exit__ runs, even if the block raises; an __exit__ that returns a true value suppresses the exception. Several managers may be listed, separated by commas. Files, io.StringIO objects, contextlib.contextmanager generators, contextlib.suppress and user classes can all be used.

```
import contextlib

class Section:
    def __init__(self, name): self.name = name
    def __enter__(self):
        print("enter", self.name)
        return self
    def __exit__(self, et, ev, tb):
        print("exit", self.name, et.__name__ if et else None)
        return et is KeyError

with Section("a") as s, Section("b"):
    print("body", s.name)
with Section("c"):
    raise KeyError("swallowed")
print("after")

@contextlib.contextmanager
def tag(name):
    print("<" + name + ">")
    try:
        yield name.upper()
    finally:
        print("</" + name + ">")

with tag("b") as t:
    print("inside", t)
with contextlib.suppress(ZeroDivisionError):
    1 / 0
print("done")

$ py prog.py
enter a
enter b
body a
exit b None
exit a None
enter c
exit c KeyError
after
<b>
inside B
</b>
done
```

Figure 37. Context managers

Other statements

```
lst = [1, 2, 3, 4, 5]
del lst[0]
del lst[1:3]
d = {"a": 1, "b": 2}
del d["a"]
x = 10
del x
print(lst, d, "x" in globals())
print(eval("2 + 3 * 4"), eval("a * 2", {"a": 21}))
ns = {}
exec("def sq(n):\n    return n * n\nr = sq(7)", ns)
print(ns["r"], callable(ns["sq"]))
print(hasattr(__import__("math"), "isqrt"), hasattr(__import__("math"), "nothing"))
def f(a, b=1):
    z = a + b
    print(sorted(locals()))
f(1)
print(type(None).__name__, type(1).__name__, type(1.5).__name__, type("").__name__,
      type([]).__name__, type({}).__name__, type(()).__name__, type(f).__name__)

$ py prog.py
[2, 5] {'b': 2} False
14 42
49 True
True False
['a', 'b', 'z']
NoneType int float str list dict tuple function
```

Figure 38. del, assert, global, eval, exec and the namespaces

```
import sys
print(sys.argv)
with open("/tmp/py_manual_demo.txt", "w") as f:
    f.write("one\ntwo\n")
    print("three", file=f)
with open("/tmp/py_manual_demo.txt") as f:
    for line in f:
        print(repr(line))
with open("/tmp/py_manual_demo.txt") as f:
    print(f.read().split())
print(len(sys.argv) - 1, "arguments")
sys.stdout.write("no newline")
sys.stdout.write("\n")
print("to stderr", file=sys.stderr)

$ py prog.py a b
['prog.py', 'a', 'b']
'one\n'
'two\n'
'three\n'
['one', 'two', 'three']
2 arguments
no newline
```

Figure 39. Files, standard input and the command line

Chapter 4. Built-in Functions and Types

This chapter is the reference for the built-in functions, constants and types that are available without an import. A superscript-free convention is used in the tables: **C** after an entry means the entry exists only in the C interpreter and **X** means only in the CREXX interpreter; entries without a mark exist in both.

Built-in functions

Function	Description
abs(x)	absolute value of an int, float or object with __abs__
all(it) any(it)	true if all / any element of the iterable is true
ascii(x) X	repr() with non-ASCII characters escaped (X: returns repr)
bytes(...) bytearray(...)	immutable and mutable byte strings; bytes(3), bytes([72, 105]), bytes("é", "utf-8"), bytearray(b"ab") (see below)
bin(n) hex(n) oct(n)	integer to string in base 2, 16, 8 with the 0b 0x 0o prefix
bool(x)	truth value
callable(x)	true if x can be called
complex([real[,imag]])	complex number; also complex("1+2j"). See below and Chapter 3
chr(n) ord(c)	character of code n; code of character c
classmethod(f) staticmethod(f) property(...)	descriptors, normally used as decorators
delattr(o,n) getattr(o,n[,d]) hasattr(o,n) setattr(o,n,v)	attribute access by name
dir([o]) vars([o]) globals() locals()	names of an object or scope; globals() and locals() return snapshot dictionaries
divmod(a,b)	pair (a // b, a % b)
enumerate(it[,start])	pairs (index, item)
eval(src[,g[,l]]) exec(src[,g[,l]])	evaluate an expression / execute statements from a string
exit([code]) quit([code])	raise SystemExit
filter(f,it) map(f,it,...) zip(it,...)	lazy iterators; filter(None, it) keeps true items
format(x[,spec])	format with the format mini-language
hash(x)	hash value (CPython's value for int, bool, float, complex, None, tuple of those and frozenset in both interpreters; not equal to CPython's for str, which changes in every CPython run; TypeError for set, list, dict, bytearray)
id(x)	identity of an object
input([prompt])	read a line from standard input; EOFError at end of file
isinstance(x,c) issubclass(a,c)	c may be a class or a tuple of classes
iter(x) next(it[,default])	iterator protocol (the two-argument form iter(f, sentinel) is not available in CREXX)
frozenset(it)	immutable, hashable set; a separate type from set
len(x)	number of items (in C, bytes of a string)
max(it) min(it)	also max(a, b, ...), key= and default=
open(path[,mode])	text files only: modes r, w and a (a b in the mode does not give bytes); methods read([n]) readline readlines write writelines flush close, iteration and with. There is no seek, tell, name or mode on files
pow(x,y[,m])	x ** y, or x ** y % m computed efficiently
print(*a, sep=" ", end="\n", file=None, flush=False)	write to standard output or to file
range(stop) range(start,stop[,step])	arithmetic sequence (C: no .start .count .index)
repr(x)	the printable representation
reversed(seq)	reverse iterator; uses __reversed__ or __len__ and __getitem__
round(x[,n])	round half to even; returns int when n is omitted
slice(a,b,c)	slice object
sorted(it, key=None, reverse=False)	new sorted list; the sort is stable
sum(it[,start])	sum; floats are added with compensated summation as in CPython 3.12
super()	proxy to the next class in the MRO (zero-argument form)
type(x) type(name,bases,dict)	type of an object; or create a class

Function	Description
__import__(name)	import a module by name

Figure 40. Built-in functions

Not available: compile, memoryview, aiter, anext, breakpoint, help. The constants True, False, None, Ellipsis (...) and NotImplemented exist.

```
words = ["pear", "fig", "apple", "kiwi"]
print(sorted(words), sorted(words, key=len), max(words, key=len), min(words))
print(list(zip(words, range(10))), dict(enumerate(words)))
print(list(map(len, words)), list(filter(lambda w: "i" in w, words)))
print(sum([0.1] * 10), sum(range(101)), sum([[1], [2]], []))
print(any(w.startswith("a") for w in words), all(len(w) > 2 for w in words))
print(round(2.5), round(3.5), round(-2.5), round(2.567, 2), round(1250, -2))
print(divmod(17, 5), divmod(-17, 5), pow(3, 4), pow(3, 4, 5), pow(2, -1), abs(-2.5))
print(bin(10), hex(255), oct(64), int("ff", 16), int("101", 2), chr(65), ord("a"))
print(isinstance(1, int), isinstance(True, int), isinstance("s", (int, float)))
print(list(reversed(range(5))), list(range(10, 0, -3)), len(range(0, 100, 7)))
print(min([], default="none"), max(3, 9, 4), min("b", "a"))
print(repr("a'b"), repr('a"b'), repr(1.0), repr(None), str(b"x" if False else 3))
print(getattr(words, "count")("fig"), hasattr(words, "nope"), getattr(words, "nope", 0))
print(type(3) is int, type("x") == str, callable(len), callable(3))
print(id(words) == id(words), hash("a") == hash("a"), hash((1, 2)) == hash((1, 2)))

$ py prog.py
['apple', 'fig', 'kiwi', 'pear'] ['fig', 'pear', 'kiwi', 'apple'] apple apple
[('pear', 0), ('fig', 1), ('apple', 2), ('kiwi', 3)] {0: 'pear', 1: 'fig', 2: 'apple',
    3: 'kiwi'}
[4, 3, 5, 4] ['fig', 'kiwi']
1.0 5050 [1, 2]
True True
2 4 -2 2.57 1200
(3, 2) (-4, 3) 81 1 0.5 2.5
0b1010 0xff 0o100 255 5 A 97
True True False
[4, 3, 2, 1, 0] [10, 7, 4, 1] 15
none 9 a
"a'b" 'a"b' 1.0 None 3
1 False 0
True True True False
True True True
```

Figure 41. Built-in functions at work

Methods of str

Method	Description
capitalize() title() swapcase() casefold()	case conversion (ASCII and Latin-1 letters in C; Unicode in CREXX)
lower() upper()	case conversion
center(w[,fill]) ljust(w[,fill]) rjust(w[,fill]) zfill(w)	padding
count(s[,a[,b]])	non-overlapping occurrences
encode([encoding[,errors]])	bytes: utf-8 (the default), ascii and latin-1; errors strict, replace, ignore, backslashreplace
startswith(p) endswith(s)	prefix or suffix test; p may be a tuple
find(s) rfind(s) index(s) rindex(s)	position of a substring; find returns -1, index raises ValueError
format(*a, **k)	replacement-field formatting; format_map() is not available
isalnum() isalpha() isdigit() isdecimal() isnumeric() isidentifier() isspace() islower() isupper() istitle() isascii()	character-class tests
isprintable() C	character-class test (not available in CREXX)
join(it)	concatenate the strings of an iterable with this string between them

Method	Description
lstrip([chars]) rstrip([chars]) strip([chars])	remove leading / trailing characters
partition(sep) rpartition(sep)	split into (head, sep, tail)
removeprefix(p) removesuffix(s)	remove a prefix or suffix if present
replace(old,new[,count])	replace occurrences
split([sep[,max]]) rsplit([sep[,max]]) splitlines([keep])	split into a list
expandtabs([n])	replace tabs by spaces
translate(table) X	map characters by a dict of ordinals (not available in C; str.maketrans does not exist in either)

Figure 42. str methods

```
s = "  The quick brown fox  "
print(repr(s.strip()), repr(s.lstrip()), repr(s.rstrip()))
t = s.strip()
print(t.split(), t.split("o"), t.rsplit(" ", 1), t.partition(" "), t.rpartition(" "))
print(t.upper(), t.lower(), t.title(), t.swapcase(), t.capitalize())
print(t.find("quick"), t.find("slow"), t.index("brown"), t.rfind("o"), t.count("o"))
print(t.startswith(("The", "A")), t.endswith("fox"), "abc".isalpha(), "12".isdigit())
print(",".join(str(i) for i in range(5)), "x".join("abc"), "ab" * 3)
print("a\nb\n".splitlines(), "a\nb".splitlines(True), "one  two".split(" "))
print("%-8s|%8s|" % ("left", "right"), "name".center(10, "."), "5".zfill(4))
print("Hello".removeprefix("He"), "Hello".removesuffix("lo"), "tab\there".expandtabs(4))
print("a-b_c".replace("-", " ").replace("_", " ").split(), "xxxyx".strip("x"))
print(str(12), str(1.5), str(True), str(None), str([1, "a"]), str((1,)), str({"k":
    "v"}))

$ py prog.py
'The quick brown fox' 'The quick brown fox  ' '  The quick brown fox'
['The', 'quick', 'brown', 'fox'] ['The quick br', 'wn f', 'x'] ['The quick brown',
    'fox'] ('The', ' ', 'quick brown fox') ('The quick brown', ' ', 'fox')
THE QUICK BROWN FOX the quick brown fox The Quick Brown Fox tHE QUICK BROWN FOX The
    quick brown fox
4 -1 10 17 2
True True True True
0,1,2,3,4 axbxc ababab
['a', 'b'] ['a\n', 'b'] ['one', '', 'two']
left    |   right| ...name... 0005
llo Hel tab here
['a', 'b', 'c'] y
12 1.5 True None [1, 'a'] (1,) {'k': 'v'}
```

Figure 43. String methods

Methods of list and tuple

Method	Description
list.append(x)	add x at the end
list.extend(it)	append every item of the iterable
list.insert(i,x)	insert x before position i
list.remove(x)	remove the first x; ValueError if absent
list.pop([i])	remove and return item i (default last)
list.clear() list.copy()	empty the list; shallow copy
list.index(x[,a[,b]]) list.count(x)	position of the first x; number of x
list.sort(key=None, reverse=False)	sort in place, stable
list.reverse()	reverse in place
tuple.count(x) tuple.index(x)	as for list
+ * in == < [] [a:b:c] len min max sum del l[i]	operators and built-ins that work on both; slices may be assigned and deleted for lists

Figure 44. list and tuple methods

```
a = [5, 3, 8, 1]
a.append(9); a.extend([2, 7]); a.insert(0, 4)
print(a, a.index(8), a.count(3), len(a))
print(a.pop(), a.pop(0), a)
a.remove(8); a.sort(); print(a)
a.sort(reverse=True); print(a, a[::2], a[-2:], a[1:3])
a[1:3] = [0, 0, 0]; print(a)
del a[::2]; print(a)
b = a.copy(); b.reverse(); print(a, b, a is b, a == b[::-1])
m = [[0] * 3 for _ in range(2)]
m[0][1] = 5
print(m, [1, 2] + [3], [0] * 4)
t = (1, 2, 3)
print(t[1:], t + (4,), t * 2, t.index(3), t.count(2), (1,), ())
x, y, z = t
print(x + y + z, min(t), max(t), sum(t), tuple([1, 2]), list((1, 2)))
words = ["banana", "Apple", "cherry"]
words.sort(key=lambda w: w.lower()); print(words)
print(sorted([(2, "b"), (1, "z"), (2, "a")]))

$ py prog.py
[4, 5, 3, 8, 1, 9, 2, 7] 3 1 8
7 4 [5, 3, 8, 1, 9, 2]
[1, 2, 3, 5, 9]
[9, 5, 3, 2, 1] [9, 3, 1] [2, 1] [5, 3]
[9, 0, 0, 0, 2, 1]
[0, 0, 1]
[0, 0, 1] [1, 0, 0] False True
[[0, 5, 0], [0, 0, 0]] [1, 2, 3] [0, 0, 0, 0]
(2, 3) (1, 2, 3, 4) (1, 2, 3, 1, 2, 3) 2 1 (1,) ()
6 1 3 6 (1, 2) [1, 2]
['Apple', 'banana', 'cherry']
[(1, 'z'), (2, 'a'), (2, 'b')]
```

Figure 45. Lists and tuples

Methods of dict and set

Method	Description
d[k] d[k] = v del d[k] k in d len(d)	item access; KeyError for a missing key
get(k[,default])	value or default (None)
setdefault(k[,default])	return d[k]; insert default if missing
pop(k[,default]) popitem()	remove a key; remove and return the last pair
update(other) update(**kw)	merge pairs from a dict, an iterable of pairs or keywords
keys() values() items()	live views, ordered by insertion; support len, in, iteration
copy() clear()	shallow copy; remove all
dict.fromkeys(it[,v])	new dict with the given keys
d | e d |= e	merge (right operand wins)

Figure 46. dict methods

Method	Description
add(x) discard(x) remove(x) pop()	add; remove if present; remove (KeyError if absent); remove an arbitrary item
clear() copy()	empty; shallow copy
union(...) | intersection(...) & difference(...) - symmetric_difference(s) ^	new set
update(...)	add the items of the arguments (also |=, &=, -=, ^= on sets)
difference_update(...) intersection_update(...) symmetric_difference_update(s)	in place (available in both interpreters)
issubset(s) <= issuperset(s) >= isdisjoint(s)	relations; also < and >

Figure 47. set methods (frozenset has all but the mutating ones)

```
d = {"b": 2, "a": 1}
d["c"] = 3
print(d, d["a"], d.get("z"), d.get("z", 0), "a" in d, len(d))
print(list(d), list(d.values()), list(d.items()))
d.update({"a": 10, "d": 4}); d.update(e=5)
print(d, d.pop("b"), d.pop("zz", None), d.popitem())
print(d.setdefault("x", []), d.setdefault("a", 99), d)
print(dict.fromkeys("abc", 0), dict([("k", 1), ("j", 2)]), dict(a=1))
inv = {v: k for k, v in {"a": 1, "b": 2}.items()}
print(inv, sorted(inv), {**{"a": 1}, **{"b": 2}})
counts = {}
for ch in "mississippi":
    counts[ch] = counts.get(ch, 0) + 1
print(counts, sorted(counts.items(), key=lambda kv: -kv[1]))
for k, v in counts.items():
    if v > 3: print(k, v)
keys = d.keys()
d["new"] = 0
print(len(keys), "new" in keys)

s = {3, 1, 2}
s.add(5); s.discard(1); s.discard(42)
print(sorted(s), 2 in s, len(s), sorted(s | {9}), sorted(s & {2, 3, 7}), sorted(s -
    {2}))
print({1, 2} <= {1, 2, 3}, {1, 2}.isdisjoint({3}), sorted({1,
    2}.symmetric_difference({2, 3})))
print(sorted(set("hello")), set(), len(set([1, 1, 2])), sorted({1, 2}.union([5], [6])))

$ py prog.py
{'b': 2, 'a': 1, 'c': 3} 1 None 0 True 3
['b', 'a', 'c'] [2, 1, 3] [('b', 2), ('a', 1), ('c', 3)]
{'a': 10, 'c': 3, 'd': 4} 2 None ('e', 5)
[] 10 {'a': 10, 'c': 3, 'd': 4, 'x': []}
{'a': 0, 'b': 0, 'c': 0} {'k': 1, 'j': 2} {'a': 1}
{1: 'a', 2: 'b'} [1, 2] {'a': 1, 'b': 2}
{'m': 1, 'i': 4, 's': 4, 'p': 2} [('i', 4), ('s', 4), ('p', 2), ('m', 1)]
i 4
s 4
5 True
[2, 3, 5] True 3 [2, 3, 5, 9] [2, 3] [3, 5]
True True [1, 3]
['e', 'h', 'l', 'o'] set() 2 [1, 2, 5, 6]
```

Figure 48. Dictionaries and sets

Methods of int, float and bool

Name	Description
int.bit_length()	number of bits needed
int.to_bytes(n, order, signed=False) int.from_bytes(b, order, signed=False)	conversion to and from bytes of any length
int.is_integer() X	true (CREXX only)
int.real .imag .numerator .denominator .conjugate()	present for compatibility
complex.real .imag .conjugate()	the parts as floats; the conjugate
float.is_integer()	true if the value is integral
float.real .imag .conjugate()	present for compatibility
float(x)	accepts "inf", "-inf", "nan", exponents and surrounding blanks (not underscores)

Name	Description
int(x[,base])	accepts a string in base 2..36, base 0 for prefixes; truncates floats toward zero
bool	subclass of int: True + 1 == 2

Figure 49. Numeric methods and attributes

complex, bytes and bytearray

complex is a native type in both interpreters: two doubles, with the literals 3j, 1.5 + 2j and complex(a, b), arithmetic that agrees with CPython to the last digit, abs(), pow(), sum(), ==, hash, format specifications and the repr and str forms of CPython ((1+2j), 1j, (-0-1j)). Chapter 3 shows the operators; Chapter 5 describes cmath, which holds the functions.

bytes (immutable) and **bytearray** (mutable) hold sequences of integers from 0 to 255. Literals are b"..." and rb"..." (adjacent literals join, \xHH escapes), indexing gives an int, slices give bytes, and + * in < == hash and iteration work as for str. bytes(n) is n zero bytes, bytes(iterable) takes integers, bytes(s, encoding) encodes a str, and str(b, encoding) or b.decode(encoding, errors) goes back; an invalid sequence raises UnicodeDecodeError with the text CPython prints. open() still reads and writes text only, so bytes are not read from files in either interpreter.

Method	Description
decode([encoding[,errors]]) hex() bytes.fromhex(s)	conversion to str and hexadecimal text and back
find rfind index rindex count startswith endswith replace	search and replace; arguments are bytes
split rsplit splitlines partition rpartition join strip lstrip rstrip	splitting, joining and trimming; the separator and the pieces are bytes
upper lower title capitalize swapcase zfill center ljust rjust expandtabs	case conversion and padding
isalpha isdigit isalnum isspace islower isupper istitle	tests (isascii is missing in the C interpreter)
translate maketrans removeprefix removesuffix	as for str
bytearray: append extend insert pop remove clear copy reverse	mutation; item and slice assignment and deletion (ba[0] = 65, ba[1:3] = b"xy", del ba[0])
int.to_bytes int.from_bytes	integers to and from bytes, big or little endian, signed or unsigned

Figure 50. bytes and bytearray methods

```
b = b"hello"
print(b, b[0], b[1:3], b + b" world", b * 2, b"ell" in b, len(b), b.upper(), b.hex())
print(b.decode(), "héllo".encode(), "héllo".encode("latin-1"),
    b"\xc3\xa9".decode("utf-8"))
print(bytes([72, 105]), bytes(3), bytes.fromhex("48 69"), list(b"abc"), b.find(b"l"),
    b.split(b"l"))
print(b"a,b,c".split(b","), b",".join([b"x", b"y"]), b.startswith(b"he"), b == b"hello",
    b < b"help")
ba = bytearray(b"abc")
ba.append(100)
ba[0] = 65
ba.extend(b"ef")
print(ba, len(ba), bytes(ba))
last = ba.pop()
print(last, ba)
print(int.from_bytes(b"\x01\x00", "big"), (258).to_bytes(2, "little"), (-2).to_bytes(2,
    "big", signed=True))
try:
    b"\xff".decode("utf-8")
except UnicodeDecodeError as e:
    print("UnicodeDecodeError:", e)
print(b"\xff".decode("utf-8", "replace") == "\ufffd", repr(b"it's \"q\"\n\x00"),
    repr(bytearray(b"x")))

$ py prog.py
b'hello' 104 b'el' b'hello world' b'hellohello' True 5 b'HELLO' 68656c6c6f
hello b'h\xc3\xa9llo' b'h\xe9llo' é
b'Hi' b'\x00\x00\x00' b'Hi' [97, 98, 99] 2 [b'he', b'', b'o']
[b'a', b'b', b'c'] b'x,y' True True True
bytearray(b'Abcdef') 6 b'Abcdef'
102 bytearray(b'Abcde')
256 b'\x02\x01' b'\xff\xfe'
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff in position 0: invalid start
    byte
True b'it\'s "q"\n\x00' bytearray(b'x')
```

Figure 51. bytes, bytearray and struct

Not available: memoryview, the array module, the % operator on bytes (b"%d" % 5), bytes patterns in re, and the half-float struct format e.

Chapter 5. The Standard Library

The standard library is small. Five modules are implemented natively by both interpreters (math, random, sys, os and time, besides numpy, see Chapter 6). The C interpreter has five more native modules: re, json, cmath, struct, and socket with select. The CREXX interpreter has native typing, warnings, abc and __future__ modules, and a few modules of its own (socket and select on top of the rxsocket library, in crexx/pylib). Everything else is ordinary Python source in the directory pylib, which both interpreters load through the normal import mechanism, and a module that is native in one interpreter is the Python file from pylib in the other (re, json, cmath, struct), so a program sees the same module in both. Because the library files are written in the subset of Python that the interpreters support, they run unchanged on both, and a program can read them to see exactly what a function does. Appendix C is the module index and says, for every module, which interpreter implements it natively.

Module	C interpreter	CREXX interpreter
re	native: tokenizer and parser ported from CPython, compiler, backtracking VM (src/p7_re.c)	Python (pylib/re.py), a slower engine with fewer features
json	native (src/p7_json.c)	Python (pylib/json.py)
cmath	native (src/p7_cplx.c)	Python (pylib/cmath.py, after CPython's cmathmodule.c)
struct	native (src/p7_bytes.c)	Python (pylib/struct.py)
socket, select	native, IPv4 TCP; sockets only in a build with -DPY_POSIX	Python (crexx/pylib) on rxsocket, IPv4 TCP
numpy.linalg, numpy.random	native	Python (pylib/numpy)
numpy.fft, scipy.fft	Python (pylib), on the native complex type	Python (pylib), on the _CArr class of pylib/numpy/_cplx.py

Figure 52. Native or Python: the modules whose implementation differs

In the tables below, an entry marked **C** is available only in the C interpreter and an entry marked **X** only in the CREXX interpreter.

math

Native in both interpreters. In the CREXX interpreter the functions that the native module lacks are taken from pylib/math.py.

Name	Description
pi e tau inf nan	constants
sqrt cbrt exp exp2 expm1 log log2 log10 log1p pow	roots, exponentials and logarithms; log(x[,base])
sin cos tan asin acos atan atan2 sinh cosh tanh asinh acosh atanh hypot dist degrees radians	trigonometry; hypot and dist take any number of coordinates
ceil floor trunc fabs copysign fmod modf frexp ldexp remainder	rounding and floating-point manipulation
factorial comb perm gcd lcm isqrt prod fsum	integer functions (exact for big integers) and exact summation
isnan isinf isfinite isclose	tests
erf erfc gamma lgamma	special functions, within about 1e-15 of the C library
fma nextafter sumprod ulp X	available in CREXX only

Figure 53. math

```
import math
print(math.sqrt(2), math.pi, math.e, math.inf, -math.inf, math.nan)
print(math.floor(-2.5), math.ceil(-2.5), math.trunc(-2.5), math.fabs(-3))
print(math.log(8, 2), math.log10(1000), math.exp(1), math.hypot(3, 4))
print(math.factorial(20), math.comb(10, 3), math.perm(5, 2), math.gcd(84, 36),
    math.lcm(4, 6))
print(math.isclose(0.1 + 0.2, 0.3), math.isnan(math.nan), math.fsum([1, 2, 3]))
print(round(math.degrees(math.pi), 6), round(math.sin(math.radians(30)), 10),
    math.atan2(1, 1))
print(round(math.erf(0.5), 12), math.gamma(5), math.isqrt(99), math.copysign(3, -0.0))

$ py prog.py
1.4142135623730951 3.141592653589793 2.718281828459045 inf -inf nan
-3 -2 -2 3.0
3.0 3.0 2.718281828459045 5.0
2432902008176640000 120 20 12 12
True True 6.0
180.0 0.5 0.7853981633974483
0.520499877813 24.0 9 -3.0
```

Figure 54. math

random

The C interpreter reproduces CPython 3.13 exactly: after seed(n) every function returns what CPython returns, bit for bit, including getstate and setstate, seed(str), seed(bytes) and seed(float), randrange over ranges of any size, sample (the set-based and the pool algorithm, counts), choices and every distribution. The generator is the Mersenne Twister MT19937, seeded with init_by_array from the 32-bit chunks of abs(n). A str, bytes or bytearray seed goes through version 2 seeding (the seed followed by its SHA-512 digest, read as one big-endian integer; SHA-512 is computed natively), a float seed through its hash, and no seed (or None) from the operating system. Random(seed) creates generators with independent state, a subclass may override random, getrandbits, seed, getstate and setstate (and the algorithm that Random.__init_subclass__ picks for randrange is reproduced), and SystemRandom reads the operating system. The CREXX interpreter has its own generator, which is not CPython's; the exact algorithms are available there as the pure Python module pyrandom (pylib/pyrandom.py), which the C interpreter also runs.

Name	Description
seed([a[,version]]) getstate() setstate(state)	initialise the generator (int, float, str, bytes, bytearray, None); save and restore its state
random() getrandbits(k) randbytes(n)	float in [0.0, 1.0) with 53 bits; k random bits; n random bytes
randint(a,b) randrange(a[,b[,step]])	random integer, for ranges of any size
choice(seq) shuffle(list) sample(seq,k,counts=) choices(seq,weights,cum_weights=,k=)	pick one, shuffle in place, pick k distinct, pick k with replacement
uniform triangular gauss normalvariate lognormvariate expovariate	real-valued distributions
betavariate gammavariate vonmisesvariate paretovariate weibullvariate binomialvariate	more distributions
Random([seed]) SystemRandom()	generators with their own state; the operating system source

Figure 55. random

```
import random
random.seed(42)
a = random.random()
random.seed(42)
print(a == random.random(), 0 <= a < 1)
xs = [random.randint(1, 6) for _ in range(1000)]
print(min(xs), max(xs), 3 < sum(xs) / len(xs) < 4)
deck = list(range(10))
random.shuffle(deck)
print(sorted(deck) == list(range(10)), len(random.sample(deck, 3)),
    random.choice("abc") in "abc")
print(random.randrange(10, 20, 5) in (10, 15), 1.0 <= random.uniform(1, 2) <= 2.0)

$ py prog.py
True True
1 6 True
True 3 True
True True
```

Figure 56. random

```
import random
random.seed(42)
print(random.random(), random.randint(1, 100), random.choice("abcdef"),
    random.sample(range(100), 3))
rng = random.Random("hello")
print(rng.random(), rng.gauss(0, 1), rng.getrandbits(40), rng.choices("abc", k=4))

$ py prog.py
0.6394267984578837 4 f [35, 31, 28]
0.3537754404730722 -0.6533794003395967 387109078255 ['a', 'b', 'c', 'a']
```

Figure 57. random, exact sequences (C interpreter only)

sys and os

Name	Description
argv	command line: the script name followed by its arguments
exit([code])	raise SystemExit; the exit status of the interpreter
stdin stdout stderr	file objects: read, readline, readlines, write, flush, iteration
modules path	loaded modules (dict); module search path (list)
version version_info platform byteorder executable maxsize	information about the interpreter (version is "3.12.0 (C89 subset interpreter)" or "3.12.0 (crexx)"; platform is "c89" in C and "linux" in CREXX; maxsize is 999999999999999999 in C)
getrecursionlimit() setrecursionlimit(n)	recursion limit (1000 in C, about 900 in CREXX; setrecursionlimit has no effect in CREXX)
exc_info() builtin_module_names C	C only
getsizeof getdefaultencoding float_info hexversion implementation maxunicode X	CREXX only

Figure 58. sys

Name	Description
getcwd() getenv(name[,default]) environ	current directory, environment
name sep linesep curdir pardir extsep	constants
path.join exists isfile isdir basename dirname split splitext abspath normpath expanduser	path manipulation and tests

Name	Description
remove rename unlink devnull pathsep C	C only
extsep fspath X	CREXX only (listdir exists in CREXX but raises OSError; it does not exist in C)

Figure 59. os and os.path

time

time(), perf_counter(), monotonic(), process_time() and sleep(seconds). The CREXX interpreter adds time_ns() and perf_counter_ns(). In the C interpreter the clock functions need -DPY_POSIX (otherwise time() counts whole seconds and sleep() does not wait). pylib/time.py (loaded as an overlay of the native module in the C interpreter since Build 10) adds struct_time, gmtime, localtime, mktime, strftime, strptime, asctime, ctime, timezone, altzone, daylight and tzname; with -DPY_POSIX the C interpreter follows the time zone of the process (the TZ variable, tm_zone, tm_gmtoff, %Z and %z), otherwise the local zone is UTC.

json

Native in the C interpreter (src/p7_json.c), Python (pylib/json.py) in the CREXX interpreter; both give the same results. dumps() and dump() accept indent, sort_keys, separators, ensure_ascii, allow_nan, skipkeys, default and cls; loads() and load() accept object_hook, object_pairs_hook, parse_float, parse_int, parse_constant and strict. JSONEncoder and JSONDecoder can be subclassed (an overridden default() is called for objects that are not JSON types), and JSONDecodeError, a subclass of ValueError, carries msg, doc, pos, lineno and colno with the message text of CPython 3.13. Floats are written with repr(), so 1.0 stays 1.0 and large exponents follow CPython. In the C interpreter, whose strings are bytes, the error position pos is counted in characters as in CPython.

```
import json
data = {"name": "Ann", "tags": ["a", "b"], "n": 3, "pi": 3.14, "ok": True, "none": None}
s = json.dumps(data)
print(s)
print(json.dumps(data, indent=2, sort_keys=True))
back = json.loads(s)
print(back == data, back["tags"][1], type(back["n"]).__name__)
print(json.dumps([1, 2, {"a": (1, 2)}], separators=(",", ":")), json.dumps("é\n"))
print(json.dumps("é", ensure_ascii=False), json.loads('"\\u0041\\n"'),
    json.loads("[1.5e2, -0, null]"))
try:
    json.loads("{bad")
except ValueError as e:
    print("ValueError:", e)

$ py prog.py
{"name": "Ann", "tags": ["a", "b"], "n": 3, "pi": 3.14, "ok": true, "none": null}
{
  "n": 3,
  "name": "Ann",
  "none": null,
  "ok": true,
  "pi": 3.14,
  "tags": [
    "a",
    "b"
  ]
}
True b int
[1,2,{"a":[1,2]}] "\u00e9\n"
"é" A
 [150.0, 0, None]
ValueError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)
```

Figure 60. json

re

A backtracking regular-expression engine. In the C interpreter it is native (src/p7_re.c): the tokenizer and parser are a port of CPython's re._parser, so patterns are accepted or rejected with CPython's error messages and positions; the parsed pattern is compiled to a flat instruction array and run by a virtual machine with an explicit backtrack stack, so there is no C recursion per character, and compiled patterns are cached. It supports literals, ., character classes and ranges, the escapes \d \D \w \W \s \S \b \B \A \Z, anchors ^ $, groups (capturing, non-capturing (?:...), named (?P<n>...)), backreferences, alternation, the quantifiers * + ? {m,n} with lazy and possessive variants, atomic groups (?>...), lookahead and fixed-width lookbehind, conditional patterns (?(1)a|b), inline flags, and the flags IGNORECASE, MULTILINE, DOTALL, VERBOSE and ASCII. A search that needs more than 6 million backtrack entries raises RuntimeError ("regular expression too complex") instead of running for hours or crashing. The functions are compile, match, search, fullmatch, findall, finditer, sub, subn, split, escape and purge, with templates (\1, \g<name>) and function replacements; Match objects provide group, groups, groupdict, start, end, span, expand and string; Pattern objects have the same functions as methods. The CREXX interpreter runs the pure-Python engine of pylib/re.py, which has the same API but no conditional patterns, possessive quantifiers or CPython-exact error positions, and is much slower (Chapter 11).

```
import re
text = "Alice <alice@example.com>, Bob <bob@test.org>; tel 555-1234"
print(re.findall(r"[\w.]+@[\w.]+", text))
m = re.search(r"(?P<user>\w+)@(?P<host>[\w.]+)", text)
print(m.group(), m.group("user"), m.groups(), m.groupdict(), m.span(), m.start("host"))
print(re.sub(r"(\d+)-(\d+)", r"\2-\1", text))
print(re.split(r"[,;]\s*", text))
print(re.match(r"\d+", "abc"), bool(re.fullmatch(r"[a-z]+", "abc")))
pat = re.compile(r"(\w)\1")
print([m.group() for m in pat.finditer("aabbcd")], re.sub(r"a+?", "X", "caaat"))
print(re.findall(r"\b\w{4}\b", "this is some text"), re.sub(r"\s+", " ", "a   b \n c"))
print(re.findall(r"(?i)hello", "Hello HELLO"), re.escape("a.b"), re.sub("x", lambda m:
    m.group().upper(), "axbx"))
print(re.findall(r"^\w+", "one\ntwo", re.M), re.findall(r"(?<=\$)\d+", "cost $15 and
    $20"))

$ py prog.py
['alice@example.com', 'bob@test.org']
alice@example.com alice ('alice', 'example.com') {'user': 'alice', 'host':
    'example.com'} (7, 24) 13
Alice <alice@example.com>, Bob <bob@test.org>; tel 1234-555
['Alice <alice@example.com>', 'Bob <bob@test.org>', 'tel 555-1234']
None True
['aa', 'bb'] cXXXt
['this', 'some', 'text'] a b c
['Hello', 'HELLO'] a\.b aXbX
['one', 'two'] ['15', '20']
```

Figure 61. re

Differences from CPython: in the C interpreter strings are bytes, so the offsets of match objects (span, start, end) and the pos and endpos arguments are byte offsets, and \w, \s, \b and case-insensitive matching know only ASCII letters; bytes patterns (rb"a+") and re.Scanner are not available, the flags are plain ints rather than a RegexFlag enum, and a pattern that would take exponential time stops at the stack limit with RuntimeError where CPython would keep going. A differential fuzzer (tests/tools/fuzz_re_json.py, which is not part of the suite) compared about 30,000 random patterns and inputs for re and 40,000 JSON cases with CPython and found them in agreement.

```
import re
print(re.findall(r"(?<![a-z])\d+", "a1 22 b33"))
print(re.match(r"a*+a", "aaa"), re.match(r"(?>a+)a", "aaa"), re.match(r"a*a",
    "aaa").group())
pat = re.compile(r"(\()?\d+(?(1)\))")
print([m.group() for m in pat.finditer("(12) 34 (56")])
print(re.sub(r"(?P<x>a)", r"[\g<x>]", "bab"), re.split(r"(,)", "a,b,c", maxsplit=1))
try:
    re.compile("a{2,1}")
except re.error as e:
    print("re.error:", e)

$ py prog.py
['22', '3']
None None aaa
['(12)', '34', '56']
b[a]b ['a', ',', 'b,c']
re.error: min repeat greater than max repeat at position 2
```

Figure 62. re: lookbehind, possessive quantifiers, atomic groups and conditionals (C interpreter only)

cmath

Complex-number versions of the math functions, native in the C interpreter and a Python port of CPython's cmathmodule.c in the CREXX interpreter: sqrt exp log log10 sin cos tan asin acos atan sinh cosh tanh asinh acosh atanh, phase polar rect, isclose isfinite isnan isinf, and the constants pi e tau inf nan infj nanj. The branch cuts follow CPython for finite arguments; the tables of C99 Annex G for infinities and NaNs are not reproduced, so cmath.sqrt(complex(float("inf"), 1)) and similar special cases may differ in the sign of a zero.

```
import cmath, math
print(cmath.sqrt(-4), cmath.exp(1j * math.pi), cmath.phase(-1), cmath.polar(1 + 1j))
print(cmath.rect(2, math.pi / 2), cmath.log(-1), cmath.sin(1 + 1j), cmath.isclose(1 +
    1j, 1 + 1.0000001j))
print(cmath.pi, cmath.inf, cmath.infj, cmath.isnan(complex(1, float("nan"))))
roots = [cmath.exp(2j * math.pi * k / 4) for k in range(4)]
print([complex(round(r.real, 6) + 0.0, round(r.imag, 6) + 0.0) for r in roots])
a, b, c = 1, 2, 5
d = cmath.sqrt(b * b - 4 * a * c)
print((-b + d) / (2 * a), (-b - d) / (2 * a))

$ py prog.py
2j (-1+1.2246467991473532e-16j) 3.141592653589793 (1.4142135623730951,
    0.7853981633974483)
(1.2246467991473532e-16+2j) 3.141592653589793j (1.2984575814159773+0.6349639147847361j)
    False
3.141592653589793 inf infj True
[(1+0j), 1j, (-1+0j), -1j]
(-1+2j) (-1-2j)
```

Figure 63. cmath

struct

Native in the C interpreter, Python (pylib/struct.py) in the CREXX interpreter: pack, unpack, calcsize, pack_into, unpack_from, iter_unpack, the Struct class and struct.error, for the format characters x c ? b B h H i I l L q Q n N f d s p with the prefixes @ = < > !. Only @ uses native alignment and sizes (LP64 long, IEEE floats). The half-float format e and memoryview-based interfaces are not available.

```
import struct
data = struct.pack("<hif", 1, -2, 2.5)
print(data, len(data), struct.unpack("<hif", data), struct.calcsize("<hif"),
    struct.calcsize("@hif"))
print(struct.pack(">I", 0xDEADBEEF).hex(), struct.unpack(">2H", b"\x00\x01\x00\x02"),
    struct.pack("4s", b"ab"))
s = struct.Struct("<Bh")
print(s.size, s.pack(255, -1), s.unpack(b"\x01\x02\x00"))
try:
    struct.unpack("<i", b"ab")
except struct.error as e:
    print("struct.error:", e)

$ py prog.py
b'\x01\x00\xfe\xff\xff\xff\x00\x00 @' 10 (1, -2, 2.5) 10 12
deadbeef (1, 2) b'ab\x00\x00'
3 b'\xff\xff\xff' (1, 2)
struct.error: unpack requires a buffer of 4 bytes
```

Figure 64. struct

socket and select

IPv4 TCP sockets, in both interpreters. socket(AF_INET, SOCK_STREAM) has bind, listen, accept, connect, connect_ex, send, sendall, recv, shutdown, close, settimeout, setblocking, setsockopt, getsockname and getpeername and works in a with statement; the module has create_connection, gethostname, gethostbyname, inet_aton, inet_ntoa, htons, ntohs, htonl, ntohl, setdefaulttimeout, the usual constants (AF_INET, SOCK_STREAM, SOL_SOCKET, SO_REUSEADDR, SHUT_RD, ...), socket.error (the same class as OSError) and socket.timeout (TimeoutError). Errors are OSError with errno and strerror, with the CPython subclasses ConnectionRefusedError, ConnectionResetError, ConnectionAbortedError, BrokenPipeError, BlockingIOError, InterruptedError and TimeoutError, and a message of the form [Errno 111] Connection refused. select.select(r, w, x, timeout) takes sockets, ints or objects with fileno().

In the C interpreter the BSD socket calls are compiled only with -DPY_POSIX (make posix) and appear in one place, the host_sock_* block of src/p0_os.c; in a build without it import socket works but creating a socket raises OSError: sockets not available, so that the same program text runs on a host with no network. Both modules also have recv_into, getsockopt, fileno, detach, getaddrinfo and getfqdn. The C module adds inet_pton and inet_ntop. The C interpreter has socket.socketpair() (two connected AF_UNIX stream sockets) and sock.makefile(mode) (r, w or rw, each with b; closing the file leaves the socket open) since Build 10. The CREXX interpreter (crexx/pylib/socket.py and select.py) calls the rxsocket library of CREXX through an internal primitive; it adds socketpair, makefile and create_server. In both, sockets are blocking unless a timeout is set, a timeout puts the socket in non-blocking mode internally, and the garbage collector never closes a socket, so close it explicitly. Not available in either: UDP, IPv6, sendto and recvfrom, selectors, ssl and poll; in the CREXX interpreter also DNS lookups other than local names and dotted quads. In the CREXX interpreter select.select polls the number of pending bytes, and a listening socket is readable when a connection is waiting.

```
import socket, select

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("127.0.0.1", 0))
srv.listen(5)
host, port = srv.getsockname()
print(host, port > 1023)

cli = socket.socket()
cli.connect((host, port))
conn, addr = srv.accept()
print(addr[0], conn.getpeername()[1] == cli.getsockname()[1])

cli.sendall(b"GET /index\r\n")
req = conn.recv(100)
print(req)
conn.sendall(b"echo: " + req.strip())
print(cli.recv(100).decode())

r, w, x = select.select([conn], [], [], 0.1)
print("readable before:", len(r))
cli.send(b"ping")
r, w, x = select.select([conn], [conn], [], 1.0)
print("readable:", conn in r, "writable:", conn in w)
print(conn.recv(10))

conn.close()
print(cli.recv(10))
cli.close()
srv.close()

c2 = socket.socket()
c2.settimeout(1.0)
try:
    c2.connect(("127.0.0.1", 1))
except ConnectionRefusedError as e:
    print("ConnectionRefusedError", e.errno)
c2.close()
print(socket.inet_aton("10.1.2.3"), socket.inet_ntoa(b"\x7f\x00\x00\x01"),
    socket.htons(80) in (80, 20480))

$ py prog.py
127.0.0.1 True
127.0.0.1 True
b'GET /index\r\n'
echo: GET /index
readable before: 0
readable: True writable: True
b'ping'
b''
ConnectionRefusedError 111
b'\n\x01\x02\x03' 127.0.0.1 True
```

Figure 65. An echo exchange over a loopback connection

datetime

Pure Python and exact: the classes date, time, datetime, timedelta, timezone and tzinfo, with arithmetic, comparison, isoformat(), fromisoformat(), strftime(), strptime(), weekday(), isoweekday(), isocalendar(),

toordinal(), replace(), combine(), date.today() and datetime.now()/utcnow() (which use the clock functions of the time module). The calendar is proleptic Gregorian.

```
from datetime import date, datetime, timedelta, timezone
d = date(2026, 10, 11)
print(d, d.weekday(), d.isoformat(), d.strftime("%A %d %B %Y"), d + timedelta(days=30))
dt = datetime(2026, 10, 11, 14, 30, 5)
print(dt, dt.strftime("%Y-%m-%d %H:%M:%S"), dt - datetime(2026, 1, 1),
    dt.replace(hour=0))
print(datetime.strptime("2025-02-28 23:59", "%Y-%m-%d %H:%M") + timedelta(minutes=2))
print(date(2024, 2, 29).toordinal(), (date(2026, 12, 25) - d).days, d.isocalendar())
t = timedelta(hours=36, minutes=15)
print(t, t.total_seconds(), t * 2, str(timedelta(days=-1, seconds=5)))
print(datetime(2026, 1, 1, tzinfo=timezone.utc).isoformat(),
    date.fromisoformat("2000-12-31"))

$ py prog.py
2026-10-11 6 2026-10-11 Sunday 11 October 2026 2026-11-10
2026-10-11 14:30:05 2026-10-11 14:30:05 283 days, 14:30:05 2026-10-11 00:30:05
2025-03-01 00:01:00
738945 75 datetime.IsoCalendarDate(year=2026, week=41, weekday=7)
1 day, 12:15:00 130500.0 3 days, 0:30:00 -1 day, 0:00:05
2026-01-01T00:00:00+00:00 2000-12-31
```

Figure 66. datetime

collections

Pure Python in both interpreters. Classes: **defaultdict**, **Counter** (with most_common, elements, update, subtract and arithmetic), **OrderedDict** (with move_to_end and popitem(last)), **deque** (append, appendleft, pop, popleft, extend, extendleft, rotate, maxlen, indexing), **namedtuple**, **ChainMap**, **UserDict** and **UserList**. collections.abc does not exist.

```
from collections import defaultdict, Counter, OrderedDict, deque, namedtuple, ChainMap
dd = defaultdict(list)
for w in ["apple", "avocado", "banana", "blueberry", "cherry"]:
    dd[w[0]].append(w)
print(dict(dd), dd["z"], len(dd))
c = Counter("mississippi")
print(c.most_common(2), c["s"], c["zz"], sorted(c.elements())[:4], c + Counter("sip"))
q = deque([1, 2, 3], maxlen=4)
q.append(4); q.append(5); q.appendleft(0); q.rotate(1)
print(q, q.popleft(), q.pop(), len(q))
Point = namedtuple("Point", ["x", "y"])
p = Point(3, y=4)
print(p, p.x + p.y, p._asdict(), p._replace(x=0), tuple(p), Point._fields)
od = OrderedDict([("a", 1), ("b", 2)])
od.move_to_end("a")
print(list(od), ChainMap({"a": 1}, {"a": 2, "b": 3})["b"])

$ py prog.py
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']} [] 4
[('i', 4), ('s', 4)] 4 0 ['i', 'i', 'i', 'i'] Counter({'i': 5, 's': 5, 'p': 3, 'm': 1})
deque([0, 2], maxlen=4) 4 3 2
Point(x=3, y=4) 7 {'x': 3, 'y': 4} Point(x=0, y=4) (3, 4) ('x', 'y')
['b', 'a'] 3
```

Figure 67. collections

itertools and functools

Pure Python. itertools: accumulate, batched, chain (and chain.from_iterable), combinations, combinations_with_replacement, compress, count, cycle, dropwhile, filterfalse, groupby, islice, pairwise,

permutations, product, repeat, starmap, takewhile, tee and zip_longest. functools: reduce, partial, wraps, update_wrapper, lru_cache (with cache_info and cache_clear), cache, cmp_to_key, total_ordering and singledispatch.

```
import itertools as it
import functools as ft
print(list(it.islice(it.count(10, 5), 4)), list(it.chain("ab", [1, 2])),
    list(it.repeat("x", 3)))
print(list(it.permutations("abc", 2))[:4], list(it.combinations(range(4), 2)))
print(list(it.product("ab", [0, 1])), list(it.accumulate([1, 2, 3, 4])),
    list(it.pairwise("abcd")))
print([(k, list(g)) for k, g in it.groupby("aabbbc")], list(it.zip_longest("ab", "xyz",
    fillvalue="-")))
print(list(it.takewhile(lambda x: x < 4, range(10))), list(it.starmap(pow, [(2, 3), (3,
    2)])))
print(ft.reduce(lambda a, b: a * b, range(1, 8)), ft.partial(int, base=2)("1010"))
@ft.lru_cache(maxsize=None)
def f(n): return n if n < 2 else f(n - 1) + f(n - 2)
print(f(60), f.cache_info())
@ft.total_ordering
class V:
    def __init__(self, n): self.n = n
    def __eq__(self, o): return self.n == o.n
    def __lt__(self, o): return self.n < o.n
print(V(1) <= V(2), V(3) > V(2), sorted(["bb", "a", "ccc"], key=ft.cmp_to_key(lambda a,
    b: len(b) - len(a))))

$ py prog.py
[10, 15, 20, 25] ['a', 'b', 1, 2] ['x', 'x', 'x']
[('a', 'b'), ('a', 'c'), ('b', 'a'), ('b', 'c')] [(0, 1), (0, 2), (0, 3), (1, 2), (1,
    3), (2, 3)]
[('a', 0), ('a', 1), ('b', 0), ('b', 1)] [1, 3, 6, 10] [('a', 'b'), ('b', 'c'), ('c',
    'd')]
[('a', ['a', 'a']), ('b', ['b', 'b', 'b']), ('c', ['c'])] [('a', 'x'), ('b', 'y'), ('-',
    'z')]
[0, 1, 2, 3] [8, 9]
5040 10
1548008755920 CacheInfo(hits=58, misses=61, maxsize=None, currsize=61)
True True ['ccc', 'bb', 'a']
```

Figure 68. itertools and functools

statistics, string, copy, operator, textwrap

Pure Python. **statistics**: mean, fmean, geometric_mean, harmonic_mean, median, median_low, median_high, median_grouped, mode, multimode, variance, pvariance, stdev, pstdev, covariance, correlation, linear_regression, quantiles and StatisticsError. **string**: ascii_letters, ascii_lowercase, ascii_uppercase, digits, hexdigits, octdigits, punctuation, whitespace, printable, capwords and Template (substitute, safe_substitute). **copy**: copy and deepcopy (with the __copy__ and __deepcopy__ hooks; cycles are handled). **operator**: the functional forms of the operators (add, mul, lt, getitem, ...), itemgetter, attrgetter and methodcaller. **textwrap**: wrap, fill, shorten, dedent and indent.

```
import statistics as st, string, copy, operator, textwrap
xs = [2, 4, 4, 4, 5, 5, 7, 9]
print(st.mean(xs), st.median(xs), st.mode(xs), st.pstdev(xs), round(st.stdev(xs), 4),
    st.variance(xs))
print(st.quantiles(xs, n=4), round(st.correlation(xs, xs[::-1]), 4),
    st.harmonic_mean([1, 2, 4]))
print(string.ascii_lowercase[:5], string.digits, string.capwords("hello big world"))
print(string.Template("$who owes $$${amt}").substitute(who="Ann", amt=5))
a = [[1, 2], [3]]
b, c = copy.copy(a), copy.deepcopy(a)
a[0].append(9)
print(b, c)
people = [("ann", 30), ("bob", 25)]
print(sorted(people, key=operator.itemgetter(1)), operator.add(2, 3), operator.mul(4,
    5))
print(textwrap.fill("the quick brown fox jumps over the lazy dog", width=20))
print(textwrap.shorten("hello   wide world", width=14), repr(textwrap.dedent("  a\n
    b")), textwrap.indent("x\ny", "> "))

$ py prog.py
5 4.5 4 2.0 2.1381 4.571428571428571
[4.0, 4.5, 6.5] -0.875 1.7142857142857142
abcde 0123456789 Hello Big World
Ann owes $5
[[1, 2, 9], [3]] [[1, 2], [3]]
[('bob', 25), ('ann', 30)] 5 20
the quick brown fox
jumps over the lazy
dog
hello [...] 'a\n b' > x
> y
```

Figure 69. statistics, string, copy, operator, textwrap

csv, heapq, bisect

Pure Python. **csv**: reader, writer, DictReader, DictWriter, the dialects excel, excel_tab and unix_dialect, register_dialect and the QUOTE_ *constants; they work on any iterable of lines or any object with write(), such as a file or io.StringIO. heapq: heappush, heappop, heapify, heapreplace, heappushpop, nlargest, nsmallest and merge (a min-heap on a list). bisect**: bisect, bisect_left, bisect_right, insort, insort_left and insort_right.

```
import csv, heapq, bisect, io
src = io.StringIO('name,qty\n"Smith, J",3\nDoe,5\n')
for row in csv.DictReader(src):
    print(row)
out = io.StringIO()
w = csv.writer(out)
w.writerow(["a", "b,c", 'say "hi"'])
w.writerows([[1, 2, 3]])
print(repr(out.getvalue()))
h = []
for x in [5, 1, 4, 2, 3]:
    heapq.heappush(h, x)
print([heapq.heappop(h) for _ in range(3)], heapq.nlargest(2, [4, 9, 1, 7]),
    heapq.nsmallest(2, [4, 9, 1, 7]))
a = [1, 3, 5, 7]
print(bisect.bisect(a, 5), bisect.bisect_left(a, 5), bisect.insort(a, 4), a)

$ py prog.py
{'name': 'Smith, J', 'qty': '3'}
{'name': 'Doe', 'qty': '5'}
'a,"b,c","say ""hi"""\r\n1,2,3\r\n'
[1, 2, 3] [9, 7] [1, 4]
3 2 None [1, 3, 4, 5, 7]
```

Figure 70. csv, heapq, bisect

fractions and decimal

Pure Python. **Fraction** has exact rational arithmetic, limit_denominator, as_integer_ratio, the usual operators, and conversion from strings, floats and Decimals. **Decimal** is a decimal floating-point type with a context (prec, rounding), the usual operators, quantize, sqrt, exp, ln, log10, to_integral_value, as_tuple, compare, normalize and the rounding constants ROUND_HALF_EVEN, ROUND_HALF_UP, ROUND_DOWN, ROUND_UP, ROUND_CEILING, ROUND_FLOOR, ROUND_HALF_DOWN and ROUND_05UP; getcontext(), setcontext() and localcontext() manage the context and the signals (InvalidOperation, DivisionByZero, Inexact, Rounded, Overflow) are exceptions.

```
from fractions import Fraction
from decimal import Decimal, getcontext, localcontext, ROUND_HALF_UP
print(Fraction(1, 3) + Fraction(1, 6), Fraction("0.75"), Fraction(3, 12), Fraction(5,
    2) ** 2, float(Fraction(1, 8)))
print(Fraction(0.5), Fraction(1, 3).limit_denominator(2), sum([Fraction(1, n) for n in
    range(1, 6)]))
print(Decimal("0.1") + Decimal("0.2"), Decimal("1.10") * 3, Decimal(1) / Decimal(7))
getcontext().prec = 10
print(Decimal(1) / Decimal(7), Decimal(2).sqrt())
print(Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP),
    Decimal("1E+2"), Decimal("100").normalize())
with localcontext() as ctx:
    ctx.prec = 3
    print(Decimal(22) / Decimal(7))
print(Decimal("3.14").as_tuple(), Decimal("-0.5").copy_abs(), Decimal(10) ** 20)

$ py prog.py
1/2 3/4 1/4 25/4 0.125
1/2 1/2 137/60
0.3 3.30 0.14285714285714285714285714286
0.1428571429 1.414213562
2.68 1E+2 1E+2
3.14
DecimalTuple(sign=0, digits=(3, 1, 4), exponent=-2) 0.5 1.000000000E+20
```

Figure 71. fractions and decimal

enum, dataclasses, abc, contextlib, typing

Pure Python (abc, typing and warnings are native in the CREXX interpreter, with the pure library supplying what is missing). **enum**: Enum, IntEnum, StrEnum, Flag, IntFlag, auto and unique; members have name and value and may have methods and properties on the class (iterating over an Enum class, len(Color), Color["NAME"] and Color(value) are not reliable, see Chapter 12). **dataclasses**: dataclass (init, repr, eq, order, frozen, unsafe_hash), field (default, default_factory), fields, asdict, astuple, replace, is_dataclass, make_dataclass, KW_ONLY and __post_init__. **abc**: ABC, ABCMeta and abstractmethod (the decorator is accepted and documents intent, but instantiating a class that still has abstract methods is **not** prevented). **contextlib**: contextmanager, suppress, closing, nullcontext, redirect_stdout, redirect_stderr, ExitStack and AbstractContextManager. **typing**: the names that programs use in annotations (List, Dict, Optional, Union, Callable, Any, TypeVar, Generic, Protocol, TypedDict, cast, overload, ...; NamedTuple works only in its functional form and only in the C interpreter, use collections.namedtuple instead); annotations are parsed and ignored except by dataclasses and NamedTuple.

```
from enum import Enum, IntEnum, auto
from dataclasses import dataclass, field, asdict, replace
from typing import List

class Color(Enum):
    RED = 1
    GREEN = auto()
    BLUE = auto()
    def describe(self):
        return self.name.lower() + "=" + str(self.value)

class Prio(IntEnum):
    LOW = 1
    HIGH = 3

print(Color.GREEN, repr(Color.RED), Color.BLUE.value, Color.BLUE.describe(),
    Color.RED.name)
print(Prio.HIGH > Prio.LOW, Prio.HIGH + 1, Prio.LOW == 1, Color.RED is Color.RED,
    isinstance(Color.RED, Color))

@dataclass
class Item:
    name: str
    price: float = 0.0
    tags: List[str] = field(default_factory=list)
    def total(self, n):
        return self.price * n

@dataclass(frozen=True, order=True)
class Ver:
    major: int
    minor: int = 0

i = Item("pen", 1.5)
i.tags.append("blue")
print(i, i.total(4), asdict(i), replace(i, name="ink"), i == Item("pen", 1.5, ["blue"]))
print(Ver(1, 2) < Ver(1, 10), Ver(2), sorted([Ver(2), Ver(1, 5)]))

$ py prog.py
Color.GREEN <Color.RED: 1> 3 blue=3 RED
True 4 True True True
Item(name='pen', price=1.5, tags=['blue']) 6.0 {'name': 'pen', 'price': 1.5, 'tags':
    ['blue']} Item(name='ink', price=1.5, tags=['blue']) True
True Ver(major=2, minor=0) [Ver(major=1, minor=5), Ver(major=2, minor=0)]
```

Figure 72. enum and dataclasses

```
from abc import ABC, abstractmethod
from typing import Optional, Dict, Callable

class Animal(ABC):
    @abstractmethod
    def speak(self) -> str: ...
    def twice(self) -> str:
        return self.speak() * 2

class Dog(Animal):
    def speak(self) -> str:
        return "woof"

def apply(f: Callable[[int], int], x: int) -> Optional[int]:
    return f(x) if x else None

print(Dog().twice(), apply(lambda v: v + 1, 4), apply(abs, 0))
inv: Dict[str, int] = {"a": 1}
print(inv)

$ py prog.py
woofwoof 5 None
{'a': 1}
```

Figure 73. abc and typing

pprint, io, warnings, types

pprint: pprint, pformat, pp, saferepr, PrettyPrinter, isreadable and isrecursive, with width, indent, depth, compact and sort_dicts. **io**: StringIO only (read, readline, readlines, write, writelines, getvalue, seek, tell, truncate, iteration and the context-manager protocol). **warnings**: warn, simplefilter, filterwarnings, resetwarnings, catch_warnings and showwarning; warnings are written to standard error. **types**: FunctionType, LambdaType, MethodType, BuiltinFunctionType, GeneratorType, ModuleType, NoneType and SimpleNamespace.

```
import pprint, io, types
data = {"name": "widget", "sizes": [1, 2, 3], "nested": {"a": list(range(12)), "b": "x"
    * 30}}
pprint.pprint(data, width=50)
print(pprint.pformat([1, 2, 3]), pprint.pformat({"b": 1, "a": 2}, sort_dicts=False))
f = io.StringIO()
print("hello", file=f)
f.write("world\n")
f.seek(0)
print(f.readline().strip(), f.read().strip(), repr(f.getvalue()))
ns = types.SimpleNamespace(a=1, b="two")
ns.c = 3
print(ns, ns.a + ns.c, type(len) is types.BuiltinFunctionType)

$ py prog.py
{'name': 'widget',
 'nested': {'a': [0,
                  1,
                  2,
                  3,
                  4,
                  5,
                  6,
                  7,
                  8,
                  9,
                  10,
                  11],
            'b': 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'},
 'sizes': [1, 2, 3]}
[1, 2, 3] {'b': 1, 'a': 2}
hello world 'hello\nworld\n'
namespace(a=1, b='two', c=3) 4 True
```

Figure 74. pprint, io and types

Chapter 6. NumPy

import numpy as np provides a subset of NumPy built into both interpreters: an N-dimensional array type (ndarray) in row-major layout with the data types bool, int64, float64 and complex128, broadcasting arithmetic, indexing, reductions, sorting, linear algebra, random numbers and the fast Fourier transform (numpy.fft), and printing in NumPy's own layout. The array engine is native code in each interpreter (src/p7_np.c, p7_np2.c, p7_la.c and p7_rnd.c in C; j_np.crexx and k_np2.crexx in CREXX); the less frequently used functions are written in Python in pylib/numpy and are loaded on demand. All the examples in this chapter were compared with the output of NumPy 2.5 itself.

Creating arrays

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([[1.5, 2.5], [3.5, 4.5]])
print(a, a.dtype, a.shape, a.ndim, a.size)
print(b, b.dtype, b.shape)
print(np.zeros(3), np.ones((2, 3), dtype=int), np.full((2, 2), 7))
print(np.eye(3), np.identity(2))
print(np.arange(10), np.arange(2, 11, 3), np.arange(0, 1, 0.25))
print(np.linspace(0, 1, 5), np.zeros_like(b), np.array([True, False]))
print(np.array([1, 2.5]), np.array([[1, 2], [3, 4]]).tolist(), np.empty(2).shape)
x, y = np.meshgrid([1, 2, 3], [10, 20])
print(x, y)

$ py prog.py
[1 2 3] int64 (3,) 1 3
[[1.5 2.5]
 [3.5 4.5]] float64 (2, 2)
[0. 0. 0.] [[1 1 1]
 [1 1 1]] [[7 7]
 [7 7]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]] [[1. 0.]
 [0. 1.]]
[0 1 2 3 4 5 6 7 8 9] [2 5 8] [0.   0.25 0.5  0.75]
[0.   0.25 0.5  0.75 1.  ] [[0. 0.]
 [0. 0.]] [ True False]
[1.  2.5] [[1, 2], [3, 4]] (2,)
[[1 2 3]
 [1 2 3]] [[10 10 10]
 [20 20 20]]
```

Figure 75. array, zeros, ones, arange, linspace

Array data types are bool, int64, float64 and complex128 (see Data types and scalar types, and Complex arrays, below). np.float32, np.int32, np.int8 and the other sized names are aliases of the four real types, so no 32-bit arithmetic is ever performed. astype() converts.

Indexing and slicing

Arrays are indexed with integers, slices (with step and negative values), tuples of these, Boolean masks and lists of integers; elements and slices may be assigned. A one-index form on a 2-d array returns a row. In real NumPy, basic slices are views of the same memory; here a slice is a **copy** (in the CREXX interpreter a[i] and a[i:j] on the first axis are views), so assignment through a chained index such as b[0][1] = 5 does not change b. Assign with a single multi-part index, b[0, 1] = 5, instead.

```
import numpy as np
a = np.arange(12).reshape(3, 4)
print(a)
print(a[1], a[1, 2], a[-1, -1], a[:, 1], a[1:, ::2], a[::-1, 0])
print(a[a > 8], a[[0, 2]], a[a % 2 == 0].sum())
a[0, 0] = 100
a[:, 3] = -1
a[a < 0] = 0
print(a)
b = np.arange(5) * 1.5
b[1:3] = [9, 9]
print(b, b[::-1], b[np.array([True, False, True, False, True])])
print(np.where(b > 5, b, 0), np.nonzero(b > 5))

$ py prog.py
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[4 5 6 7] 6 11 [1 5 9] [[ 4  6]
 [ 8 10]] [8 4 0]
[ 9 10 11] [[ 0  1  2  3]
 [ 8  9 10 11]] 30
[[100   1   2   0]
 [  4   5   6   0]
 [  8   9  10   0]]
[0.  9.  9.  4.5 6. ] [6.  4.5 9.  9.  0. ] [0. 9. 6.]
[0. 9. 9. 0. 6.] (array([1, 2, 4]),)
```

Figure 76. Indexing

Arithmetic and broadcasting

Operators + - * / // % * @, the comparisons and unary minus work element by element and broadcast arrays of different shapes by the NumPy rules (a trailing dimension of 1, or a missing leading dimension, is stretched). An integer operation that overflows 64 bits wraps around. Division always gives floats. Python scalars and arrays mix freely.

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
v = np.array([10, 20, 30])
col = np.array([[100], [200]])
print(a + v, a * col, a + 1, 2 * a, a / 2, a // 2, a % 3, a ** 2)
print(a > 2, a == v[0], (a > 2) & (a < 6), ~(a > 2), -a)
print(a @ np.ones((3, 2)), a.T, np.dot(v, v), np.outer([1, 2], [3, 4]))
print(np.array([1, 2, 3]) / np.array([2, 2, 2]), np.sqrt([1, 4, 9]), np.exp([0, 1]),
    np.power(2, [1, 2, 3]))
print(np.maximum(a, 3), np.minimum(a, v), np.clip(a, 2, 4), np.abs([-1, 2, -3]),
    np.round([1.234, 5.678], 1))

$ py prog.py
[[11 22 33]
 [14 25 36]] [[ 100  200  300]
 [ 800 1000 1200]] [[2 3 4]
 [5 6 7]] [[ 2  4  6]
 [ 8 10 12]] [[0.5 1.  1.5]
 [2.  2.5 3. ]] [[0 1 1]
 [2 2 3]] [[1 2 0]
 [1 2 0]] [[ 1  4  9]
 [16 25 36]]
[[False False  True]
 [ True  True  True]] [[False False False]
 [False False False]] [[False False  True]
 [ True  True False]] [[ True  True False]
 [False False False]] [[-1 -2 -3]
 [-4 -5 -6]]
[[ 6.  6.]
 [15. 15.]] [[1 4]
 [2 5]
 [3 6]] 1400 [[3 4]
 [6 8]]
[0.5 1.  1.5] [1. 2. 3.] [1.         2.71828183] [2 4 8]
[[3 3 3]
 [4 5 6]] [[1 2 3]
 [4 5 6]] [[2 2 3]
 [4 4 4]] [1 2 3] [1.2 5.7]
```

Figure 77. Broadcasting

Reductions, sorting and searching

```
import numpy as np
a = np.array([[3, 1, 4], [1, 5, 9], [2, 6, 5]])
print(a.sum(), a.sum(axis=0), a.sum(axis=1), a.mean(), a.mean(axis=0))
print(a.min(), a.max(axis=1), a.argmax(), a.argmin(axis=0), a.prod(), a.cumsum())
print(a.std(), a.var(axis=0), np.median(a), np.percentile(a, 75), np.ptp(a))
print(np.sort(a), np.argsort([3, 1, 2]), np.unique([3, 1, 3, 2, 1]))
print(np.cumsum(a, axis=1), np.cumprod([1, 2, 3, 4]), np.diff([1, 4, 9, 16]), np.any(a
    > 8), np.all(a > 0))
print(np.concatenate([a, a], axis=1).shape, np.vstack([a, a]).shape, np.hstack([a,
    a]).shape, np.stack([a, a]).shape)
print(np.searchsorted([1, 3, 5, 7], 4), np.bincount([0, 1, 1, 3]), np.histogram([1, 2,
    1, 3, 3, 3], bins=3))
print(np.trace(a), np.diag(a), np.tril(a), a.reshape(1, 9).shape, a.flatten().tolist(),
    a.ravel()[:3])
print(np.allclose([1.0, 2.0], [1.0, 2.0 + 1e-12]), np.array_equal(a, a.copy()),
    np.isnan([1.0, np.nan]))

$ py prog.py
36 [ 6 12 18] [ 8 15 13] 4.0 [2. 4. 6.]
1 [4 9 6] 5 [1 0 0] 32400 [ 3  4  8  9 14 23 25 31 36]
2.449489742783178 [0.66666667 4.66666667 4.66666667] 4.0 5.0 8
[[1 3 4]
 [1 5 9]
 [2 5 6]] [1 2 0] [1 2 3]
[[ 3  4  8]
 [ 1  6 15]
 [ 2  8 13]] [ 1  2  6 24] [3 5 7] True True
(3, 6) (6, 3) (3, 6) (2, 3, 3)
2 [1 2 0 1] (array([2, 1, 3]), array([1.        , 1.66666667, 2.33333333, 3.        ]))
13 [3 5 5] [[3 0 0]
 [1 5 0]
 [2 6 5]] (1, 9) [3, 1, 4, 1, 5, 9, 2, 6, 5] [3 1 4]
True True [False  True]
```

Figure 78. Reductions with axis

Linear algebra

numpy.linalg handles real double matrices of two dimensions: det, slogdet, inv, pinv, solve, lstsq, norm, cond, matrix_rank, matrix_power, trace, cholesky, qr, svd, eigvalsh, eigh, eigvals and eig, with LinAlgError. Matrix products are computed in plain loops and may differ from NumPy's BLAS in the last digit. eigvals() and eig() return complex arrays when an eigenvalue is complex, and real arrays when all eigenvalues are real (NumPy 2.5 returns a complex128 array in both cases; older NumPy versions returned real arrays like these); conjugate pairs stay together, with the positive imaginary part first. In both interpreters eigvals, eig, eigh, eigvalsh, svd and qr are ports of the LAPACK routines that NumPy calls (dgeev, dsyevd, dgesdd, dgeqr2 with dorg2r), not general solvers, so the order of the eigenvalues, the normalisation of the eigenvectors and the signs of the singular vectors and of Q and R agree with NumPy in nearly all cases (see the differences below). Complex input to the other linalg functions raises TypeError.

```
import numpy as np
from numpy import linalg as la
A = np.array([[4.0, 2.0], [1.0, 3.0]])
b = np.array([6.0, 5.0])
print(round(la.det(A), 6), la.inv(A), la.solve(A, b), la.norm(b), la.matrix_rank(A))
print(la.eigvalsh(np.array([[2.0, 1.0], [1.0, 2.0]])))
Q, R = la.qr(A)
print(np.allclose(Q @ R, A), np.round(np.abs(R), 6))
U, S, Vt = la.svd(A)
print(np.round(S, 6), np.allclose(U @ np.diag(S) @ Vt, A))
x, res, rank, sv = la.lstsq(np.array([[1.0, 1], [1, 2], [1, 3]]), np.array([1.0, 2, 2]),
    rcond=None)
print(np.round(x, 6), rank, la.matrix_power(A, 3))
print(la.cholesky(np.array([[4.0, 2], [2, 3]])))
print(la.eigvals(np.array([[1.0, 2.0], [-2.0, 1.0]])))
try:
    la.inv(np.zeros((2, 2)))
except la.LinAlgError as e:
    print("LinAlgError:", e)

$ py prog.py
10.0 [[ 0.3 -0.2]
 [-0.1  0.4]] [0.8 1.4] 7.810249675906654 2
[1. 3.]
True [[4.123106 2.667892]
 [0.       2.425356]]
[5.116673 1.954395] True
[0.666667 0.5     ] 2 [[86. 78.]
 [39. 47.]]
[[2.         0.        ]
 [1.         1.41421356]]
[1.+2.j 1.-2.j]
LinAlgError: Singular matrix
```

Figure 79. numpy.linalg

```
import numpy as np
from numpy import linalg as la
C = np.array([[0.0, 1.0, 0.0], [0.0, 0.0, 1.0], [-6.0, -11.0, -6.0]])
print(np.round(np.real(la.eigvals(C)), 6))
w, v = la.eig(np.diag([1.0, 2.0, 3.0]))
print(np.real(w), np.real(v).tolist())
S = np.array([[2.0, 1.0, 0.0], [1.0, 2.0, 1.0], [0.0, 1.0, 2.0]])
vals, vecs = la.eigh(S)
print(np.round(vals, 6), np.allclose(S @ vecs, vecs * vals))
M = np.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
U, s, Vt = la.svd(M, full_matrices=False)
print(np.round(s, 6), np.allclose(U * s @ Vt, M))
Q, R = la.qr(M)
print(np.round(R, 6))

$ py prog.py
[-1. -2. -3.]
[1. 2. 3.] [[1.0, 0.0, 0.0], [0.0, 1.0, 0.0], [0.0, 0.0, 1.0]]
[0.585786 2.       3.414214] True
[9.525518 0.514301] True
[[-5.91608  -7.437357]
 [ 0.        0.828079]]
```

Figure 80. Eigenvalues and decompositions in the style of LAPACK

Differences from NumPy, which are the same in both interpreters. (1) NumPy returns a complex128 array from eig and eigvals also when every eigenvalue is real; these interpreters return a real array in that case and a complex one only when an eigenvalue is complex, so print(la.eigvals(A)) shows 3. and not 3.+0.j. Use np.real(), or compare with np.allclose, in code that must run on both. (2) The last bit of an eigenvector can decide a sign. NumPy uses an optimised BLAS with fused multiply-add kernels, which sums in a different order, so a QR sweep or a Householder reflector that is decided by the last bit can end with the opposite sign (or, for a complex pair, a different phase). In tests with random matrices about 5 per cent of dense non-symmetric matrices gave an eigenvector that differs from NumPy by a sign; eigenvalues, residuals and orthogonality agree. (3) Exactly degenerate cases are arbitrary in LAPACK as well and are not reproducible: the eigenvectors of a repeated eigenvalue of a defective matrix, the null space of a singular matrix, tied singular values and the columns of Q beyond the rank. (4) For matrices larger than 25 by 25 NumPy switches eigh and svd to divide and conquer (dstedc, dbdsdc); these interpreters keep the QL/QR iteration, so vectors can differ in sign there, and eigvalsh and svd(compute_uv=False) use the same iteration as with vectors where NumPy uses dsterf and dqds (values agree to rounding). (5) Singular detection: inv, solve and det follow the operation order of OpenBLAS getf2 and raise LinAlgError for the same matrices as NumPy except for a few matrices of order 6 and above; beyond a few dozen rows NumPy uses blocked algorithms, and the rounding of the result (and so the verdict for a nearly singular matrix) can differ. (6) Matrices with a condition number above about 1e10 give pinv, lstsq and cond results that agree only to a few digits.

```
import numpy as np
A = np.array([[2.0, 1.0], [1.0, 2.0]])
print(repr(np.linalg.eigvals(A)))
B = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]])
print(np.linalg.matrix_rank(B))
try:
    np.linalg.inv(B)
except np.linalg.LinAlgError as e:
    print("LinAlgError:", e)
C = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 10.0]])
print(np.linalg.inv(C).round(4))
print(np.linalg.det(C).round(6))

$ py prog.py   (C interpreter)
array([3., 1.])
2
LinAlgError: Singular matrix
[[-0.6667 -1.3333  1.    ]
 [-0.6667  3.6667 -2.    ]
 [ 1.     -2.      1.    ]]
-3.0

$ py prog.py   (CREXX interpreter)
array([3., 1.])
2
LinAlgError: Singular matrix
[[-0.6667 -1.3333  1.    ]
 [-0.6667  3.6667 -2.    ]
 [ 1.     -2.      1.    ]]
-3.0

$ python3 prog.py   (CPython 3)
array([3.+0.j, 1.+0.j])
2
LinAlgError: Singular matrix
[[-0.6667 -1.3333  1.    ]
 [-0.6667  3.6667 -2.    ]
 [ 1.     -2.      1.    ]]
-3.0
```

Figure 81. Real or complex result, and singular-matrix detection

The matrix B is singular, and inv says so, as NumPy does. Up to Build 3 inv and solve raised LinAlgError only for a pivot that was exactly zero, and B (whose rounded pivot is not zero) was inverted to numbers of the order of 1e15. Since Build 4 inv, solve and det use a left-looking LU factorisation in the operation order of OpenBLAS getf2 (the pivot is inverted once and the column scaled by the reciprocal; dot products use a fused multiply-add, which is emulated in C89 by a Dekker product in the C interpreter and in plain arithmetic in pylib/numpy/linalg.py for CREXX; the update is made in groups of four columns), so the pivots round as they do in NumPy. On 3,000 random integer matrices of orders 2 to 6 compared with NumPy 2.5.3 the verdict singular or not was identical for orders 2 to 5, and differed for 7 of 613 matrices of order 6 (OpenBLAS kernel details); the elements of the result agree to the last digits or to a few units in the last place.

Random numbers

In the C interpreter numpy.random reproduces NumPy 2.x exactly (checked against NumPy 2.5.3). The legacy interface, seed(n), RandomState(n) and the module-level functions, uses MT19937 seeded with init_genrand (init_by_array for an array seed): rand, random_sample, randint (masked rejection), randn and normal (the polar Box-Muller method with its cached second value), choice, permutation, shuffle, bytes and the distributions exponential, gamma, beta, chisquare, f, standard_t, pareto, weibull, power, laplace, gumbel, logistic, lognormal, rayleigh, wald, vonmises, triangular, poisson, binomial, geometric, negative_binomial, logseries, zipf, hypergeometric, multinomial and dirichlet, with get_state and set_state. default_rng(seed) is SeedSequence and PCG64 (a 128-bit state held in two 64-bit halves) with the ziggurat method for normal and exponential variates, Lemire's method for integers(), Floyd's algorithm for choice without replacement, permutation and shuffle; PCG64, MT19937 and SeedSequence objects (random_raw, state, generate_state, spawn) can be used on their own. The six ziggurat tables cannot be recomputed bit for bit and are stored as constants taken from NumPy. Not provided are multivariate_normal, an array n for multinomial, integers() with array bounds, and the sized dtypes of integers(), which draw as NumPy does but return int64. wald, pareto, power and rayleigh can differ in the last digit from a NumPy built with fused multiply-add. The CREXX interpreter has the same algorithms in pure Python (pylib/numpy/random.py and _rng.py).

```
import numpy as np
rng = np.random.default_rng(7)
x = rng.random(5)
y = rng.normal(0, 1, size=1000)
print(x.shape, bool((x >= 0).all() and (x < 1).all()), abs(y.mean()) < 0.2, 0.8 <
    y.std() < 1.2)
np.random.seed(1)
a = np.random.rand(3)
np.random.seed(1)
print(np.array_equal(a, np.random.rand(3)), np.random.randint(0, 10, size=5).max() < 10)
print(sorted(np.random.permutation(5).tolist()), rng.integers(0, 5, 3).shape,
    np.random.choice([1, 2, 3], size=4).shape)
np.random.seed(0)
print(np.random.rand(3).round(6).tolist(), np.random.randint(0, 100, 5).tolist(),
    np.random.randn(2).round(6).tolist())
g = np.random.default_rng(42)
print(g.random(3).round(6).tolist(), g.integers(0, 100, 4).tolist(), g.choice(10, 3,
    replace=False).tolist())

$ py prog.py
(5,) True True True
True True
[0, 1, 2, 3, 4] (3,) (4,)
[0.548814, 0.715189, 0.602763] [67, 9, 83, 21, 36] [-0.291394, -0.13309]
[0.773956, 0.438878, 0.858598] [8, 69, 20, 9] [4, 8, 7]
```

Figure 82. numpy.random

Printing

print() and repr() of an array follow NumPy's rules: integer and bool arrays are right-aligned in columns of the width of the widest element; floating-point arrays use a common number of decimals (or exponent form when the range is large); long rows wrap at 75 columns; arrays of more than 1000 elements are summarised with ... after the first and last three rows or columns; nested rows are printed on separate lines. The repr() form with array(...) and dtype= is printed as in NumPy for the types that exist here. np.set_printoptions is not available.

```
import numpy as np
print(np.array([1.5, 2.0, 3.25]), np.array([[1, 20], [300, 4]]), np.array([1e10, 1.0]),
    np.array([0.1, 0.25]))
print(np.arange(30), np.linspace(0, 1, 7))
print(np.zeros((2, 2, 2), dtype=int))
print(np.arange(2000).reshape(40, 50)[:3], np.arange(1001))
print(repr(np.array([1, 2])), repr(np.array([1.0, 2.5])), np.array([]),
    np.array([True]))
print(np.array([1, 2, 3]).mean(), np.array([1, 2]).sum(), float(np.array([2.5])[0]))

$ py prog.py
[1.5  2.   3.25] [[  1  20]
 [300   4]] [1.e+10 1.e+00] [0.1  0.25]
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29] [0.         0.16666667 0.33333333 0.5        0.66666667 0.83333333
 1.        ]
[[[0 0]
  [0 0]]

 [[0 0]
  [0 0]]]
[[  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
   18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35
   36  37  38  39  40  41  42  43  44  45  46  47  48  49]
 [ 50  51  52  53  54  55  56  57  58  59  60  61  62  63  64  65  66  67
   68  69  70  71  72  73  74  75  76  77  78  79  80  81  82  83  84  85
   86  87  88  89  90  91  92  93  94  95  96  97  98  99]
 [100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117
  118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135
  136 137 138 139 140 141 142 143 144 145 146 147 148 149]] [   0    1    2 ...  998
     999 1000]
array([1, 2]) array([1. , 2.5]) [] [ True]
2.0 3 2.5
```

Figure 83. Printing arrays

Data types and scalar types

a.dtype is a numpy.dtype object, one object for each of bool, int64, float64 and complex128. It prints as dtype('float64') with repr and as float64 with str, has name, kind (b, i, f, c), char, str, itemsize and type, hashes, and compares equal to another dtype, to a Python or NumPy type (float, int, complex, bool, np.float64) and to a name string ('float64', 'f8', 'complex128'). np.dtype(x) builds one from any of these, and a dtype object may be passed as dtype=. A comparison such as a.dtype == float therefore works as in NumPy.

The scalar type names np.float64, np.int64, np.complex128 and np.bool_ are the Python types float, int, complex and bool, so np.float64(3.5) is a plain float, and the reductions of an array (sum, mean, max, ...) return plain Python numbers: type(np.array([1.5]).sum()) is float here, np.float64 in NumPy, and repr() shows 2.5 where NumPy 2 shows np.float64(2.5). The abstract classes np.generic, number, integer, signedinteger, unsignedinteger, inexact, floating and complexfloating exist, and isinstance and issubclass answer by the value's type: isinstance(1.5, np.floating) and isinstance(1, np.integer) are true (in NumPy they are false for Python scalars and true for NumPy scalars). The sized names int8 ... uint64, float16, float32, float128 and

complex64 are aliases of the four real types in the C interpreter; the CREXX interpreter has only the names listed in the function reference below.

```
import numpy as np
a = np.array([1.5, 2.5])
i = np.array([1, 2, 3])
c = np.array([1j])
print(repr(a.dtype), str(i.dtype), c.dtype.name, a.dtype.kind, i.dtype.itemsize,
    c.dtype.itemsize)
print(a.dtype == float, a.dtype == np.float64, a.dtype == "float64", i.dtype == int,
    i.dtype != float)
print(np.dtype("f8"), np.dtype(int) == i.dtype, np.array([1, 2], dtype=np.float64),
    np.zeros(2, dtype=np.dtype("complex128")))
x = np.float64(3.5)
print(x, np.int64(7) + 1, np.complex128(1 + 2j), np.bool_(1))
print(isinstance(a.sum(), np.floating), isinstance(i.sum(), np.integer),
    isinstance(c.sum(), np.complexfloating), isinstance(i.sum(), np.number))
print(issubclass(np.float64, np.floating), issubclass(np.int64, np.integer))

$ py prog.py
dtype('float64') int64 complex128 f 8 16
True True True True True
float64 True [1. 2.] [0.+0.j 0.+0.j]
3.5 8 (1+2j) True
True True True True
True True
```

Figure 84. dtype objects and scalar types

Complex arrays

An array whose elements are complex (np.array([1 + 2j, 3j]), dtype=complex, astype(complex), or the result of an operation with a complex scalar) has the dtype complex128. In the C interpreter the data are interleaved doubles in the same array type; in the CREXX interpreter they are an instance of the class _CArr of pylib/numpy/_cplx.py, which holds two float64 arrays (real and imaginary parts) and to which the evaluator hands every operation that involves a complex array. Both support arithmetic with broadcasting, comparison with == and !=, .real .imag .conj() and .T, the functions abs, angle, sqrt, exp, log, sin, cos and the rest of the elementary functions, real, imag, conj, iscomplex, isreal, iscomplexobj, sum, mean, prod, cumsum, dot and @, indexing, assignment, iteration and tolist(), and numpy's own print layout ([1.+2.j 3.-1.j], exponent form for large ranges, wrapping at 75 columns). np.roots finds the roots of a polynomial with real coefficients and np.linalg.eigvals and eig return complex arrays when an eigenvalue is complex.

```
import numpy as np
a = np.array([1 + 2j, 3 - 1j, 2 - 2j])
print(a, a.dtype, a.shape)
print(a.real, a.imag, a.conj(), np.abs(a).round(4))
print(a * 2, a + 1, a * a, a / (1 + 1j))
print(a.sum(), a.mean(), np.angle(a).round(3))
print(np.sqrt(np.array([-4.0 + 0j, 9 + 0j])), np.exp(1j * np.pi *
    np.arange(3)).round(3))
print(np.zeros(2, dtype=complex), np.array([1, 2]).astype(complex), a.tolist())
print(np.roots([1, 0, 1]), np.roots([1, -3, 2]), np.roots([1, 2, 5]))
print(np.linalg.eigvals(np.array([[0.0, -1.0], [1.0, 0.0]])))
print(np.linalg.eigvals(np.array([[1.0, 2.0, 0.0], [-2.0, 1.0, 0.0], [0.0, 0.0, 5.0]])))
print(np.iscomplexobj(a), np.isreal(a), a.dtype == complex)

$ py prog.py
[1.+2.j 3.-1.j 2.-2.j] complex128 (3,)
[1. 3. 2.] [ 2. -1. -2.] [1.-2.j 3.+1.j 2.+2.j] [2.2361 3.1623 2.8284]
[2.+4.j 6.-2.j 4.-4.j] [2.+2.j 4.-1.j 3.-2.j] [-3.+4.j  8.-6.j  0.-8.j] [1.5+0.5j 1.
    -2.j  0. -2.j ]
(6-1j) (2-0.3333333333333333j) [ 1.107 -0.322 -0.785]
[0.+2.j 3.+0.j] [ 1.+0.j -1.+0.j  1.-0.j]
[0.+0.j 0.+0.j] [1.+0.j 2.+0.j] [(1+2j), (3-1j), (2-2j)]
[-0.+1.j  0.-1.j] [2. 1.] [-1.+2.j -1.-2.j]
[0.+1.j 0.-1.j]
[1.+2.j 1.-2.j 5.+0.j]
True [False False False] True
```

Figure 85. Complex arrays

Limits. In the C interpreter sort, argsort, unique, diag, trace, allclose, array_equal, count_nonzero and where raise TypeError for complex input, and so do linalg functions other than eig and eigvals; the CREXX interpreter has most of the first group (cumsum, sort, argsort, unique, diag, trace, count_nonzero, allclose, array_equal) but not np.concatenate of complex with real arrays and not complex linspace. Comparison with < is lexicographic in C. complex64 is complex128. A complex array indexed to a single element gives a plain complex number.

Fourier transforms: numpy.fft

numpy.fft is the pure-Python module pylib/numpy/fft.py, on the complex arrays described above: fft, ifft, rfft, irfft, hfft, ihfft, fft2, ifft2, rfft2, irfft2, fftn, ifftn, rfftn, irfftn, fftfreq, rfftfreq, fftshift and ifftshift, with n, s, axis, axes and norm ("backward", "ortho", "forward") as in NumPy. The one-dimensional transform is a radix-2 Cooley-Tukey algorithm for lengths that are powers of two, a mixed-radix split by the smallest prime factor for other composite lengths, and, for a prime length, a direct sum up to 40 points and Bluestein's chirp transform beyond, so a transform costs O(n log n) for every length except the short primes. import numpy.fft, from numpy import fft and from numpy.fft import fft all work (the native numpy module has a package path for the library files). Results agree with NumPy to about 1e-12; round them before printing. The CREXX interpreter, which never frees memory, can run out of it on long transforms: five transforms of 4096 points in one program use more than 6 GB there (Chapter 11).

```
import numpy as np
x = np.array([1.0, 2.0, 3.0, 4.0, 0.0, -1.0, -2.0, 1.0])
X = np.fft.fft(x)
print(np.round(X, 3))
print(np.round(np.abs(X), 3))
print(np.round(np.fft.ifft(X).real, 6))
print(np.fft.fftfreq(8, d=0.5), np.fft.rfftfreq(8))
print(np.round(np.fft.rfft(x), 3))

# find the frequencies in a signal
fs = 64
t = np.arange(fs) / fs
sig = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 12 * t)
spec = np.abs(np.fft.rfft(sig)) / (fs / 2)
peaks = [int(i) for i in range(len(spec)) if spec[i] > 0.1]
print(peaks, [round(float(spec[i]), 3) for i in peaks])

# circular convolution through the FFT
a = np.array([1.0, 2.0, 3.0, 0.0, 0.0])
b = np.array([0.0, 1.0, 0.5, 0.0, 0.0])
c = np.fft.ifft(np.fft.fft(a) * np.fft.fft(b)).real
print(np.round(c, 6) + 0.0)

img = np.arange(12.0).reshape(3, 4)
print(np.round(np.fft.fft2(img)[0], 3))
print(np.fft.fftshift(np.arange(6)), np.fft.ifftshift(np.fft.fftshift(np.arange(6))))

$ py prog.py
[ 8.+0.j     1.-9.243j  0.+4.j     1.+0.757j -4.+0.j     1.-0.757j
  0.-4.j     1.+9.243j]
[8.    9.297 4.    1.254 4.    1.254 4.    9.297]
[ 1.  2.  3.  4.  0. -1. -2.  1.]
[ 0.    0.25  0.5   0.75 -1.   -0.75 -0.5  -0.25] [0.    0.125 0.25  0.375 0.5  ]
[ 8.+0.j     1.-9.243j  0.+4.j     1.+0.757j -4.+0.j   ]
[5, 12] [1.0, 0.5]
[0.  1.  2.5 4.  1.5]
[66.+0.j -6.+6.j -6.+0.j -6.-6.j]
[3 4 5 0 1 2] [0 1 2 3 4 5]
```

Figure 86. numpy.fft

Function reference

The figure below lists the functions of the top-level numpy namespace. Names in the first column are native code in both interpreters. Names in the second column are written in Python (pylib/numpy/__init__.py) in both interpreters, or, when followed by (X) or (C), only in the CREXX or only in the C interpreter respectively, the other interpreter having a native version. The third column holds names that exist in one interpreter only (C or X after the name).

Group	Native in both	Python library	Only in one
Creation	array asarray zeros ones empty full zeros_like ones_like full_like eye identity arange linspace diag	asanyarray ascontiguousarray empty_like (X) meshgrid (X) tril (X) triu (X) diagflat vander (X) tril_indices triu_indices diag_indices copy (X)	tri C
Shape manipulation	reshape ravel transpose concatenate vstack hstack	ndim (X) shape (X) size (X) swapaxes (X) squeeze (X) expand_dims (X) atleast_1d atleast_2d broadcast_to stack (X) dstack column_stack split array_split flip (X) fliplr (X) flipud (X) roll (X) repeat (X) tile (X) append (X) delete take (X)	

Group	Native in both	Python library	Only in one
Element-wise arithmetic	power square sqrt abs absolute maximum minimum clip	add (X) subtract (X) multiply (X) divide (X) floor_divide (X) mod (X) remainder (X) fmod negative (X) reciprocal (X) cbrt (X) sign (X) fmax (X) fmin (X) hypot (X)	true_divide C copysign C
Exponentials and logarithms	exp log log2 log10	exp2 (X) expm1 (X) log1p (X)	
Trigonometry	sin cos tan	arcsin (X) arccos (X) arctan (X) arctan2 (X) sinh (X) cosh (X) tanh (X) degrees (X) radians (X) deg2rad (X) rad2deg (X)	arcsinh C arccosh C arctanh C sinc C
Rounding	round around floor ceil	trunc (X) fix (X) rint (X)	
Tests and logic	isnan where any all allclose array_equal	isinf (X) isfinite (X) isclose (X) isscalar (X) logical_and (X) logical_or (X) logical_xor (X) logical_not (X) nonzero (X) flatnonzero count_nonzero (X) isin (X) in1d	argwhere C
Reductions and statistics	sum prod mean median std var min max amin amax cumsum argmax argmin	average (X) ptp (X) percentile (X) quantile (X) cumprod (X) nansum (X) nanmean (X) nanmedian nanstd (X) nanvar (X) nanmin (X) nanmax (X) nan_to_num (X) cov (X) corrcoef (X) bincount (X) histogram (X) digitize	nanprod C nanargmax C nanargmin C
Sorting, searching and sets	sort unique	argsort (X) lexsort searchsorted (X) intersect1d union1d setdiff1d	
Differences, calculus and polynomials		diff (X) gradient (X) trapz (X) trapezoid (X) interp convolve (X) polyfit (X) polyval (X) polyder polyadd	ediff1d C
Products	dot matmul trace	inner (X) outer (X) vdot (X) cross (X) kron (X) tensordot	
Constants	pi e inf nan newaxis		Inf C NaN C NINF C PINF C euler_gamma C
Complex numbers		real (X) imag (X) conj (X) conjugate (X) angle (X) iscomplex (X) isreal (X) iscomplexobj (X) roots (X)	
Types and submodules	ndarray dtype number integer floating complexfloating float64 float32 int64 int32 bool_ complex128 fft		generic C signedinteger C unsignedinteger C inexact C float16 C float_ C double C single C half C longdouble C float128 C int16 C int8 C int_ C intc C intp C long C longlong C short C uint C uint8 C uint16 C uint32 C uint64 C uintp C ubyte C byte C ulonglong C ushort C bool C complex64 C complex_ C cdouble C csingle C clongdouble C complex256 C LinAlgError C

Figure 87. numpy functions and constants

The ndarray methods are: T, shape, ndim, size, dtype, tolist, copy, reshape, flatten, ravel, astype, item, transpose, sort, argsort, sum, mean, min, max, prod, argmax, argmin, cumsum, cumprod, std, var, round, dot, any, all, clip, nonzero, trace, len(), iteration over the first axis, and the operators described above. In the CREXX interpreter some of them are reached through the numpy function of the same name.

Module	Names
numpy.linalg (native in C, Python in CREXX)	det slogdet inv pinv solve lstsq norm cond matrix_rank matrix_power cholesky qr svd eigvalsh eigh eigvals eig LinAlgError; multi_dot and trace are Python-only in CREXX and missing from C
numpy.random (native in C, Python in CREXX)	seed rand randn random random_sample ranf sample randint choice permutation shuffle bytes get_state set_state normal uniform standard_normal exponential standard_exponential gamma standard_gamma beta chisquare noncentral_chisquare f noncentral_f standard_cauchy standard_t pareto weibull power laplace gumbel logistic lognormal rayleigh wald vonmises triangular poisson binomial geometric negative_binomial logseries zipf hypergeometric multinomial dirichlet default_rng Generator RandomState PCG64 MT19937 SeedSequence

Figure 88. numpy.linalg and numpy.random

Differences from real NumPy

- Only the data types bool, int64, float64 and complex128 exist. float32, int32 and the other sized names are aliases; there are no string, object, datetime or structured arrays and no masked arrays. dtype objects exist (a.dtype == float works) but cannot describe a record.
- Scalars returned by reductions and by indexing one element are plain Python int, float, bool or complex, not numpy scalar objects. repr() of such a value therefore looks like a Python number (for example 2.5 and not np.float64(2.5)), and type() reports int, float, bool or complex; np.float64(3.5) is a float and np.float32(0.1) == 0.1 is true.
- Slices, transpose, reshape and T return copies (CREXX: a[i] and a[i:j] on the first axis are views). Code that relies on writing through a view must be changed to assign to the array itself.
- In the C interpreter np.sort works along the last axis only (the CREXX interpreter accepts axis) and arrays have at most 8 dimensions. N-d matmul is not supported. Indexing with two index arrays at once (a[[0, 1], [1, 2]]) does not give NumPy's pairwise result but selects the outer product of the indices, so it must be avoided.
- Linear algebra is for real two-dimensional matrices (eig and eigvals return complex results when needed). The eigenvalue and decomposition routines follow the LAPACK algorithms, so order and signs agree with NumPy in nearly all cases; the last bit of an eigenvector can still flip a sign (about 5 per cent of random non-symmetric matrices), degenerate cases are arbitrary, eigvals and eig of a matrix with only real eigenvalues return a real array where NumPy returns a complex one, and a singular matrix with a rounded zero pivot is not always detected by inv and solve (see Linear algebra above).
- Float sums use NumPy's pairwise summation, so they agree bit for bit. Matrix products and linear algebra do not use BLAS.
- numpy.random reproduces NumPy's sequences (see Random numbers above) except for the functions listed there as missing. numpy.ma, np.set_printoptions, np.save/np.load, ufunc objects (np.add.reduce) and np.vectorize are not available. numpy.fft is available (above).
- Integer overflow: in the C interpreter integer arrays wrap around at 64 bits as in NumPy; in the CREXX interpreter the overflowing result of an array multiplication turns into a float.
- Arrays cannot hold strings or ragged lists (a ValueError or TypeError is raised); a.item(i) takes no argument.

Chapter 7. pandas

import pandas as pd provides a subset of pandas written entirely in Python (the package pylib/pandas, about 12,800 lines) on top of the numpy subset of Chapter 6. Because it is ordinary Python source it runs unchanged on both interpreters, and it is also the largest program that the test suite runs. The package provides Series, DataFrame, Index, RangeIndex and MultiIndex, label and position based indexing, arithmetic with index alignment, missing-value handling, group-by aggregation, merging and reshaping, string and datetime accessors, Timestamp, Timedelta and DatetimeIndex, resampling, rolling windows, reading and writing CSV and JSON, and pandas' text layout for printing. The examples of this chapter were compared with the output of pandas 3.0.5 itself; the differences are listed at the end.

Series

A Series is a one-dimensional labelled array: a numpy array of values plus an Index of labels and an optional name.

```
import pandas as pd
import numpy as np
s = pd.Series([10, 20, 30, 40], index=["a", "b", "c", "d"], name="score")
print(s)
print(s["b"], s.iloc[2], s[["a", "d"]].tolist(), s["b":"c"].tolist(), s.index.tolist())
print(s + 1, s * s, (s > 15).sum(), s[s > 15].index.tolist(), s.mean(), s.max(),
    s.idxmax())
t = pd.Series({"x": 1.5, "y": np.nan, "z": 3.0})
print(t, t.isna().tolist(), t.fillna(0).tolist(), t.dropna().tolist(), t.sum(),
    t.count())
print(pd.Series([3, 1, 2]).sort_values().tolist(),
    pd.Series(list("abca")).value_counts().to_dict(), pd.Series([1, 2,
    3]).cumsum().tolist())
print(s.describe())

$ py prog.py
a    10
b    20
c    30
d    40
Name: score, dtype: int64
20 30 [10, 40] [20, 30] ['a', 'b', 'c', 'd']
a    11
b    21
c    31
d    41
Name: score, dtype: int64 a     100
b     400
c     900
d    1600
Name: score, dtype: int64 3 ['b', 'c', 'd'] 25.0 40 d
x    1.5
y    NaN
z    3.0
dtype: float64 [False, True, False] [1.5, 0.0, 3.0] [1.5, 3.0] 4.5 2
[1, 2, 3] {'a': 2, 'b': 1, 'c': 1} [1, 3, 6]
count     4.000000
mean     25.000000
std      12.909944
min      10.000000
25%      17.500000
50%      25.000000
75%      32.500000
max      40.000000
Name: score, dtype: float64
```

Figure 89. Creating and using a Series

DataFrame

A DataFrame is a table of named columns sharing one index. It is built from a dict of columns, a list of records (dicts) or a list of lists with columns=, and printed with pandas' own layout: index column on the left, right-aligned columns, NaN for missing values.

```
import pandas as pd
df = pd.DataFrame({
    "name": ["Ann", "Bob", "Cy", "Dee", "Eve"],
    "dept": ["ops", "dev", "dev", "ops", "dev"],
    "age": [31, 25, 47, 38, 29],
    "pay": [52.5, 61.0, 83.25, 57.0, 66.5],
})
print(df)
print(df.shape, list(df.columns), df.dtypes.to_dict())
print(df.head(2))
print(df.describe())
print(df["age"].mean(), df[["name", "pay"]].tail(2), df.loc[1, "name"], df.iloc[0, 2])
print(df[df["age"] > 30], df[(df.dept == "dev") & (df.pay > 62)])
print(pd.DataFrame([{"a": 1, "b": "x"}, {"a": 2}]), pd.DataFrame([[1, 2], [3, 4]],
    columns=["p", "q"]))

$ py prog.py
  name dept  age    pay
0  Ann  ops   31  52.50
1  Bob  dev   25  61.00
2   Cy  dev   47  83.25
3  Dee  ops   38  57.00
4  Eve  dev   29  66.50
(5, 4) ['name', 'dept', 'age', 'pay'] {'name': <StringDtype(storage='python',
    na_value=nan)>, 'dept': <StringDtype(storage='python', na_value=nan)>, 'age':
    dtype('int64'), 'pay': dtype('float64')}
  name dept  age   pay
0  Ann  ops   31  52.5
1  Bob  dev   25  61.0
             age        pay
count   5.000000   5.000000
mean   34.000000  64.050000
std     8.660254  11.906406
min    25.000000  52.500000
25%    29.000000  57.000000
50%    31.000000  61.000000
75%    38.000000  66.500000
max    47.000000  83.250000
34.0   name   pay
3  Dee  57.0
4  Eve  66.5 Bob 31
  name dept  age    pay
0  Ann  ops   31  52.50
2   Cy  dev   47  83.25
3  Dee  ops   38  57.00   name dept  age    pay
2   Cy  dev   47  83.25
4  Eve  dev   29  66.50
   a    b
0  1    x
1  2  NaN    p  q
0  1  2
1  3  4
```

Figure 90. Creating and inspecting a DataFrame

```
import pandas as pd
df = pd.DataFrame({"k": ["b", "a", "c", "a"], "v": [4, 1, 3, 2], "w": [0.5, 1.5, 2.5,
    3.5]})
df["z"] = df["v"] * 2 + df["w"]
df["big"] = df["v"] > 2
print(df)
print(df.sort_values("v"), df.sort_values(["k", "v"], ascending=[True,
    False]).index.tolist())
print(df.loc[df.v > 1, ["k", "z"]], df.iloc[1:3, :2],
    df.drop(columns=["big"]).columns.tolist())
print(df.rename(columns={"k": "key"}).columns.tolist(), df.set_index("k").loc["a"],
    df.reset_index(drop=True).shape)
print(df["v"].apply(lambda x: x ** 2).tolist(), df["k"].map({"a": 1, "b": 2}).tolist(),
    df.apply(lambda r: r["v"] + r["w"], axis=1).tolist())
print(df.T, df.values.shape, df.to_dict("records")[0], df.query("v > 2").shape,
    df.isin({"k": ["a"]}).sum().tolist())
for i, row in df.iterrows():
    if i < 2: print(i, row["k"], row["v"])
print(df.drop_duplicates("k").shape, df["k"].unique().tolist(), df["k"].nunique(),
    df.nlargest(2, "v")["v"].tolist())

$ py prog.py
   k  v    w    z    big
0  b  4  0.5  8.5   True
1  a  1  1.5  3.5  False
2  c  3  2.5  8.5   True
3  a  2  3.5  7.5  False
   k  v    w    z    big
1  a  1  1.5  3.5  False
3  a  2  3.5  7.5  False
2  c  3  2.5  8.5   True
0  b  4  0.5  8.5   True [3, 1, 0, 2]
   k    z
0  b  8.5
2  c  8.5
3  a  7.5    k  v
1  a  1
2  c  3 ['k', 'v', 'w', 'z']
['key', 'v', 'w', 'z', 'big']    v    w    z    big
k
a  1  1.5  3.5  False
a  2  3.5  7.5  False (4, 5)
[16, 1, 9, 4] [2.0, 1.0, nan, 1.0] [4.5, 2.5, 5.5, 5.5]
       0      1     2      3
k      b      a     c      a
v      4      1     3      2
w    0.5    1.5   2.5    3.5
z    8.5    3.5   8.5    7.5
big  True  False  True  False (4, 5) {'k': 'b', 'v': 4, 'w': 0.5, 'z': 8.5, 'big':
    True} (2, 5) [2, 0, 0, 0, 0]
0 b 4
1 a 1
(3, 5) ['b', 'a', 'c'] 3 [4, 3]
```

Figure 91. Selecting, adding, sorting and applying

Missing data

```
import pandas as pd
import numpy as np
df = pd.DataFrame({"a": [1.0, np.nan, 3.0, np.nan], "b": [np.nan, 2.0, 2.0, 4.0], "c":
    ["x", None, "z", "w"]})
print(df)
print(df.isna().sum().tolist(), df.dropna().shape, df.dropna(how="all", subset=["a",
    "b"]).shape)
print(df.fillna({"a": 0, "b": df["b"].mean(), "c": "?"}))
print(df["a"].ffill().tolist(), df["a"].bfill().tolist(),
    df["a"].interpolate().tolist())
print(df[["a", "b"]].mean(), df[["a", "b"]].sum(axis=1).tolist(), df[["a",
    "b"]].count().tolist())

$ py prog.py
     a    b    c
0  1.0  NaN    x
1  NaN  2.0  NaN
2  3.0  2.0    z
3  NaN  4.0    w
[2, 1, 1] (1, 3) (4, 3)
     a         b  c
0  1.0  2.666667  x
1  0.0  2.000000  ?
2  3.0  2.000000  z
3  0.0  4.000000  w
[1.0, 1.0, 3.0, 3.0] [1.0, 3.0, 3.0, nan] [1.0, 2.0, 3.0, 3.0]
a    2.000000
b    2.666667
dtype: float64 [1.0, 2.0, 5.0, 4.0] [2, 3]
```

Figure 92. NaN handling

Grouping and aggregation

groupby() splits a frame by one or more columns; aggregations are mean, sum, min, max, count, size, median, std, var, first, last, nunique, prod, cumsum, cumcount, rank, shift, diff, transform, apply, filter and agg() with a function name, a list of names, a dict of column to name(s), or named aggregation (NamedAgg).

```
import pandas as pd
df = pd.DataFrame({
    "dept": ["ops", "dev", "dev", "ops", "dev", "hr"],
    "team": ["a", "a", "b", "b", "b", "a"],
    "pay": [52.5, 61.0, 83.25, 57.0, 66.5, 49.0],
    "age": [31, 25, 47, 38, 29, 52],
})
g = df.groupby("dept")
print(g["pay"].mean(), g.size(), g["age"].max().to_dict())
print(g.agg({"pay": "mean", "age": ["min", "max"]}))
print(df.groupby(["dept", "team"])["pay"].sum())
print(g["pay"].transform(lambda s: s - s.mean()).round(2).tolist(), g.ngroups,
    list(g.groups))
print(g.agg(total=("pay", "sum"), oldest=("age", "max")))
print(df.groupby("dept")["pay"].apply(lambda s: s.max() - s.min()).to_dict(),
    g.get_group("hr").shape)

$ py prog.py
dept
dev    70.25
hr     49.00
ops    54.75
Name: pay, dtype: float64 dept
dev    3
hr     1
ops    2
dtype: int64 {'dev': 47, 'hr': 52, 'ops': 38}
        pay age
       mean min max
dept
dev   70.25  25  47
hr    49.00  52  52
ops   54.75  31  38
dept  team
dev   a        61.00
      b       149.75
hr    a        49.00
ops   a        52.50
      b        57.00
Name: pay, dtype: float64
[-2.25, -9.25, 13.0, 2.25, -3.75, 0.0] 3 ['dev', 'hr', 'ops']
       total  oldest
dept
dev   210.75      47
hr     49.00      52
ops   109.50      38
{'dev': 22.25, 'hr': 0.0, 'ops': 4.5} (1, 4)
```

Figure 93. groupby

Combining frames

```
import pandas as pd
emp = pd.DataFrame({"id": [1, 2, 3, 4], "name": ["Ann", "Bob", "Cy", "Dee"], "dept_id":
    [10, 20, 10, 30]})
dep = pd.DataFrame({"dept_id": [10, 20, 40], "dept": ["ops", "dev", "hr"]})
print(emp.merge(dep, on="dept_id"))
print(emp.merge(dep, on="dept_id", how="left"))
print(emp.merge(dep, on="dept_id", how="outer").shape, pd.merge(emp, dep, how="inner",
    left_on="dept_id", right_on="dept_id").shape)
print(pd.concat([emp.head(2), emp.tail(2)]).index.tolist(), pd.concat([emp[["id"]],
    emp[["name"]]], axis=1).shape)
long = pd.DataFrame({"d": ["m", "m", "t", "t"], "k": ["x", "y", "x", "y"], "v": [1, 2,
    3, 4]})
print(long.pivot_table(index="d", columns="k", values="v", aggfunc="sum"))
print(long.pivot(index="d", columns="k", values="v"))
wide = pd.DataFrame({"id": [1, 2], "x": [5, 6], "y": [7, 8]})
print(wide.melt(id_vars="id", var_name="var", value_name="val"))
print(pd.crosstab(long["d"], long["k"]))

$ py prog.py
   id name  dept_id dept
0   1  Ann       10  ops
1   2  Bob       20  dev
2   3   Cy       10  ops
   id name  dept_id dept
0   1  Ann       10  ops
1   2  Bob       20  dev
2   3   Cy       10  ops
3   4  Dee       30  NaN
(5, 4) (3, 4)
[0, 1, 2, 3] (4, 2)
k  x  y
d
m  1  2
t  3  4
k  x  y
d
m  1  2
t  3  4
   id var  val
0   1   x    5
1   2   x    6
2   1   y    7
3   2   y    8
k  x  y
d
m  1  1
t  1  1
```

Figure 94. merge, concat, pivot and reshaping

Strings, dates and windows

```
import pandas as pd
s = pd.Series(["  Alice Smith", "bob jones ", "Carol"])
print(s.str.strip().str.title().tolist(), s.str.len().tolist(),
    s.str.contains("o").tolist())
print(s.str.strip().str.split(" ").tolist(), s.str.upper().str.replace("O",
    "0").tolist())
print(pd.Series(["a1", "b22", "c"]).str.extract(r"([a-z])(\d+)"))
d = pd.to_datetime(pd.Series(["2026-10-11", "2026-12-25"]))
print(d.dt.year.tolist(), d.dt.month.tolist(), d.dt.day_name().tolist(), (d[1] -
    d[0]).days)
x = pd.Series([1, 2, 3, 4, 5, 6])
print(x.rolling(3).mean().tolist(), x.expanding().sum().tolist(), x.shift(1).tolist(),
    x.diff().tolist(), x.pct_change().round(3).tolist())
print(pd.cut(pd.Series([1, 5, 9]), bins=[0, 3, 6, 10]).astype(str).tolist(),
    pd.Series([3, 1, 2]).rank().tolist())

$ py prog.py
['Alice Smith', 'Bob Jones', 'Carol'] [13, 10, 5] [False, True, True]
[['Alice', 'Smith'], ['bob', 'jones'], ['Carol']] ['  ALICE SMITH', 'B0B J0NES ',
    'CAR0L']
     0    1
0    a    1
1    b   22
2  NaN  NaN
[2026, 2026] [10, 12] ['Sunday', 'Friday'] 75
[nan, nan, 2.0, 3.0, 4.0, 5.0] [1.0, 3.0, 6.0, 10.0, 15.0, 21.0] [nan, 1.0, 2.0, 3.0,
    4.0, 5.0] [nan, 1.0, 1.0, 1.0, 1.0, 1.0] [nan, 1.0, 0.5, 0.333, 0.25, 0.2]
['(0, 3]', '(3, 6]', '(6, 10]'] [3.0, 1.0, 2.0]
```

Figure 95. The str and dt accessors, rolling

Dates and times: Timestamp, Timedelta, DatetimeIndex

Dates are first-class. pd.Timestamp (a datetime with nanosecond resolution that prints like pandas), pd.Timedelta, pd.DatetimeIndex and the dtype datetime64 for Series and DataFrame columns are implemented in pylib/pandas/tslib.py and dtindex.py. pd.to_datetime converts strings (ISO forms, or a format= with the strptime codes), numbers with unit=, lists and Series, with errors="coerce" giving NaT; pd.date_range builds an index from start, end, periods and a frequency (D, B, h, min, s, W, MS, ME, QS, QE, YS, YE and multiples such as 3D or 15min); pd.Timedelta takes keywords or strings ("2 days 03:00:00"); pd.DateOffset and pd.offsets.MonthEnd and the like shift dates by calendar units. A Timestamp has year, month, day, hour, ..., weekday(), day_name(), month_name(), quarter, dayofyear, is_leap_year, is_month_end and the other predicates, floor/ceil/round, normalize, strftime, isoformat and replace, and subtracts to a Timedelta (days, seconds, total_seconds(), components). A datetime Series has the .dt accessor with the same attributes, plus strftime, floor, ceil, normalize and the arithmetic of timedeltas; a DatetimeIndex supports partial-string selection (s["2024-02"], s["2024-01-30":"2024-02-01"], df.loc["2024-04"]), slicing, shift, diff, union and intersection, groupby on dt parts, and df.resample(rule).agg() for the rules above with sum, mean, min, max, count, first, last, ohlc and size, as well as asfreq and rolling windows given as a number of rows or a time span (rolling("3D")). Time zones are not implemented: tz_localize raises NotImplementedError and a tz= argument is accepted and ignored. Period and PeriodIndex do not exist.

```
import pandas as pd
import numpy as np
ts = pd.Timestamp("2024-03-15 13:45:30")
print(ts, ts.year, ts.month, ts.day_name(), ts.weekday(), ts.strftime("%d/%m/%Y"),
    ts.floor("D"), ts + pd.Timedelta(days=20, hours=3))
td = pd.Timedelta(hours=36, minutes=30)
print(td, td.total_seconds(), td.days, pd.Timedelta("2 days 03:00:00"),
    pd.Timestamp("2024-03-01") - pd.Timestamp("2024-02-01"))
idx = pd.date_range("2024-01-29", periods=5, freq="D")
print(idx)
print(idx.dayofweek.tolist(), idx.month.tolist(), idx.strftime("%a %d").tolist())
print(pd.date_range("2024-01-31", periods=3, freq="ME").strftime("%Y-%m-%d").tolist(),
    pd.date_range("2024-01-01", "2024-01-10", freq="3D").strftime("%m-%d").tolist())
s = pd.Series([1.0, 2.0, 4.0, 8.0, 16.0], index=idx)
print(s)
print(s["2024-02-01"], s["2024-01-30":"2024-02-01"].tolist(), s.index[0])
df = pd.DataFrame({"when": pd.to_datetime(["2024-03-01", "2024-03-15", "2024-04-02",
    "2024-04-20"]), "v": [10, 20, 30, 40]})
print(df)
print(df.dtypes)
print(df["when"].dt.month.tolist(), df["when"].dt.day_name().tolist(),
    df["when"].dt.year.unique().tolist())
print(df.groupby(df["when"].dt.month)["v"].sum())
print(df[df["when"] > "2024-03-10"])
print((df["when"] - df["when"].iloc[0]).dt.days.tolist())
d = df.set_index("when")
print(d.resample("MS").sum())
print(d.loc["2024-04"])
print(pd.to_datetime("03/04/2024", format="%d/%m/%Y"), pd.to_datetime(["2024-01-01",
    None]).tolist())

$ py prog.py
2024-03-15 13:45:30 2024 3 Friday 4 15/03/2024 2024-03-15 00:00:00 2024-04-04 16:45:30
1 days 12:30:00 131400.0 1 2 days 03:00:00 29 days 00:00:00
DatetimeIndex(['2024-01-29', '2024-01-30', '2024-01-31', '2024-02-01',
               '2024-02-02'],
              dtype='datetime64[us]', freq='D')
[0, 1, 2, 3, 4] [1, 1, 1, 2, 2] ['Mon 29', 'Tue 30', 'Wed 31', 'Thu 01', 'Fri 02']
['2024-01-31', '2024-02-29', '2024-03-31'] ['01-01', '01-04', '01-07', '01-10']
2024-01-29     1.0
2024-01-30     2.0
2024-01-31     4.0
2024-02-01     8.0
2024-02-02    16.0
Freq: D, dtype: float64
8.0 [2.0, 4.0, 8.0] 2024-01-29 00:00:00
        when   v
0 2024-03-01  10
1 2024-03-15  20
2 2024-04-02  30
3 2024-04-20  40
when    datetime64[us]
v                int64
dtype: object
[3, 3, 4, 4] ['Friday', 'Friday', 'Tuesday', 'Saturday'] [2024]
when
3    30
4    70
Name: v, dtype: int64
        when   v
```

```
1 2024-03-15  20
2 2024-04-02  30
3 2024-04-20  40
[0, 14, 32, 50]
             v
when
2024-03-01  30
2024-04-01  70
             v
when
2024-04-02  30
2024-04-20  40
2024-04-03 00:00:00 [Timestamp('2024-01-01 00:00:00'), NaT]
```

Figure 96. Timestamps, date ranges, .dt and resample

Reading and writing

read_csv() reads a file name or a file-like object such as io.StringIO (options sep, header, names, index_col, dtype, usecols, nrows, skiprows, na_values, parse_dates); read_json() and read_table() are also available. to_csv() writes a file or returns a string; to_json(), to_dict(), to_string(), to_markdown() and to_numpy() convert. Column types are inferred (int64, float64, bool, object). Excel, SQL, parquet, HDF and clipboard formats are not available.

```
import io
import pandas as pd
text = "id,name,score\n1,Ann,3.5\n2,Bob,\n3,\"Cy, Jr\",4.25\n"
df = pd.read_csv(io.StringIO(text))
print(df, df.dtypes.to_dict())
print(df.to_csv(index=False), end="")
print(df.set_index("id").to_csv(), end="")
print(pd.read_csv(io.StringIO("a;b\n1;2\n"), sep=";").to_dict("list"),
    df.to_string().splitlines()[0])

$ py prog.py
   id    name  score
0   1     Ann   3.50
1   2     Bob    NaN
2   3  Cy, Jr   4.25 {'id': dtype('int64'), 'name': <StringDtype(storage='python',
    na_value=nan)>, 'score': dtype('float64')}
id,name,score
1,Ann,3.5
2,Bob,
3,"Cy, Jr",4.25
id,name,score
1,Ann,3.5
2,Bob,
3,"Cy, Jr",4.25
{'a': [1], 'b': [2]}    id    name  score
```

Figure 97. CSV and JSON

Printing

Series and DataFrame print with pandas' layout: right-aligned columns of equal width, the index on the left, the Name:, Length: and dtype: footer for a Series, NaN for missing values, floats with a common number of decimals, truncation with ... after 60 rows (display.max_rows) or when the frame is wider than 80 columns, which wraps the columns with a backslash. pd.set_option and pd.get_option are available for display.max_rows, display.max_columns, display.width and display.precision.

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.arange(24).reshape(2, 12) * 1001, columns=[f"column{i}" for i in
    range(12)])
print(df)
long = pd.Series(range(100))
print(long)
pd.set_option("display.max_rows", 6)
print(pd.DataFrame({"v": range(20)}))

$ py prog.py
   column0  column1  column2  column3  ...  column8  column9  column10  column11
0        0     1001     2002     3003  ...     8008     9009     10010     11011
1    12012    13013    14014    15015  ...    20020    21021     22022     23023

[2 rows x 12 columns]
0      0
1      1
2      2
3      3
4      4
      ..
95    95
96    96
97    97
98    98
99    99
Length: 100, dtype: int64
     v
0    0
1    1
2    2
..  ..
17  17
18  18
19  19

[20 rows x 1 columns]
```

Figure 98. Wide and long frames

Differences from real pandas

- Not implemented: Categorical and sparse types, timezone-aware datetimes (tz= is silently ignored) and Period/Interval types (to_period raises NotImplementedError), between_time, plotting, Excel/SQL/parquet IO, styler, eval with complex expressions, and the nullable extension dtypes (Int64, string[pyarrow]). Column dtypes are int64, float64, bool, object, datetime64 and timedelta64. A DatetimeIndex prints with dtype='datetime64[us]', as in pandas 3.
- String columns report the dtype str, as in pandas 3.
- to_json() separates items with ", " and ": " and writes NaN instead of null, and to_string(index=False) still prints the index; use to_csv() or to_dict() to export data.
- pandas is slow compared with the real library: every operation runs as interpreted Python on lists. Frames of a few thousand rows are comfortable; with the CREXX interpreter import pandas takes about two seconds on top of the start-up (see Chapter 11).
- Index alignment, groupby keys and sorting follow pandas' rules for the common cases; unusual combinations (multi-level columns, merges on indexes of different dtypes, sort stability for object keys) may differ.

Chapter 8. SciPy

import scipy provides a subset of SciPy written in Python (the package pylib/scipy, about 10,800 lines) on top of the numpy subset: the sub-packages stats, special, optimize, integrate, interpolate, linalg, spatial, signal, sparse, fft and constants. Like pandas it is the same Python source in both interpreters. The algorithms are the standard textbook ones (series and continued fractions for the special functions, Nelder-Mead, BFGS, Brent and Levenberg-Marquardt for optimisation, adaptive Gauss-Kronrod quadrature, Dormand-Prince Runge-Kutta, Jacobi and QR iterations for linear algebra), so results agree with SciPy to the precision of the method, usually to 10 or more significant digits; where the last digits matter the examples below round. The examples were compared with SciPy 1.18.

Import a sub-package either as from scipy import stats, optimize or as import scipy.stats. The names ndimage, io, cluster and odr do not exist.

scipy.stats

Name	Description
norm t chi2 f expon uniform gamma beta lognorm weibull_min laplace logistic cauchy	continuous distributions with pdf, cdf, ppf, sf, isf, logpdf, logcdf, rvs, mean, var, std, interval and stats; loc and scale are accepted, and frozen distributions norm(loc, scale)
binom poisson geom bernoulli randint	discrete distributions with pmf, cdf, ppf, sf, rvs, mean, var
describe skew kurtosis moment variation sem zscore mode iqr median_abs_deviation entropy	descriptive statistics
gmean hmean pmean tmean tmin tmax tstd tvar tsem trim_mean trimboth	means and trimmed statistics
scoreatpercentile percentileofscore rankdata	percentiles and ranks
pearsonr spearmanr kendalltau pointbiserialr linregress	correlation and simple regression
ttest_1samp ttest_ind ttest_ind_from_stats ttest_rel f_oneway mannwhitneyu chisquare chi2_contingency fisher_exact binomtest ks_2samp jarque_bera	hypothesis tests; they return a result object with statistic and pvalue (and df, confidence_interval and so on for some)

Figure 99. scipy.stats

```
import numpy as np
from scipy import stats
def show(*v):
    print(*[round(float(x), 8) for x in v])
show(stats.norm.cdf(1.96), stats.norm.pdf(0), stats.norm.ppf(0.975), stats.norm.sf(2))
show(stats.t.cdf(2.0, 10), stats.chi2.cdf(3.84, 1), stats.expon.cdf(1.0, scale=2),
    stats.binom.pmf(3, 10, 0.5), stats.poisson.cdf(2, 3.0))
show(stats.norm(loc=10, scale=2).cdf(12), stats.t.ppf(0.975, 5), stats.uniform.cdf(0.3),
    stats.gamma.pdf(2.0, 3.0))
x = np.array([2.1, 2.5, 3.6, 4.0, 4.4, 5.3, 6.0])
y = np.array([1.1, 2.2, 2.9, 4.1, 4.2, 6.1, 7.4])
show(stats.pearsonr(x, y)[0], stats.spearmanr(x, y)[0], stats.kendalltau(x, y)[0])
r = stats.ttest_ind(x, y)
show(r.statistic, r.pvalue)
show(stats.ttest_1samp(x, 3.0).pvalue, stats.sem(x), stats.skew(x), stats.kurtosis(x))
lr = stats.linregress(x, y)
show(lr.slope, lr.intercept, lr.rvalue, lr.pvalue)
show(stats.mode(np.array([1, 2, 2, 3, 3, 3])).mode, stats.gmean([1, 2, 4]),
    stats.iqr(x), stats.chisquare([16, 18, 16, 14, 12, 12]).pvalue)

$ py prog.py
0.9750021 0.39894228 1.95996398 0.02275013
0.96330598 0.94995648 0.39346934 0.1171875 0.42319008
0.84134475 2.57058184 0.3 0.27067057
0.9862862 1.0 1.0
-0.01450342 0.9886667
0.11311631 0.53158728 0.03052058 -1.16062231
1.53851709 -2.13208955 0.9862862 4.199e-05
3.0 2.0 1.8 0.84914504
```

Figure 100. Distributions and tests

scipy.special

Name	Description
gamma gammaln loggamma gammasgn rgamma digamma psi	gamma function and relatives
beta betaln betainc betaincinv gammainc gammaincc gammaincinv gammainccinv	beta function and incomplete beta / gamma functions
erf erfc erfcx erfinv erfcinv ndtr ndtri log_ndtr	error function and the normal distribution
comb perm binom factorial factorial2	combinatorics (exact for integers when exact=True)
expit logit softmax log_softmax logsumexp	logistic and softmax functions
xlogy xlog1py entr rel_entr kl_div boxcox inv_boxcox	information-theory helpers
exp2 exp10 expm1 log1p cbrt sindg cosdg	elementary functions
ellipk ellipe roots_legendre	elliptic integrals; Gauss-Legendre nodes and weights

Figure 101. scipy.special

```
from scipy import special as sp
def show(*v):
    print(*[round(float(x), 10) for x in v])
show(sp.gamma(0.5), sp.gammaln(10), sp.erf(1.0), sp.erfc(1.0), sp.erfinv(0.5))
print(sp.comb(10, 3), sp.comb(10, 3, exact=True), sp.perm(5, 2), sp.factorial(5),
    sp.factorial(10, exact=True))
show(sp.beta(2, 3), sp.betainc(2, 3, 0.5), sp.gammainc(2, 1.5), sp.expit(0.0),
    sp.logit(0.25))
show(sp.ndtr(1.0), sp.ndtri(0.975), sp.digamma(1.0), sp.xlogy(2, 3.0), sp.logsumexp([1,
    2, 3]))
print(sp.softmax([1.0, 2.0, 3.0]).round(8), round(sp.ellipk(0.5), 10))

$ py prog.py
1.7724538509 12.8018274801 0.8427007929 0.1572992071 0.4769362762
120.0 120 20.0 120.0 3628800
0.0833333333 0.6875 0.4421745996 0.5 -1.0986122887
0.8413447461 1.9599639845 -0.5772156649 2.1972245773 3.4076059644
[0.09003057 0.24472847 0.66524096] 1.8540746773
```

Figure 102. scipy.special

scipy.optimize

Name	Description
minimize(f, x0, method=..., jac=, bounds=, constraints=)	Nelder-Mead, BFGS (default for unconstrained problems), CG, Powell, L-BFGS-B and TNC (a projected BFGS), SLSQP, COBYLA and trust-constr (augmented Lagrangian); returns OptimizeResult with x, fun, success, nit, nfev, message
minimize_scalar brent golden bracket fminbound	one-dimensional minimisation
fmin fmin_bfgs fmin_cg fmin_powell fmin_l_bfgs_b	legacy minimizers
root root_scalar fsolve brentq brenth bisect ridder newton toms748	root finding
curve_fit least_squares leastsq nnls	nonlinear and linear least squares; curve_fit returns (popt, pcov)
linprog linear_sum_assignment	linear programming; assignment problem
approx_fprime check_grad Bounds LinearConstraint NonlinearConstraint	helpers

Figure 103. scipy.optimize

```
import numpy as np
from scipy import optimize as op
f = lambda p: (p[0] - 1) ** 2 + (p[1] - 2.5) ** 2 + 3
r = op.minimize(f, [0.0, 0.0])
print(np.round(r.x, 4), round(r.fun, 6), r.success)
rosen = lambda p: (1 - p[0]) ** 2 + 100 * (p[1] - p[0] ** 2) ** 2
print(np.round(op.minimize(rosen, [-1.2, 1.0], method="BFGS").x, 4))
print(op.brentq(lambda x: x ** 3 - 2 * x - 5, 2, 3), op.minimize_scalar(lambda x: (x -
    2) ** 2 + 1).x)
print(np.round(op.fsolve(lambda v: [v[0] + v[1] - 3, v[0] - v[1] - 1], [0.0, 0.0]), 6))
xs = np.linspace(0, 4, 20)
ys = 2.5 * np.exp(-0.7 * xs) + 0.1
popt, pcov = op.curve_fit(lambda x, a, b, c: a * np.exp(-b * x) + c, xs, ys, p0=[1, 1,
    0])
print(np.round(popt, 4))
print(op.root_scalar(lambda x: x * x - 2, bracket=[0, 2]).root, op.newton(lambda x: x *
    x - 2, 1.0))

$ py prog.py
[1.  2.5] 3.0 True
[1. 1.]
2.094551481542327 1.9999999999999998
[2. 1.]
[2.5 0.7 0.1]
1.4142135623731364 1.414213562373095
```

Figure 104. scipy.optimize

scipy.integrate and scipy.interpolate

Name	Description
quad dblquad tplquad nquad fixed_quad	adaptive and fixed Gauss quadrature; quad returns (value, error)
trapezoid cumulative_trapezoid simpson romb	integration of sampled data
solve_ivp(f, t_span, y0, method=, t_eval=, rtol=, atol=)	initial-value problems: RK45 (default) and RK23; other method names fall back to RK45, except DOP853, which raises ImportError; returns an OdeResult with t, y, nfev, success
odeint	legacy interface (rows are time points)

Figure 105. scipy.integrate

Name	Description
interp1d(x, y, kind=)	linear, nearest, previous, next, quadratic, cubic
CubicSpline PchipInterpolator pchip_interpolate CubicHermiteSpline PPoly BSpline make_interp_spline	splines and piecewise polynomials, with derivatives and integrals
UnivariateSpline InterpolatedUnivariateSpline	smoothing and interpolating splines
lagrange BarycentricInterpolator barycentric_interpolate	polynomial interpolation
RegularGridInterpolator	linear interpolation on a regular grid

Figure 106. scipy.interpolate

```
import numpy as np
from scipy import integrate, interpolate
print(integrate.quad(lambda x: np.exp(-x * x), 0, 1), integrate.quad(np.sin, 0,
    np.pi)[0])
print(integrate.dblquad(lambda y, x: x * y, 0, 1, 0, 2)[0], integrate.trapezoid([1, 2,
    3], dx=0.5), integrate.simpson([0, 1, 4, 9, 16], x=[0, 1, 2, 3, 4]))
sol = integrate.solve_ivp(lambda t, y: -2 * y, (0, 1), [1.0], rtol=1e-8, atol=1e-10)
print(sol.success, round(float(sol.y[0, -1]), 6), round(float(np.exp(-2)), 6))
f = interpolate.interp1d([0, 1, 2, 3], [0, 1, 4, 9])
print(f(1.5), f([0.5, 2.5]), interpolate.interp1d([0, 1, 2, 3], [0, 1, 8, 27],
    kind="cubic")(1.5).round(6))
cs = interpolate.CubicSpline([0, 1, 2, 3], [0, 1, 8, 27])
print(round(float(cs(1.5)), 6), round(float(cs(1.5, 1)), 6), round(float(cs.integrate(0,
    3)), 6))

$ py prog.py
(0.7468241328124271, 8.291413475940725e-15) 2.0
0.9999999999999999 2.0 21.333333333333332
True 0.135335 0.135335
2.5 [0.5 6.5] 3.375
3.375 6.75 20.25
```

Figure 107. Integration and interpolation

scipy.linalg and scipy.sparse

Name	Description
det inv pinv pinvh solve solve_triangular solve_banded lstsq norm	solving and inversion
lu lu_factor lu_solve qr cholesky cho_factor cho_solve svd svdvals diagsvd polar	factorisations
eig eigh eigvals eigvalsh	eigenvalue problems (real matrices; the eigenvalues are complex where needed)
expm logm sqrtm fractional_matrix_power	matrix functions
null_space orth matrix_balance	subspaces
block_diag circulant toeplitz hankel hilbert invhilbert pascal	special matrices
LinAlgError LinAlgWarning	exceptions

Figure 108. scipy.linalg

Name	Description
csr_matrix csc_matrix coo_matrix lil_matrix dok_matrix bsr_matrix csr_array csc_array coo_array	sparse formats with toarray(), todense(), nnz, shape, T, +, -, *, @, dot, multiply, sum, transpose, tocsr(), tocsc(), tocoo(), indexing
eye identity diags spdiags block_diag kron hstack vstack tril triu random	constructors
issparse isspmatrix isspmatrix_csr isspmatrix_csc isspmatrix_coo	type tests
sparse.linalg: spsolve cg inv norm expm eigsh svds	sparse linear algebra (spsolve solves a dense copy of the matrix)

Figure 109. scipy.sparse and scipy.sparse.linalg

```
import numpy as np
from scipy import linalg, sparse
from scipy.sparse import linalg as spl
A = np.array([[4.0, 1.0, 0.0], [1.0, 3.0, 1.0], [0.0, 1.0, 2.0]])
print(round(linalg.det(A), 6), np.round(linalg.solve(A, [1.0, 2.0, 3.0]), 6),
    np.round(linalg.eigvalsh(A), 6))
P, L, U = linalg.lu(A)
print(np.allclose(P @ L @ U, A), np.round(linalg.expm(np.zeros((2, 2))), 3).tolist(),
    round(linalg.norm(A), 6))
print(np.round(linalg.sqrtm(np.array([[4.0, 0.0], [0.0, 9.0]])), 6).tolist(),
    linalg.toeplitz([1, 2, 3]).tolist())
S = sparse.csr_matrix(A)
print(S.nnz, S.shape, S.toarray().tolist()[0], (S @ np.ones(3)).tolist(), S.T.nnz)
print(sparse.eye(3).toarray().tolist(), sparse.diags([1.0, 2.0,
    3.0]).toarray().tolist()[1])
print(np.round(spl.spsolve(S, np.array([1.0, 2.0, 3.0])), 6), sparse.issparse(S))

$ py prog.py
18.0 [0.222222 0.111111 1.444444] [1.267949 3.       4.732051]
True [[1.0, 0.0], [0.0, 1.0]] 5.744563
[[2.0, 0.0], [0.0, 3.0]] [[1, 2, 3], [2, 1, 2], [3, 2, 1]]
7 (3, 3) [4.0, 1.0, 0.0] [5.0, 5.0, 3.0] 7
[[1.0, 0.0, 0.0], [0.0, 1.0, 0.0], [0.0, 0.0, 1.0]] [0.0, 2.0, 0.0]
[0.222222 0.111111 1.444444] True
```

Figure 110. scipy.linalg and scipy.sparse

scipy.fft

scipy.fft (pylib/scipy/fft.py) is the scipy-style interface to the transforms of numpy.fft: fft, ifft, rfft, irfft, hfft, ihfft, fft2, ifft2, rfft2, irfft2, fftn, ifftn, rfftn and irfftn (the extra arguments overwrite_x and workers are accepted and ignored), plus the discrete cosine and sine transforms dct, idct, dst and idst for types 1 to 4 with norm=None (or "backward"), "ortho" or "forward", their n-dimensional forms dctn, idctn, dstn and idstn, and next_fast_len and prev_fast_len. Real input to fft gives a complex result as in SciPy. The signal functions freqz, hilbert, welch, periodogram and spectrogram use the same transforms.

```
import numpy as np
from scipy import fft as sfft, signal
y = np.array([1.0, 2.0, 3.0, 4.0])
print(np.round(sfft.fft(y), 3))
print(np.round(sfft.dct(y, norm="ortho"), 4), np.round(sfft.idct(sfft.dct(y,
    norm="ortho"), norm="ortho"), 4))
print(np.round(sfft.dct(y), 4), np.round(sfft.dst(y), 4), sfft.next_fast_len(13))

fs = 100.0
t = np.arange(200) / fs
x = np.sin(2 * np.pi * 10 * t) + 0.3 * np.sin(2 * np.pi * 25 * t)
f, pxx = signal.welch(x, fs, nperseg=64)
print(f[:5], int(np.argmax(pxx)), f[int(np.argmax(pxx))])
fr, tt, S = signal.spectrogram(x, fs, nperseg=64, noverlap=32)
print(S.shape, tt, fr[int(np.argmax(S[:, 0]))])
b = np.array([0.25, 0.5, 0.25])
w, h = signal.freqz(b, worN=5)
print(np.round(w, 4), np.round(np.abs(h), 4))
an = signal.hilbert(np.sin(2 * np.pi * 5 * np.arange(32) / 32))
print(an.dtype, np.round(np.abs(an)[:4], 4), np.round(an.imag[:4], 4))

$ py prog.py
[10.-0.j -2.+2.j -2.-0.j -2.-2.j]
[ 5.     -2.2304  0.     -0.1585] [1. 2. 3. 4.]
[20.     -6.3086  0.     -0.4483] [13.0656 -5.6569  5.412  -4.    ] 14
```

```
[0.     1.5625 3.125  4.6875 6.25  ] 6 9.375
(33, 5) [0.32 0.64 0.96 1.28 1.6 ] 9.375
[0.     0.6283 1.2566 1.885  2.5133] [1.     0.9045 0.6545 0.3455 0.0955]
complex128 [1. 1. 1. 1.] [-1.     -0.5556  0.3827  0.9808]
```

Figure 111. scipy.fft and scipy.signal spectra

scipy.spatial, scipy.signal and scipy.constants

Name	Description
distance.euclidean sqeuclidean cityblock chebyshev minkowski cosine correlation hamming jaccard canberra braycurtis dice mahalanobis seuclidean	distances between two vectors
distance.cdist pdist squareform is_valid_dm distance_matrix	distances between sets of points
KDTree cKDTree	nearest-neighbour search (query, query_ball_point, data), by exhaustive search

Figure 112. scipy.spatial

Name	Description
convolve correlate convolve2d correlate2d fftconvolve oaconvolve correlation_lags	convolution and correlation (fftconvolve and oaconvolve are aliases that compute directly, not through the FFT)
lfilter lfilter_zi filtfilt savgol_filter savgol_coeffs medfilt detrend	filtering
find_peaks peak_prominences peak_widths argrelextrema argrelmax argrelmin	peak finding
get_window windows.hann hamming blackman bartlett boxcar triang ...	window functions
sawtooth square unit_impulse	waveforms
freqz	frequency response of a digital filter: freqz(b, a=1, worN=512, whole=False, fs=2*pi)
hilbert	analytic signal (a complex array) through numpy.fft
periodogram welch spectrogram	power spectral density estimates; the arguments window, nperseg, noverlap, nfft, detrend, return_onesided and scaling ("density", "spectrum") are as in SciPy

Figure 113. scipy.signal

scipy.constants provides the CODATA values and unit conversions of SciPy: pi, golden, c, speed_of_light, h, hbar, G, g, e, R, k, Boltzmann, N_A, Avogadro, epsilon_0, mu_0, Stefan_Boltzmann, electron_mass, proton_mass, eV, the SI prefixes (kilo, mega, milli, micro, nano, ...), the binary prefixes (kibi, mebi, ...), length, mass, time, pressure, energy, power, force, temperature and angle units (inch, foot, mile, pound, hour, day, year, atm, bar, psi, calorie, hp, degree, arcmin, ...), the dictionary physical_constants with value(), unit() and precision(), and the functions convert_temperature, lambda2nu, nu2lambda and find.

```
import numpy as np
from scipy.spatial import distance, KDTree
from scipy import signal, constants
print(distance.euclidean([0, 0], [3, 4]), float(distance.cityblock([1, 2], [4, 6])),
    round(distance.cosine([1, 0], [1, 1]), 6))
pts = np.array([[0.0, 0.0], [1.0, 0.0], [0.0, 2.0]])
print(distance.cdist(pts, pts)[0].tolist(), distance.pdist(pts).round(4).tolist())
d, i = KDTree(pts).query([0.9, 0.1])
print(round(float(d), 6), int(i))
print(signal.convolve([1, 2, 3], [0, 1, 0.5]).tolist(), signal.lfilter([0.5, 0.5],
    [1.0], [1, 2, 3, 4]).tolist())
print(signal.find_peaks([0, 2, 1, 3, 1, 0])[0].tolist(),
    np.round(signal.get_window("hann", 5), 4).tolist())
print(constants.c, constants.pi, constants.kilo, constants.inch,
    constants.convert_temperature(100, "C", "F"))
print(constants.physical_constants["electron mass"][0], constants.eV, constants.hour,
    constants.golden)

$ py prog.py
5.0 7.0 0.292893
[0.0, 1.0, 2.0] [1.0, 2.0, 2.2361]
0.141421 1
[0.0, 1.0, 2.5, 4.0, 1.5] [0.5, 1.5, 2.5, 3.5]
[1, 3] [0.0, 0.3455, 0.9045, 0.9045, 0.3455]
299792458.0 3.141592653589793 1000.0 0.0254 212.0
9.1093837139e-31 1.602176634e-19 3600.0 1.618033988749895
```

Figure 114. scipy.spatial, scipy.signal and scipy.constants

Differences from real SciPy

- Only the sub-packages and functions listed in this chapter exist. Missing: ndimage, io, cluster, odr, sparse.csgraph, stats.rv_continuous subclassing, stats.gaussian_kde, stats.qmc and most distributions not listed above.
- Results are computed with the methods named above in double precision; the last one or two digits of probabilities, quantiles and optimiser results may differ from SciPy. Optimiser iteration counts and the exact points reached by Nelder-Mead differ.
- Random variates (rvs) use the generator of numpy.random and do not reproduce SciPy's sequences.
- Sparse matrices are stored in the format requested but many operations convert through a dense or coordinate form; they are intended for correctness, not for large problems.
- Linear algebra routines accept real double matrices only; they are slow on matrices larger than a few hundred rows.

Chapter 9. Errors and Exceptions

Errors in a Py program are exceptions. An exception that is not caught ends the program: the interpreter writes a traceback to standard error and exits with status 1. This chapter describes the exception classes, the text of the messages (which follow CPython's wording for the common cases, so that programs that print str(e) behave the same) and the format of the uncaught-exception report.

Uncaught exceptions and exit status

The report starts with the line Traceback (most recent call last): followed by a line that names the file and, in the C interpreter, the line number where the error was raised, and ends with the exception class and message. The report is simplified: it does not list every frame and does not print the source text. The CREXX interpreter prints the same last line, but the file line says line 1 in tracebacks and also lists the function frames it knows of. The exit status is 1 for an uncaught exception, 2 when the CREXX launcher cannot compile the interpreter, and the argument of sys.exit() (or 0) otherwise.

```
def parse(s):
    return int(s)

def load(items):
    return [parse(x) for x in items]

print("start")
print(load(["1", "2", "three"]))
print("not reached")

$ py prog.py
start
Traceback (most recent call last):
  File "prog.py", line 2
ValueError: invalid literal for int() with base 10: 'three'
```

Figure 115. An uncaught exception

A call to sys.exit(n) or exit(n) raises SystemExit; with an integer the interpreter exits with that status, with another object it prints it to standard error and exits with status 1. SyntaxError and IndentationError are raised while a file is being read, before anything runs; the message gives the line number.

```
import sys
try:
    compile_me = eval("1 +")
except SyntaxError as e:
    print("SyntaxError caught")
try:
    exec("if True:\nprint(1)")
except SyntaxError as e:
    print("bad indentation caught")
try:
    sys.exit(3)
except SystemExit as e:
    print("SystemExit", e.args)
sys.exit(7)

$ py prog.py
SyntaxError caught
bad indentation caught
SystemExit (3,)
```

Figure 116. Syntax errors, sys.exit and the exit status

The exception hierarchy

Every class listed in the figure below exists in both interpreters except where a mark says otherwise (C means only the C interpreter has it; X means only the CREXX interpreter). The hierarchy and the names are those of CPython.

Class	Raised when
BaseException	root of all exceptions
SystemExit	sys.exit() or exit() is called
KeyboardInterrupt	the user interrupts the program (defined; not raised by the interpreters)
GeneratorExit	a generator is closed
Exception	root of the ordinary exceptions
StopIteration	next() on an exhausted iterator; .value holds the return value of a generator
StopAsyncIteration	defined for compatibility
ArithmeticError	base of the arithmetic errors
FloatingPointError	defined for compatibility
OverflowError	a result is too large (for example float(10**400), math.exp(1000), a bit shift by more than 2**63 places)
ZeroDivisionError	division by zero, integer division or modulo by zero, integer modulo by zero, float division by zero (also 0 ** -1)
AssertionError	an assert statement fails
AttributeError	an attribute is not found
BufferError	defined for compatibility
EOFError	input() hits the end of the file
ImportError	an import fails
ModuleNotFoundError	no module of that name
LookupError	base of IndexError and KeyError
IndexError	sequence index out of range
KeyError	mapping key not found
MemoryError	allocation failed
NameError	a name is not defined
UnboundLocalError	a local variable is used before assignment
OSError (IOError)	an operating-system or file error
FileNotFoundError FileExistsError PermissionError IsADirectoryError NotADirectoryError TimeoutError	subclasses of OSError for file operations
ConnectionError	base of ConnectionRefusedError, ConnectionResetError, ConnectionAbortedError and BrokenPipeError, which the socket module raises; BlockingIOError and InterruptedError are further subclasses of OSError
ReferenceError SystemError C	defined in the C interpreter only
RuntimeError	generic run-time error
NotImplementedError	a method is not implemented
RecursionError	the recursion limit is exceeded
SyntaxError	invalid source text
IndentationError	invalid indentation
TabError C	mixed tabs and spaces (C interpreter only)
TypeError	an operation is applied to an object of the wrong type
ValueError	an argument has the right type but an unsuitable value
UnicodeError	base of the Unicode errors
UnicodeDecodeError UnicodeEncodeError	defined for compatibility

Class	Raised when
Warning	base of the warnings: UserWarning, DeprecationWarning, RuntimeWarning, SyntaxWarning, FutureWarning, PendingDeprecationWarning, ImportWarning, UnicodeWarning, BytesWarning, ResourceWarning

Figure 117. Exception classes (indentation shows inheritance)

BaseExceptionGroup and ExceptionGroup exist (since Build 9; the attributes message and exceptions, split and subgroup with a type or tuple of types, and str() as "msg (2 sub-exceptions)") together with try/except *(a plain exception is wrapped in an ExceptionGroup for the handler; what a handler raises is grouped with the unhandled rest; yield inside an except* block is not supported in either interpreter). Not available: UnicodeTranslateError, EncodingWarning and PythonFinalizationError in either interpreter; ChildProcessError, ProcessLookupError and EnvironmentError in the C interpreter. The socket module raises the OSError subclasses ConnectionError, ConnectionRefusedError, ConnectionResetError, ConnectionAbortedError, BrokenPipeError, BlockingIOError, InterruptedError and TimeoutError, which exist in both interpreters; OSError(errno, message) has the attributes errno and strerror and prints as [Errno 111] message, but OSError(2, "x") does not turn itself into FileNotFoundError as it does in CPython.

Exception objects

An exception instance has args (the tuple of constructor arguments), str(e) (the message, or the arguments joined when there are several), repr(e) (such as ValueError('bad')) and, for KeyError, the key as the message in quotes. The attributes name, obj, errno, strerror, filename, code, value and msg exist on the classes that CPython gives them to: StopIteration.value, SystemExit.code, and so on. In the C interpreter (since Build 10) raise ... from ... sets __cause__ and __suppress_context__, an exception raised in a handler gets __context__, __traceback__ is a chain of traceback objects (tb_frame, tb_lineno, tb_next; the frame has f_code, f_lineno, f_globals, f_locals and f_back), with_traceback() exists, sys.exc_info() returns the traceback and sys._getframe() returns the caller frame; traceback.format_exc(), extract_tb(), extract_stack() and TracebackException use them (without the ~~^~~ marker lines of CPython 3.13), inspect.signature() works on functions, methods and classes, and logging fills funcName, lineno and pathname. The CREXX interpreter has them since Build 10, except with_traceback() and add_note(): raise ... from ... sets __cause__ and __suppress_context__, an exception raised in a handler gets __context__, __traceback__ is built from the frames the exception passed through (tb_frame.f_code.co_name, tb_lineno, tb_next), sys.exc_info() and sys._getframe() work, and so do traceback.format_exc(), extract_tb(), inspect.signature(), inspect.stack() and the funcName and lineno of logging. The line of a statement that spans several lines is its last line there.

```
class AppError(Exception):
    """base class for this program"""

class NotFound(AppError):
    def __init__(self, key):
        super().__init__("no such key: " + repr(key))
        self.key = key

def find(d, k):
    try:
        return d[k]
    except KeyError:
        raise NotFound(k)

try:
    find({"a": 1}, "b")
except AppError as e:
    print(type(e).__name__, e, e.key, e.args, repr(e))

try:
    try:
        int("x")
    except ValueError:
        print("logging")
        raise
except Exception as e:
    print("re-raised:", type(e).__name__)

def f():
    raise ValueError("a", 2)
try:
    f()
except ValueError as e:
    print(e.args, str(e))
e = ValueError("one")
print(e.args, repr(e), isinstance(e, Exception), str(KeyError("k")),
    repr(StopIteration(3)))
print(issubclass(FileNotFoundError, OSError), issubclass(KeyError, LookupError),
    issubclass(ZeroDivisionError, ValueError))

$ py prog.py
NotFound no such key: 'b' b ("no such key: 'b'",) NotFound("no such key: 'b'")
logging
re-raised: ValueError
('a', 2) ('a', 2)
('one',) ValueError('one') True 'k' StopIteration(3)
True True False
```

Figure 118. Raising, re-raising and defining exceptions

Messages of the common errors

The program below provokes each of the common run-time errors and prints the class and message. The messages are those of CPython 3.13 for all the cases shown, except the ones noted afterwards.

```
def show(f):
    try:
        f()
    except Exception as e:
        print(type(e).__name__ + ":", e)

show(lambda: 1 / 0)
show(lambda: 1 // 0)
show(lambda: 5 % 0)
show(lambda: 1.5 / 0)
show(lambda: [1, 2, 3][10])
show(lambda: (1, 2)[5])
show(lambda: "abc"[3])
show(lambda: {"a": 1}["b"])
show(lambda: int("abc"))
show(lambda: float("x.y"))
show(lambda: undefined_name)
show(lambda: "a" + 1)
show(lambda: 1 + "a")
show(lambda: [1] + (2,))
show(lambda: len(5))
show(lambda: None.foo)
show(lambda: (1).bar)
show(lambda: "x".nope())
show(lambda: int(None))
show(lambda: [].pop())
show(lambda: {}.pop("k"))
show(lambda: [1, 2].index(9))
show(lambda: "abc".index("z"))
show(lambda: list(5))
show(lambda: 5())
show(lambda: "abc"(1))
show(lambda: sorted([1, "a"]))
show(lambda: 3 < "a")
show(lambda: abs("x"))
show(lambda: -"x")
show(lambda: {[1]: 2})
show(lambda: {1, [2]})
show(lambda: next(iter([])))
show(lambda: iter(5))
show(lambda: max([]))
show(lambda: min())
show(lambda: chr(-1))
show(lambda: "a" * "b")
show(lambda: [1, 2, 3].remove(7))
show(lambda: round("x"))
def one(x):
    return x
show(lambda: one())
show(lambda: one(1, 2))
show(lambda: one(y=1))
show(lambda: "%d" % "x")
show(lambda: "{} {}".format(1))
show(lambda: "{0.a}".format(1))
show(lambda: open("/nonexistent/file"))
show(lambda: __import__("no_such_module"))
x, y = 1, 2
def unpack():
    a, b = [1, 2, 3]
show(unpack)
```

```
def unpack2():
    a, b, c = [1, 2]
show(unpack2)
def recurse(n):
    return recurse(n + 1)
show(recurse)
def local():
    print(v)
    v = 1
show(local)

$ py prog.py
ZeroDivisionError: division by zero
ZeroDivisionError: integer division or modulo by zero
ZeroDivisionError: integer modulo by zero
ZeroDivisionError: float division by zero
IndexError: list index out of range
IndexError: tuple index out of range
IndexError: string index out of range
KeyError: 'b'
ValueError: invalid literal for int() with base 10: 'abc'
ValueError: could not convert string to float: 'x.y'
NameError: name 'undefined_name' is not defined
TypeError: can only concatenate str (not "int") to str
TypeError: unsupported operand type(s) for +: 'int' and 'str'
TypeError: can only concatenate list (not "tuple") to list
TypeError: object of type 'int' has no len()
AttributeError: 'NoneType' object has no attribute 'foo'
AttributeError: 'int' object has no attribute 'bar'
AttributeError: 'str' object has no attribute 'nope'
TypeError: int() argument must be a string, a bytes-like object or a real number, not
    'NoneType'
IndexError: pop from empty list
KeyError: 'k'
ValueError: 9 is not in list
ValueError: substring not found
TypeError: 'int' object is not iterable
TypeError: 'int' object is not callable
TypeError: 'str' object is not callable
TypeError: '<' not supported between instances of 'str' and 'int'
TypeError: '<' not supported between instances of 'int' and 'str'
TypeError: bad operand type for abs(): 'str'
TypeError: bad operand type for unary -: 'str'
TypeError: unhashable type: 'list'
TypeError: unhashable type: 'list'
StopIteration:
TypeError: 'int' object is not iterable
ValueError: max() iterable argument is empty
TypeError: min expected at least 1 argument, got 0
ValueError: chr() arg not in range(0x110000)
TypeError: can't multiply sequence by non-int of type 'str'
ValueError: list.remove(x): x not in list
TypeError: type str doesn't define __round__ method
TypeError: one() missing 1 required positional argument: 'x'
TypeError: one() takes 1 positional argument but 2 were given
TypeError: one() got an unexpected keyword argument 'y'
TypeError: %d format: a real number is required, not str
IndexError: Replacement index 1 out of range for positional args tuple
AttributeError: 'int' object has no attribute 'a'
FileNotFoundError: [Errno 2] No such file or directory: '/nonexistent/file'
```

```
ModuleNotFoundError: No module named 'no_such_module'
ValueError: too many values to unpack (expected 2)
ValueError: not enough values to unpack (expected 3, got 2)
TypeError: recurse() missing 1 required positional argument: 'n'
UnboundLocalError: cannot access local variable 'v' where it is not associated with a
    value
```

Figure 119. Common error messages

Where the two interpreters differ from CPython in the wording, the behaviour is the same, only the message differs; the cases are listed in Appendix E. In particular the message of an uncaught SyntaxError names the line but not the offending source text, and the RecursionError message is maximum recursion depth exceeded.

Warnings and the standard error stream

warnings.warn(message, category) writes a line of the form file:line: Category: message to standard error (once per location), subject to the filters set with simplefilter() and filterwarnings(); catch_warnings(record=True) collects them. Anything written to sys.stderr, including the tracebacks, never mixes with standard output, which is the stream the test suite compares.

Chapter 10. How Python/CREXX Works

This chapter is for readers who want to read, port or extend the interpreters. Both implementations have the same architecture, a tree-walking interpreter of an abstract syntax tree; they differ in how they represent values and in how they manage memory. The description of the C interpreter comes first, then what is different in the CREXX interpreter.

Overview

```
source text
   |  lexer       p2_lex.c / b_lex.crexx      tokens, with INDENT and DEDENT
   v
   |  parser      p3_parse.c / c_parse.crexx  abstract syntax tree
   v
   |  resolver    p4_res.c / c_parse.crexx    local, cell, free and global slots
   v
   |  evaluator   p5_eval.c / g_eval.crexx    walks the tree; calls, classes, generators
   v
   builtins and methods  p7_bi.c, p7_fmt.c, p7_mod.c, p7_np*.c ...
   libraries in Python   pylib/*.py  (loaded through the import system by the same
       pipeline)
```

Figure 120. The stages of a run

The lexer

The lexer turns the text into a flat array of tokens: names, keywords, numbers, strings, operators, NEWLINE, INDENT and DEDENT. At the start of each logical line it measures the indentation (a tab advances to the next multiple of 8) and compares it with a stack of the enclosing indentation levels: a larger value pushes the stack and produces INDENT, a smaller one pops until it matches and produces one DEDENT for each level popped, and a value that matches no level is the error "unindent does not match any outer indentation level". Blank lines and comment-only lines are skipped. Inside brackets (a depth counter is kept) and after a backslash a newline is not a token, so expressions can continue on the next line. String literals support the prefixes r, b, f and their combinations, triple quotes, the escape sequences and implicit concatenation; an f-string is split into literal parts and expression parts, each expression being parsed by a recursive call of the parser, so nested quotes and format specs work. Integer literals that do not fit a machine integer become big integers.

The parser

The parser is a hand-written recursive-descent parser with one function per precedence level (lambda, conditional, or, and, not, comparison, bit-or, xor, and, shift, arithmetic, term, unary, power, primary). It builds Node structures (a kind, an operator, up to four child pointers, a list of children, a constant value, a name and the cached results of attribute lookup). Statement kinds are S_IF, S_WHILE, S_FOR, S_DEF, S_CLASS, S_TRY and so on; expression kinds are N_CALL, N_BINOP, N_COMP and so on. Several constructs are rewritten into simpler ones while parsing: a comprehension becomes a function whose body is nested for and if statements ending in an S_LISTAPP, S_SETADD or S_DICTSET statement; a generator expression becomes a generator function; augmented assignment, chained comparison and the with statement are expanded into their primitive operations. Type annotations are parsed and discarded (the class-body annotations that dataclasses need are recovered by pylib/_clsinfo.py, which rereads the source text of the class).

Scope resolution

After a function body has been parsed, the resolver decides for every name whether it is a local (a numbered slot in the frame), a cell variable (a local that an inner function uses, so it lives in a heap cell shared with the closure), a free variable (a cell taken from the enclosing function), a global (an entry in the global table) or an entry in a class namespace. All globals of all modules live in one hash table; to give each module its own namespace every global name is prefixed with the name of its module (the global x of module pkg.mod is the

entry pkg.mod.x), and builtins use the plain name. The prefix is a property of the code being run, so a function always sees the globals of the module that defined it. Local variable access is therefore an index into an array, with no hashing.

The evaluator

eval(frame, node) returns the Value of an expression and exec(frame, node) executes a statement and returns a status code: normal, break, continue, return or yield. A function call creates a Frame, binds the arguments to the first slots (positional, defaults, *args, keyword-only,* *kwargs) and executes the body; the value stack and the frame records are preallocated arrays. Attribute lookup on instances and classes uses an inline cache stored in the node (the class and a global class-change counter), so a method call in a loop does not search the MRO each time. Operators on numbers are special-cased before the general protocol; the general protocol looks up the special methods (__add__ and the reflected __radd__, comparison methods and NotImplemented, __getitem__, __iter__, ...) on the class. Classes are objects with a namespace, a list of bases and an MRO computed by C3 linearisation; super() proxies walk that list. Exceptions are implemented with setjmp and longjmp: try pushes a handler record, raise stores the exception object and jumps to the nearest handler, which restores the value stack and frame pointer, and finally clauses run through the same mechanism. The recursion limit is 1000 frames and raises RecursionError.

Generators

A generator does not get its own C stack. When a generator function is called, the interpreter allocates a Gen object that holds the local variables and, for every compound statement in the function body, a few state slots (the loop iterator, the index of the statement being run in a block, the progress through a try, and so on). Calling next() re-enters the function body from the top: each compound statement looks at its state slots, skips to the statement that was running when the generator last yielded, and continues. A yield stores the value in the generator and returns the status ST_YIELD through all the enclosing statements, each of which leaves its state in place. This is why yield inside while, for, if, try, finally and with all work, and why send(), throw() and close() are straightforward (the sent value is read where the yield expression resumes, and a thrown exception is raised at that point). The cost is that generator resumption re-walks the path from the top of the function to the yield; the CREXX interpreter keeps an explicit statement stack instead (see below).

Integers and floats

A Value is a 16-byte tagged union: a type tag and a long, a double or a heap pointer. int values are a long while they fit in 18 decimal digits (9 where long has 32 bits); any arithmetic result outside that range, and any literal that is too large, becomes a Big: a sign and an array of unsigned short limbs in base 10000. The decimal base makes conversion to and from text, the most common operation on large numbers in a program, a linear-time copy of the limbs; addition, subtraction, comparison and multiplication by a small number are linear and general multiplication and division use the schoolbook algorithms (quadratic). Floats are C doubles. The repr of a float is the shortest decimal string that reads back as the same double: the interpreter formats the number with 1, 2, 3 ... 17 significant digits using the C library and stops at the first one that strtod() reads back exactly, then lays the digits out by Python's rules (fixed notation between 1e-4 and 1e16, exponent form otherwise). sum() of floats uses compensated summation as CPython 3.12 does, and round() rounds half to even.

Memory management in the C interpreter

Every heap object starts with a header (the link in the list of all objects, the type, a mark bit). The collector is a stop-the-world mark-and-sweep collector: the roots are the value stack, the global table, the frames, the current exception and the permanent objects (classes and builtin types); marking uses an explicit gray stack instead of recursion so that long lists and deep structures cannot overflow the C stack; sweeping walks the object list and frees the unmarked objects. A collection is started at a safe point (entry to a call, the top of a loop iteration) when the bytes allocated since the last collection exceed the size of the live heap; it never happens in the middle of a C function, so native code may keep raw pointers to Values as long as it does not call back into the evaluator. -DPY_GC_STRESS collects at every safe point; the test suite passes with it.

The tables that the interpreter grows with realloc (globals, value stack) are never indexed through a pointer that is held across an allocation; the memory errors that were found in the preparation of this manual, in module attribute assignment, were of exactly that kind and have been fixed.

Strings

A str is an immutable array of bytes with a length and a hash. Names (identifiers, attribute names, keyword argument names) are interned so that comparing them is a pointer comparison. The character tests (isalpha, isdigit, ...) work on ASCII only, and the case conversions know the two-byte Latin letters (upper() turns é into É but title() and isalpha() get it wrong). The CREXX interpreter's strings are CREXX strings and so are sequences of characters; positions, lengths and slices count code points.

The pure-Python library layer

About 36,800 lines of Python in 68 files in pylib implement the standard library modules (9,400 lines in 32 files), the less common parts of numpy including the complex array class, the FFT, linear algebra and random numbers (3,800), pandas (12,800) and SciPy (10,800), and crexx/pylib adds the 490 lines of the CREXX socket and select modules. They are ordinary modules: the interpreters compile them like user programs, cache them in sys.modules, and run them with the same evaluator. The libraries are therefore a test of the interpreters: pandas, with its 12,800 lines of classes, generators, special methods and string formatting, runs unchanged on both. Native modules and the library can be combined ("hybrid" modules): when a module such as numpy is native, the interpreter first looks a name up in the native module and, if it is not there, in the pure-Python file of the same name (pylib/numpy/__init__.py), loaded on demand. The CREXX interpreter does this for math, random, sys, os, typing, warnings, abc and numpy.linalg and numpy.random as well; the C interpreter does it for numpy only. A module that is native in the C interpreter and Python in the CREXX interpreter (re, json, cmath, struct) is simply found as a file by the CREXX interpreter, and in the C interpreter the native module wins over the file of the same name. The names that the library should replace even though the native module has them (functools.wraps and the lru_cache decorators in CREXX) are listed in a table in the sources. The submodules of the native numpy module (numpy.fft, scipy.fft) are found because the native module has a package path (the search path entries joined with numpy), so that import numpy.fft, from numpy import fft and from numpy.fft import fft all load pylib/numpy/fft.py.

Native regular expressions and JSON in the C interpreter

src/p7_re.c (3,100 lines) holds the C interpreter's regular-expression engine. The tokenizer and parser are a port of CPython's re._parser, so the parse tree, the error messages and the error positions are CPython's; the tree is compiled into a flat array of instructions (literal, class, any, group start and end, repeat, alternation, assertions, backreferences, conditionals) and executed by a virtual machine that keeps its backtrack choices on an explicit stack of at most six million entries. A search therefore uses a constant amount of C stack whatever the length of the subject, and a pathological pattern ends with RuntimeError instead of a crash. Compiled patterns are cached by their text and flags. The result is about 100 times faster than the pure-Python engine that the CREXX interpreter uses (Chapter 11). src/p7_json.c (1,100 lines) is a hand-written scanner and encoder: dumps writes floats with the same shortest-repr routine as repr(), loads builds the objects directly, and the decoder reports errors with CPython 3.13's messages and character positions.

Complex numbers, bytes and subclasses of built-in types

complex is a native heap type holding two doubles (T_COMPLEX in C, K_COMPLEX with parts in ob_f and ob_g in CREXX). Division and powers follow CPython's algorithms (Smith's division, the polar form for non-integer powers), so the last digits agree. cmath in the C interpreter is src/p7_cplx.c, which also contains static replacements for the C99 libm functions that C89 lacks (hypot, log1p, expm1, asinh, acosh, atanh); numpy's complex128 is a fourth element type of the native array (interleaved doubles). In the CREXX interpreter a complex array is an object of the Python class _CArr (pylib/numpy/_cplx.py), which holds two float arrays; the evaluator sends every operation that has a complex array operand to it. bytes and bytearray (T_BYTES and T_BARR in C, K_BYTES with one character per byte in CREXX) have their own indexing, slicing, comparison and method tables, and src/p7_bytes.c also holds struct and int.to_bytes. A subclass of a

built-in type is an ordinary instance that carries the native value built by the base type's constructor (the field pay of the instance in C, the object itself in CREXX with the class id in ob_c) together with an instance dictionary; when the interpreter performs len(), iteration, indexing, comparison, hashing, formatting or an operator it first looks for an override on the subclass, and calling the base-class method explicitly bypasses the override once. type.__call__ is implemented as the CPython protocol: it calls cls.__new__(cls, ...) and then __init__ if the result is an instance of cls.

Amalgamation

The C sources are 24 files in c/src (py.c and 23 parts, 33,800 lines), numbered in dependency order: p0_os (the standard headers and every dependency on the host system, see Chapter 13), p0_base (types, memory, collector, strings, lists, dictionaries, sets), p1_num (big integers and float formatting), p2_lex, p3_parse, p4_res, p6_obj (attributes, equality, hashing, comparison, slicing, operators), p5_eval, p7_fmt (the format mini-language), p7_bi (builtins and methods), p7_bytes (bytes, bytearray, struct), p7_cplx (complex, cmath), p7_np, p7_np2, p7_la, p7_rnd (numpy.random), p7_pyrand (random), p7_imp (imports, eval, exec, os), p7_mod (math, sys, time, the exception hierarchy), p7_re (re), p7_json (json), p7_sock (socket, select) and p8_main (command line and REPL). py.c includes them in order, and tools/amalg.sh replaces each #include by the text of the file to produce c/py_all.c, one self-contained translation unit that is checked in and is what the Makefile compiles. Two further generated files serve the port to VM/370: src/pylib_embed.c, the whole of pylib as C arrays of byte values, which tools/amalg.sh embed adds to give py_embed_all.c, and the 71-column versions of both, deck/py_deck.c and deck/py_deck_embed.c (Chapter 13). Because every function is static and the file is one unit, a compiler can inline across the whole interpreter and a target system needs to compile only one file.

The CREXX interpreter

The CREXX interpreter follows the same stages in 23 source files in crexx/src (21,150 lines), which crexx/build.py concatenates into crexx/py.crexx after replacing symbolic names from consts.txt, classes.txt and builtins.txt by numbers (that is how an operation or class gets its constant, such as K_LIST or BI_NUMPY_LINSPACE). CREXX Level B has no records and no pointers, so the heap model is a set of parallel integer arrays, indexed by an object number:

- **Values are tagged integers.** A small int n is stored as 2n+1 (an odd integer, so no storage is allocated); every other value is the even integer 2*i where i is the index of an object. None, True, False, Ellipsis and NotImplemented are objects 1 to 5.
- **Objects are rows in parallel arrays.** ob_k holds the kind (K_STR, K_LIST, K_INST, ...), ob_c the class, ob_n and ob_x counts, ob_p and ob_y offsets into the shared slot array sl, ob_s the string for string objects and ob_f the value for float objects; lists, tuples, dictionaries and instances keep their items in sl. Array objects of numpy keep integer data in sl and floating-point data in a separate float store npf.
- **The tree is also integer arrays.** The parser produces node arrays (kind, children, constants) and the evaluator walks integer node numbers; scope resolution assigns frame slots as in C. Frames are slices of one array vp and are recycled through free lists by size.
- **Generators keep an explicit statement stack** in the heap: each compound statement that is active pushes its resume information, and a resumed generator replays the stack to the yield point.
- **Memory is reclaimed by a mark-and-sweep collector** (src/m_gc.crexx, described below). Frames are recycled, small integers cost nothing, and a loop that runs 300,000 iterations of a float calculation now stays at about 100 MB (about 860 MB with the collector switched off).
- **Big integers** are decimal strings of limbs; floats print with the shortest round-trip representation computed by the interpreter's own routines.
- **Start-up.** The 537,000-character source compiles to 3.1 MB of bytecode that the virtual machine must load and link before the interpreter's own initialisation runs (building the class table, the built-in names and the native-function tables). With a stock CREXX virtual machine that took about two seconds, 58 per cent of them in the link step's rebuild of the interface registries; with the patched virtual machine described in Chapter 2 (vm-startup.patch, selected with CREXX_VM) it takes about 0.4 seconds.

The numpy engine of the CREXX interpreter (j_np and k_np2) is a port of the C engine: the same dtype rules, broadcasting, the same print layout and the same pairwise summation, so that results agree in the last

digit. The bytes, complex, subclassing and socket parts are l_bytes, l_cplx, l_sub and l_sock (the last is only the primitive that crexx/pylib/socket.py calls, to reach the rxsocket library).

Memory management in the CREXX interpreter

Up to Build 2 the CREXX interpreter never freed an object. Build 3 adds a garbage collector, src/m_gc.crexx (about 490 lines). Its design in short:

- **Young objects are recorded.** While a window is open, that is a loop or a function call, every object that is born is entered in the young list yl. A window ends at the end of a loop iteration and just before a function returns. At that point, if enough objects (or enough numpy floating-point storage, counted as npw) have been allocated since the last collection, the collector runs.
- **Mark from the roots, sweep the young.** gc_run marks from the frame stack vp, the argument stack, the interned tables, the class and module tables, the generator stacks, the exception state and the value that is being returned, and also from every old mutable container (the list ctl), because an old list or dictionary may have been given a young element since the last collection. It then frees the young objects that were not marked. Survivors become part of the enclosing window; there is no tenuring, so a long-lived structure is marked again at each collection of an enclosing window.
- **Safe points only.** Collection happens only at the end of a loop iteration or before a return, never inside a primitive, so a native routine can hold object numbers across its own allocations.
- **Everything that was allocated is reused.** Freed object slots, blocks of the slot array (in size classes), dictionary entries and dictionary ids, generator regions and blocks of the numpy float store are put on free lists and reused. Frames of finished functions that define closures are kept in the list cfl and freed when no live function object or frame reaches them.
- **Never freed:** classes, modules, files, built-in objects and functions that have attributes.

Option	Effect
--memstat	print statistics on standard error at the end of the run: the number of objects (nob), slot-array size (nsl), collections (gcs), objects freed (freed) and others
--nogc	switch the collector off; the interpreter behaves as in Build 2 (useful to separate a collector problem from an interpreter problem, and for the comparison below)
--gcfix=N	collect whenever N objects have been allocated in a window (a fixed threshold instead of the adaptive one), and reuse the freed slots
--gcstress=N	like --gcfix=N, but a freed slot is never reused, so a stale reference to a freed object shows up as an error or a wrong result instead of silently reading a new object; the test suite is run this way to find them

Figure 121. Garbage-collector options of the CREXX interpreter (written before the script name)

```
$ CREXX_VM=/path/to/patched/bin/crexx ./crexx/py --memstat loop.py 2>&1 | sed 's/
    nid=.*//'
1254239854.832002
memstat nob=20343 nsl=8 nvp=73 npn=0 nde=1 ndd=1 ngn=0 ngs=0 gcs=14 freed=280012
```

Figure 122. Looking at the collector with --memstat

Program	--nogc: time, memory	collector on: time, memory
300,000 iterations of a float calculation	7.8 s, 856 MB	7.3 s, 103 MB
fft, ifft, rfft of 1024 points	4.7 s, 611 MB	3.9 s, 275 MB
five fft of 4096 points	25 to 26 s, 3,954 MB	31 to 33 s, 449 MB

Figure 123. Effect of the collector on this machine (patched virtual machine; the start of the interpreter alone uses 72 MB)

The numbers are single runs on a machine that was not otherwise idle, so differences of a few per cent are noise. The test suite passes with the default thresholds, with --gcfix and with --gcstress (tests/31_gc.py allocates linked lists, trees, generators, closures, classes, dictionaries and numpy arrays in loops and checks that the results do not depend on when objects are reclaimed).

Remaining gaps: (1) The frame of a generator that defines a closure is not freed, so a program that creates very many such generators still grows. (2) The collector costs time: the allocation-heavy FFT runs about 15 to 25 per cent slower than with --nogc, because every collection marks the old mutable containers. (3) __del__ is never called and there are no weak references.

Chapter 11. Performance

This chapter reports measurements, not promises. Everything below was timed on one machine: a two-core Intel Xeon at 2.8 GHz running Linux, with gcc 13.3 (the C interpreter built with make posix, which compiles with -O2) and CPython 3.13.16 with NumPy 2.5.3, pandas 3.0.5 and SciPy 1.18.1 for comparison; CREXX 1.0.0 beta 3 (commit 77ba820c) with the patched virtual machine of Chapter 2 unless it says otherwise. Each figure is the best of three runs of the whole process (best of two for the CREXX interpreter), including interpreter start-up, measured with the programs in c/bench and some smaller programs in the same style. Timings on a different machine will differ; the ratios are more stable than the times.

The C interpreter against CPython

Program	C interpreter	CPython	C / CPython
fib(27), recursive calls	62 ms	54 ms	1.1x
3 million iterations of an integer loop	716 ms	1.01 s	0.71x
dict, list and set operations	285 ms	329 ms	0.87x
string building, split, upper, format	233 ms	100 ms	2.3x
numpy: 1M-element array operations	176 ms	176 ms	1.0x
big integers: 3000!, Fibonacci(20000), 3**20000	197 ms	45 ms	4.4x
FFT: 5 rounds of fft, ifft and abs on 4096 points	280 ms	134 ms	2.1x
complex numbers: 3600 Mandelbrot points	14 ms	28 ms	0.49x

Figure 124. Wall-clock time, C interpreter and CPython 3.13 (best of three)

The C interpreter is competitive with CPython on programs that run for a fraction of a second: faster on integer loops (an AST walker with local variables in array slots and no reference counting) and on complex arithmetic, about the same on function calls, dictionaries and lists, slower on the work done in the library: string methods and formatting (about 2.3 times slower on the strings program), big integers (about 4 times slower, since multiplication and division are schoolbook algorithms on base-10000 limbs), the FFT (about 2 times slower, because numpy.fft is Python code in pylib where NumPy has compiled code; at 4096 points a transform takes roughly 25 ms) and the arithmetic of numpy arrays, whose loops are plain C but not vectorised. A large part of the CPython time in the numpy and FFT programs is the 0.12 to 0.15 s that real NumPy needs to import. The table below shows the operations of c/bench/numpy_bench.py, each timed inside the program.

Operation	C interpreter	NumPy 2.5
arange(1M)	5.5	2.9
a*a+1	12.9	8.3
sum (int)	1.2	0.8
float a*a+1 and sum	32.9	12.5
sqrt + mean	8.3	2.3
200x200 matmul	6.4	1.2
mask, slice, sort	79.2	13.4

Figure 125. numpy operations on 1,000,000 elements (milliseconds, best of three)

Float sums, the sort, the Boolean mask and matrix products are the slow spots: the library is a few times slower than NumPy, and about 5 times slower in the matrix product, which does not use a blocked or vectorised BLAS routine. For the sizes a program written for this interpreter is likely to use this does not matter.

Native re and json

Program	native (C)	pure Python (pylib)	CPython	native / CPython	pylib / native
re: findall, sub, split, finditer on a 50 KB text, 20 rounds	188 ms	20.79 s	323 ms	0.58x	111x
json: dumps and loads of 300 records, 30 rounds	194 ms	3.42 s	162 ms	1.2x	18x

Figure 126. The C interpreter's native re and json against the pure-Python library and CPython (best of three)

The pure-Python modules (pylib/re.py and pylib/json.py) are what the CREXX interpreter runs, and what the C interpreter ran before the native versions were written. The native regular-expression engine is about 110 times faster than the Python one and, on this benchmark, about 1.7 times faster than CPython's own engine; the native json is about 18 times faster than the Python one and about 20 per cent slower than CPython's. c/bench/run.sh prints the same three-way comparison.

Start-up and imports

Program	CPython 3.13	C interpreter	CREXX interpreter
start only (-c pass)	0.019	0.003	0.39
import json	0.031	0.003	0.56
import re	0.028	0.003	0.59
import datetime	0.021	0.006	0.58
import numpy	0.150	0.004	0.36
import pandas	0.495	0.048	2.28
import scipy.stats	1.358	0.013	0.81
import scipy.fft	0.432	0.009	0.67

Figure 127. Time to start the interpreter and import a module (seconds)

The C interpreter starts in two or three milliseconds and imports even pandas in under fifty, because it compiles only the Python that it needs (12,800 lines for pandas) and there is no C extension to load; the embedded build, py_embed, starts just as fast, since the library table is compiled into the program. The CREXX interpreter pays about 0.4 seconds before it runs the first line, no matter what the program is, when it runs on the patched virtual machine; importing pandas adds another two seconds. With a stock CREXX virtual machine the start-up is longer: 3.4 seconds on this machine for the same print(1) (1.6 to 3.4 seconds in repeated measurements).

The CREXX start-up and the patched virtual machine

The start-up of the CREXX interpreter used to be its most visible fault. A profile of a stock virtual machine running print(1) showed 58 per cent of its 1.6 billion instructions in the rebuild of the interface-method and factory registries during the link step, two linear scans over about 13,000 meta entries for each of the 457 procedures; 22 per cent went into reading py.rxbin and about 280 library modules, and 5 per cent into the program. The patch crexx/vm-startup.patch indexes the meta lists with a hash table while the registries are rebuilt. It touches one file of CREXX (interpreter/rxvmintp.c), leaves the contents of the registries unchanged, and passed the CREXX project's own 2289 tests (labels essential, smoke and comprehensive) and its 12 qualification and stress tests on a patched build exactly as on a stock one (STARTUP.md has the details). The launcher uses the patched virtual machine when CREXX_VM names it.

	stock VM	patched VM
python -c pass (best of three)	3.44 s	0.39 s
instructions executed for print(1) (callgrind)	1.60 G	0.7 G
33 tests (tests/run_tests.sh ./crexx/py)	160 s (31 tests)	74 s
CREXX project tests (essential, smoke, comprehensive)	2289 / 2289 pass	2289 / 2289 pass
CREXX project qualification and stress tests	12 / 12 pass	12 / 12 pass

Figure 128. CREXX start-up and test suite with the stock and the patched virtual machine

The CREXX interpreter

Program	CPython	C interpreter	CREXX total	CREXX minus start-up	CREXX / CPython
fib(25), recursive calls	32 ms	24 ms	2.94	2.55	81x
100,000 iterations of an integer loop	44 ms	34 ms	2.52	2.13	48x
10,000 dict/list items	24 ms	8 ms	0.96	0.57	23x
5,000 string operations	20 ms	13 ms	0.61	0.22	under 0.3 s

Program	CPython	C interpreter	CREXX total	CREXX minus start-up	CREXX / CPython
numpy: 100,000-element arrays	130 ms	9 ms	1.08	0.69	5x
FFT of 1024 points (fft, ifft, rfft)	162 ms	32 ms	4.03	3.64	22x
complex numbers: 3600 Mandelbrot points	28 ms	14 ms	1.45	1.06	37x

Figure 129. Seconds, CREXX interpreter (patched VM) against the others (best of 3 / 2)

The CREXX interpreter executes about 47,000 loop iterations or about 95,000 function calls per second (fib(25) makes 242,785 calls and takes about 2.5 seconds after start-up). That is 10 to 100 times slower than CPython and 70 to 120 times slower than the C interpreter on the same program; the work is done by the CREXX virtual machine, which interprets bytecode that itself interprets the Python tree. It is a teaching and scripting tool. The programs that are comfortable are the ones that do little work per run: scripts, short calculations, library demonstrations, and the regression tests. Since the patched virtual machine the fixed cost is small enough that a short script takes under a second; before, it took two.

Two further effects belong in the figure, both about memory. Up to Build 2 the CREXX interpreter never freed an object, so a loop that creates a new float or string on every iteration grew the process: a loop of float calculations had a resident size of 72 MB at start, 224 MB after 100,000 iterations and 668 MB after 300,000, where the C interpreter and CPython stay at 10 MB, and five rounds of fft, ifft and abs on 4096 points were killed by the operating system after 30 seconds with 6 GB in use on this 8 GB machine. Build 3 has a garbage collector (Chapter 10): the same 300,000 iterations now peak at 103 MB (856 MB with --nogc) and the five FFTs at 449 MB in about 31 seconds (the C interpreter does the same in 0.28 s, in 10 MB; one transform of 1024 points takes 4 seconds in total). The price is time: the FFT runs 15 to 25 per cent slower than with --nogc, and the benchmark tables above were measured before the collector was added, so CREXX times for allocation-heavy programs are now somewhat higher. Floats and strings are heap objects; small integers are not.

Reproducing the measurements

bench/run.sh runs each program once with ./py and once with python3 and prints the two times (the figure below shows the output of one run, which varies by 10 to 30 per cent from run to run; the numbers of the tables are the best of three), and then the native re and json against the pylib versions, which it copies next to the scripts as pyre.py and pyjson.py (the programs take the argument pure to use them). The programs fib.py, loop.py, collections.py, strings.py, numpy_bench.py, re_bench.py and json_bench.py are in c/bench; the smaller programs, the big-integer, FFT and complex programs used for the tables of this chapter are listed in Appendix D.

```
$ cd c
$ make posix
$ sh bench/run.sh
fib: py=.069349727s python3=.053545187s
loop: py=.553395532s python3=1.110655477s
collections: py=.233403842s python3=.294642567s
strings: py=.243214861s python3=.108915518s
  arange(1M)                 7.82 ms
  a*a+1                     17.62 ms
  sum (int)                  1.53 ms
  float a*a+1 and sum       46.04 ms
  sqrt + mean                9.24 ms
  200x200 matmul             6.62 ms
  matmul with transpose      6.60 ms
  mask, slice, sort         80.25 ms
numpy_bench: py=.186738674s python3=.181801520s
re_bench: native=.197929481s pylib=19.528031377s python3=.729234547s
json_bench: native=.208185430s pylib=3.191040042s python3=.305039973s
```

Figure 130. Running the benchmark set

Chapter 12. Restrictions and Known Problems

Python/CREXX implements a large subset of Python 3, not all of it. The limits below are those known at this release: they were found by the test suite, by running the examples of this manual on both interpreters and by comparing every example with CPython. C and X mark limits that apply to the C interpreter or to the CREXX interpreter only; limits without a mark apply to both.

Syntax that is not accepted

- **match statements and except* with exception groups** are available (in both interpreters since Build 9). In the CREXX interpreter BaseExceptionGroup derives from Exception, so except Exception also catches a group of BaseExceptions. **async and await** are not supported (the CREXX interpreter accepts the keywords async def and runs the function as an ordinary one; the C interpreter reports a syntax error).
- **The walrus operator** := is accepted by both interpreters, inside comprehensions too (since Build 9 in both); f"{x=}" works in both.
- **lambda containing yield** is a syntax error in the C interpreter (C). In the CREXX interpreter a lambda with several parameters as the bare element of a list display or comprehension, [lambda a, b: a for i in x], is a syntax error and needs parentheses.
- **Type-parameter syntax** (def f[T](x), class C[T], type X = ...) is not supported. Ordinary annotations on parameters, return values and variables are accepted. The C interpreter and the CREXX interpreter (Build 9) store variable annotations of a module or class in __annotations__ and builds f.__annotations__ for a function when asked; an annotation that cannot be evaluated is skipped instead of raising NameError, and annotations of local variables are not evaluated. dataclasses reads class-body annotations by rescanning the source text.
- **from __future__ import ...** raises ModuleNotFoundError in the C interpreter; remove the line (it is accepted in the CREXX interpreter). C
- **Inconsistent dedent**: the C interpreter reports the error "unindent does not match any outer indentation level"; the CREXX interpreter accepts such a file without a message. Indentation errors are reported as SyntaxError, never as IndentationError.

Strings and bytes

A str is a sequence of bytes in the C interpreter: UTF-8 text passes through input, output and concatenation unchanged, but len(), indexing, slicing, ord() of an element, reversing and the string methods operate on bytes. The CREXX interpreter's strings are sequences of Unicode code points and behave as CPython's do for text in the Basic Multilingual Plane. ASCII text behaves identically everywhere.

```
s = "né€"
print(len(s), s[1], s[::-1] == "€én", s.upper(), ord(s[1]), "é".isalpha())
print(sorted(["z", "é", "a"]))

$ py prog.py   (C interpreter)
6 \xc3 False NÉ€ 195 False
['a', 'z', 'é']

$ py prog.py   (CREXX interpreter)
3 é True NÉ€ 233 True
['a', 'z', 'é']

$ python3 prog.py   (CPython 3)
3 é True NÉ€ 233 True
['a', 'z', 'é']
```

Figure 131. Text with non-ASCII characters

- C: len, indexing and slicing count bytes; isalpha, isdigit, isupper, title, capitalize, center, ljust and rjust, and the width of format specs, are wrong for non-ASCII characters (upper() and lower() know the Latin-1 letters). Sorting is by byte, which for UTF-8 is the same order as by code point.

- X: ß.upper() gives ß, not SS; the special casing rules of Unicode are not implemented; characters outside the BMP are not tested.
- bytes and bytearray exist in both interpreters (Chapter 4) but memoryview, the array module, the % operator on bytes and bytes patterns in re do not; the "rb" and "wb" modes of open() give text, so binary files cannot be read or written, and sys.stdout has no usable buffer attribute. hasattr() on a bytes object is always true in the CREXX interpreter.
- str.format_map does not exist; str.translate and str.maketrans exist in the CREXX interpreter only (since Build 5), and str.isprintable exists in both. str.casefold is the same as lower().
- Source files and string literals are expected to be UTF-8 or ASCII. The encoding= argument of open() is accepted and ignored.

Numbers

- **Bit operations on big integers**: ~, << and >> work for any size in both interpreters. & | and ^ work for any size with two's complement semantics, pow(a, b, m) accepts huge and negative exponents (a negative exponent takes the modular inverse), range() accepts bounds, steps and lengths of any size (len(range(0, 10**30, 10**25)) is 100000, while len() of a range longer than 2**63 - 1 raises OverflowError), and int-to-float conversions and comparisons are exact and correctly rounded (2**53 + 1 > 2.0**53 is True, int(1e22) is 10000000000000000000), in both interpreters (in the CREXX interpreter since Build 5).
- **Big integer speed**: multiplication, division and powers are quadratic, so 3**100000 takes 0.12 s in the C interpreter where CPython takes 2 ms. str() of a huge number is linear and fast, but int() of a long string is quadratic in the C interpreter (20,000 digits take 0.7 s) and fast in CREXX. CPython's limit of 4300 digits for conversion between int and str does not exist here.
- **Big integers and floats**: float(10**400) raises OverflowError ("int too large to convert to float") and int / int is correctly rounded for any size, so 10**400 / 10**399 is 10.0, in both interpreters (in the CREXX interpreter since Build 5; before, float(10**400) was inf and the quotient nan).
- **Last-digit differences**: sums of floats are compensated like CPython 3.12 and math.fsum is exactly rounded in both interpreters. math.erf, erfc, gamma and lgamma agree with the C library to about 1e-15 and are not bit-identical, and math.degrees and similar functions use plain double arithmetic, so results can differ in the last digit. statistics.mean, variance, pvariance, stdev and pstdev use exact integer arithmetic like CPython.
- **Float edge cases**: denormal numbers and numbers within a few ulps of the largest float print and parse exactly in both interpreters (5e-324 prints as 5e-324; in the CREXX interpreter before Build 5 it printed as 4.94065645841247e-324).
- **hash()**: the C interpreter since Build 4 and the CREXX interpreter since Build 5 compute hash() for int, bool, big int, float and complex as CPython does (the value modulo 2**61 - 1 with the sign kept, -1 becomes -2, inf is 314159, a complex number is hash(re) + 1000003 * hash(im)), so hash(1e10) == hash(10**10), {10**10: 1}[1e10] works and hash(Fraction(1, 2)) == hash(0.5). The hash of tuples (xxHash scheme, so hash((1, 2)) is -3550055125485641917, nested tuples and tuples that contain frozensets, floats, None or big ints included) and of frozensets (_shuffle_bits scheme) is CPython's in the C interpreter since Build 6 and in the CREXX interpreter since Build 7 (the CREXX interpreter computes the 64 bit arithmetic with 16 bit limbs, and only when hash() is called or a __hash__ method returns it, because its dicts and sets use another internal key; dict lookups with tuple keys are not slower). A tuple that contains a str or bytes has a hash that differs from CPython's, because that of the str does. hash(None) is 4238894112 as in CPython 3.12 and later. Since Build 8 the C interpreter takes the result of a user __hash__ method as CPython does: an int that fits a signed 64-bit value is the hash itself (-1 becomes -2), a larger int is reduced like hash(int), modulo 2**61 - 1, and any other result raises TypeError: __hash__ method should return an integer, so class P with __hash__ returning hash((1, 2)) gives hash(P()) == hash((1, 2)) and 2**63 - 1, -2**63 and 2**64 + 5 give the same hash as in CPython (a build with a 32-bit long keeps only the low word of such values). The CREXX interpreter does the same since Build 9 for hash() and for the TypeError, also when the instance is a dict key or set element; its dicts compare the raw result of __hash__, so two objects are never mixed up because their hashes agree after the reduction. Only the order of sets of such instances differs (it is the insertion order). Decimal's hash is CPython's in both since Build 6. The hash of str and bytes differs from CPython's (which is different in every run anyway). Programs must not depend on hash values of str.
- **sys.maxsize** is 999999999999999999 in the C interpreter (the largest small integer), 2**63-1 in CREXX.

- **random and numpy.random**: in the C interpreter (since Build 9) both reproduce the sequences of CPython 3.13 and NumPy 2.x exactly; the one known deviation is a last-digit difference where the C library functions lgamma (random.binomialvariate with n*p >= 10) or a fused multiply-add build of NumPy are involved. The native random module of the CREXX interpreter does not reproduce CPython's sequences (pylib/pyrandom.py does). time.time() has a resolution of a second in a C interpreter built without -DPY_POSIX.

Containers

```
print({5, 3, 10, 1})
print({"a", "b"} | {"c"} == {"c", "b", "a"}, sorted({5, 3, 10, 1}))
d = {}
for k in (3, 1, 2):
    d[k] = None
print(list(d), list({3, 1, 2}))

$ py prog.py   (C interpreter)
{1, 10, 3, 5}
True [1, 3, 5, 10]
[3, 1, 2] [1, 2, 3]

$ py prog.py   (CREXX interpreter)
{5, 3, 10, 1}
True [1, 3, 5, 10]
[3, 1, 2] [3, 1, 2]

$ python3 prog.py   (CPython 3)
{1, 10, 3, 5}
True [1, 3, 5, 10]
[3, 1, 2] [1, 2, 3]
```

Figure 132. Order of sets

- **Set order.** CPython iterates a set in the order of its hash table. The C interpreter (since Build 6) emulates that table (probing, growth, deletion, pop, the folding of set displays), so the order of a set of ints, bools, None, floats, complex numbers, and tuples and frozensets of those is the same as CPython's, as in {5, 3, 10, 1}, which prints {1, 10, 3, 5}. The emulation needs a 64-bit unsigned long: with a 32-bit long (VM/370, -m32) the sets work the same but the order of colliding keys can differ from CPython. Instances of a class with a user __hash__ method are ordered by the value it returns (Build 8), so a set of such objects iterates as in CPython. A set that receives a str, bytes or instance key without __hash__ switches to insertion order, because CPython's order for those changes from run to run (hash randomization); programs cannot depend on it. The CREXX interpreter keeps every set in insertion order, whatever the keys (its dicts and sets are tables keyed by a text form of the key, not by the hash value, so emulating CPython's open-addressing table would add a hash computation to every insertion, which Build 7 judged not worth the cost for the dict and set speed), so {5, 3, 10, 1} prints as written and list({3, 1, 2}) is [3, 1, 2] where CPython and the C interpreter print [1, 2, 3]; union and intersection also differ. Use sorted() for output that must be the same everywhere. Sets give the same results as CPython everywhere that order is not observed.
- **frozenset** is a separate immutable type (Build 6): no mutating methods (AttributeError), hashable with CPython's algorithm, usable as a dict key or set element, repr frozenset({1}), set operators return the type of the left operand, and class X(frozenset) works. A set, a list, a dict and a bytearray are not hashable: hash({1}), {{1}}, d[{1}] = 1, {(1, {2})}, {1}.add({2}), frozenset([{1}]) and dict.get / setdefault / pop with such a key raise TypeError: unhashable type: 'set' (or 'list', 'dict', 'bytearray'; the class name for a subclass) in both interpreters, as in CPython (since Build 7 in the CREXX interpreter, which accepted a set there before and left a stray entry behind after the failed insertion of a list). A set that is only looked up is converted to a frozenset first, as CPython does: {1} in {frozenset({1})} is True, and discard and remove of a set work the same way; dict lookups do not convert. A subclass that defines __hash__ is hashable.
- **dict** is insertion ordered and complete; dict views are live. Recursive containers are not detected by repr(): repr of a list that contains itself recurses until the recursion limit.
- **range** objects have no start, stop, step, count or index in the C interpreter and no count or index in the CREXX interpreter.

- **sort** of a list with mixed types raises TypeError as in CPython; the sort is stable and uses the comparison operators of the objects (__lt__).

Classes and objects

- **Subclassing built-in types and __new__** work in both interpreters (Chapter 3): tuple, list, dict, set, str, int, float, bytes, bytearray, complex and the exceptions. Limits: in the CREXX interpreter metaclass __call__ is ignored; and OSError(2, "x") does not become FileNotFoundError.
- **Metaclasses**: the C interpreter (since Build 10) supports class A(metaclass=M) with M.__new__, M.__init__ and M.__call__, type(name, bases, namespace), methods and properties defined on the metaclass (found through the class, so len(C), iter(C), x in C, C[key] and reversed(C) call the metaclass methods), __init_subclass__ with class keyword arguments and super().__init_subclass__(), __class_getitem__, and list[int], dict[str, int] and tuple[int, ...] as types.GenericAlias; list(Color), len(Color), Color["RED"], "in" and Color.__members__ work for enum classes. The CREXX interpreter accepts the syntax class A(metaclass=M) and ignores it: type(A) is type and the metaclass methods are not called, so enum.Enum members work there (Color.RED, .name, .value, comparison, methods, Color(1)) but iteration over the class, len(Color), Color["RED"] and "in" do not.
- **abc**: @abstractmethod and ABC are accepted but instantiating a class with unimplemented abstract methods does not raise TypeError.
- **typing**: all the names exist and ignore their arguments; typing.NamedTuple works as a function in the C interpreter only and as a class (class P(NamedTuple): x: int) in neither: use collections.namedtuple. Deriving from Generic[T] raises TypeError in the C interpreter ("'type' object is not subscriptable") and works in the CREXX interpreter.
- **__slots__** is accepted and ignored. __doc__ is None: docstrings are discarded. __eq__ without __hash__ leaves the object hashable in the CREXX interpreter (CPython makes it unhashable). dir() of an int or a native module returns an empty or short list in the CREXX interpreter. X
- **super()** inside a classmethod does not work in the CREXX interpreter. X
- **Special attributes**: __class__, __dict__, __name__, __mro__, __bases__ and vars() exist; add_note() of exceptions does not; in the C interpreter (since Build 10) the exceptions have __cause__, __context__, __suppress_context__, __traceback__ and with_traceback(), and functions have __code__ (co_name, co_filename, co_firstlineno, co_argcount, co_varnames and more), __defaults__, __kwdefaults__, __annotations__, __globals__ and __closure__, and obj.__dict__ is live (obj.__dict__.update(d) sets attributes); the CREXX interpreter (since Build 10) has __cause__, __context__, __suppress_context__, __traceback__, __code__ (co_name, co_filename, co_firstlineno, co_argcount, co_varnames and more), __defaults__, __kwdefaults__, __annotations__ and __globals__, but not __closure__ or with_traceback(), and obj.__dict__ is a snapshot there. Error messages that name a function use its plain name where CPython uses the qualified name (one() and <lambda>() where CPython says f.<locals>.one() ...).
- **__del__** finalizers are never called and generators are not closed when they are garbage collected (a pending finally block of an abandoned generator does not run). gc, weakref, threading, asyncio, subprocess, ctypes, signal and calendar do not exist.

Namespaces, eval and exec

- **globals()** returns a snapshot dictionary: assigning to it does not create a global variable. exec() at module level writes new names back to the module; exec() inside a function does not change the function's locals (as in CPython). locals() is a snapshot. In the C interpreter vars() and dir() work on instances and classes; the main script has no __file__ (C).
- **eval and exec** run the text in a temporary module whose names are copied from and back to the dictionaries that were passed. They take str only, not code objects; compile() does not exist.
- **Recursion** is limited to 1000 frames in the C interpreter and to about 900 in the CREXX interpreter (it raises RecursionError: maximum recursion depth exceeded); sys.setrecursionlimit() is accepted and has no effect in either interpreter.

Input, output and the operating system

- **Files** are text files with the modes r, w and a. There is no seek(), tell() or mode on file objects, no name or closed in the C interpreter (the CREXX interpreter has both), no binary mode, no readinto and no buffering parameter; io.StringIO is complete for in-memory text.

- **os** has getcwd, getenv, environ, name, sep, linesep, curdir, pardir and the path functions join, exists, isfile, isdir, basename, dirname, split, splitext, abspath, normpath, normcase and expanduser. The C interpreter (since Build 10, in a build with -DPY_POSIX) also has native listdir, mkdir, rmdir, chdir, getpid, getppid, urandom, remove, rename, unlink and os.path.getsize, and pylib/os.py adds fspath, fsencode, fsdecode, makedirs, removedirs, replace, walk, cpu_count and strerror, so that hasattr(os, "listdir") is true and os.walk works; open errors are FileNotFoundError, FileExistsError, PermissionError, IsADirectoryError or NotADirectoryError with errno, strerror and filename. Without -DPY_POSIX the directory functions raise OSError (ENOSYS) and getpid is 1. stat, system and popen do not exist; the CREXX interpreter (since Build 10) has native listdir, mkdir, rmdir, remove, unlink and rename over the rxfs plugin, makedirs, walk and removedirs through pylib/os.py; the other os.path functions (getsize, relpath, isabs and so on) and getpid (a constant) are still missing.
- **time** has time, sleep, perf_counter, monotonic and process_time (and time_ns, perf_counter_ns in CREXX); strftime, localtime and gmtime do not exist (use datetime). There is no locale, no time zone database (datetime.timezone supports fixed offsets and utc), no calendar module, no logging, argparse, unittest, pathlib, hashlib, base64, uuid, tempfile, shutil or glob. Of the networking modules only socket and select exist (Chapter 5).
- **Standard input**: input([prompt]) and sys.stdin work with piped or redirected input; at the end of the file EOFError is raised.
- **Tracebacks** are simplified. The C interpreter reports the line number of the statement that raised; the CREXX interpreter reports "line 1, in <module>" for every error, and neither prints the source line or the intermediate frames. The messages of syntax errors differ from CPython's.

The libraries

- **collections.abc does not exist**; isinstance(x, collections.abc.Sequence) must be written with the concrete types. The module collections.OrderedDict and the others are pure Python and slower than CPython's.
- **re** is a backtracking engine and, like any backtracking engine, can take exponential time on pathological patterns. The native engine of the C interpreter stops such a search with RuntimeError ("regular expression too complex") after six million backtrack entries; its offsets are byte offsets, \w and case-insensitive matching know ASCII only, and bytes patterns and re.Scanner do not exist. The pure-Python engine of the CREXX interpreter has no conditional patterns (?(1)a|b), treats possessive quantifiers as greedy ones, reports some error positions differently and is about 100 times slower.
- **sockets** are IPv4 TCP only (Chapter 5): no UDP, IPv6, sendto, recvfrom, selectors or ssl. The C interpreter creates sockets only when built with -DPY_POSIX; without it import socket works and socket.socket() raises OSError ("sockets not available"). A socket is not closed when it is garbage collected, and in the CREXX interpreter a collected object is never finalized.
- **datetime** has no time zone database; datetime.now() is built on time.time(), so in a C interpreter built without -DPY_POSIX it has a resolution of a second.
- **decimal** and **fractions** are written in Python and are slow for long computations; the contexts, traps and rounding modes are complete.
- **NumPy**: only bool, int64, float64 and complex128 arrays (the other dtype names are aliases and the scalar types np.float64 and the rest are the Python types); slices and transposes are copies (so writes through them do not reach the original); np.sort with axis in the C interpreter works on the last axis only; no ma, vectorize, set_printoptions or ufunc methods; indexing with two index arrays at once gives the wrong result (an outer product of the indices); integer overflow gives a float in the CREXX interpreter; sort, unique, diag and linalg functions other than eig and eigvals reject complex arrays in the C interpreter; eig and eigvals return a real array when all eigenvalues are real, where NumPy returns a complex one; the signs of eigenvectors, singular vectors and QR factors can differ from NumPy in the last bit (about 5 per cent of random non-symmetric matrices), and, for a few matrices of order 6 and more, a singular matrix is reported by NumPy and not by inv and solve or the reverse. See Chapter 6.
- **pandas** and **SciPy**: see the lists at the end of Chapters 7 and 8. pandas has no time zones (tz= is ignored) and no Period type; solve_ivp(method="DOP853") raises ImportError; to_json() and to_string(index=False) differ from pandas.

The CREXX interpreter in particular

- **Start-up time**: about 0.4 seconds for every run with the patched virtual machine (CREXX_VM, Chapter 2) and 2 to 3.4 seconds with a stock one, and several minutes once for the first compilation (and again after any change to py.crexx).
- **Speed**: 10 to 100 times slower than CPython; see Chapter 11.
- **Memory**: since Build 3 the CREXX interpreter has a garbage collector (Chapter 10); a loop of 300,000 float operations stays at about 100 MB instead of 860 MB, and five FFTs of 4096 points need 450 MB instead of about 4 GB. Two gaps remain: the frame of a generator that defines a closure is never freed, so a program that creates a very large number of such generators still grows, and the collector costs time (the FFT is 15 to 25 per cent slower than with --nogc). Memory is still much larger than in the C interpreter: a float is a heap object, and the interpreter itself uses 72 MB at start.
- **The library path** /home/claude/python/pylib is compiled into the sources as the last search directory.
- **One run at a time per interpreter directory**: the CREXX driver uses a lock file next to py.crexx; concurrent runs wait for each other. A run that is killed can leave an rxvme process that holds the lock.
- **For those who change the sources**: in CREXX Level B the words main, self and dummy are reserved; variables passed as plain arguments are passed by reference (pass x + 0 to pass a copy); the literal 1.7976931348623157e308 behaves as infinity (compare through x - x); and the bytecode must be rebuilt after every change to py.crexx (the launcher does this when py.crexx is newer than py.rxbin).

Known defects

Two defects were found while this manual was prepared and were fixed: in the C interpreter a heap-buffer overflow when a module was imported while the table of globals grew (the assignment G[global_index(...)] = v evaluated G before global_index() reallocated it), which could corrupt memory or crash on programs that import several modules, and, in pylib/types.py (both interpreters), a SyntaxError on import types in the C interpreter, which used a lambda containing yield. The test suite passes under AddressSanitizer and UBSan with both fixes. No defect that crashes the C interpreter is known.

Two defects that were listed in Build 2 are fixed. numpy.linalg.eigvals in the CREXX interpreter returned wrong eigenvalues for some non-symmetric real matrices (the companion matrix [[0, 1, 0], [0, 0, 1], [-6, -11, -6]] gave 0 and -3 plus or minus 1.41j); eig and eigvals are now ports of LAPACK dgeev in both interpreters and give -1, -2 and -3 on both, and the eigenvectors of a diagonal matrix are exact (the noise of the order of 1e-55 in the C interpreter is gone). The out-of-memory failures of the CREXX interpreter on temporary-list-heavy code such as numpy.fft were caused by the missing garbage collector and are fixed by the one added in Build 3.

Build 4 fixed a family of defects that depended on the width of the C type long: values near and beyond 2**31 and 10**18 in int/float conversion, comparison of int and float, hash(), bit operations, pow(a, b, m), range, str.split(None, n), bytes.hex(sep, n), float repr at a power of two and about thirty smaller things (Chapter 13 lists them). The remaining differences from CPython are the same in every build of the C interpreter: set iteration is in insertion order only for keys whose CPython order is random (str, bytes, instances; the CREXX interpreter uses insertion order for all sets, Chapter 12), str is UTF-8 bytes, float() of non-ASCII digits fails, statistics results can differ by one unit in the last place from CPython's exact-fraction arithmetic and the repr of a bytearray that contains a quote differ. A build with a real 32-bit long adds only the limits of that width (Chapter 13). Build 5 ran the same hunt against the CREXX interpreter (Chapter 13) and fixed the same family there: bit operations, pow with big and negative exponents, ranges of any size, exact int and float comparison and conversion, hash(), float repr at powers of two and for denormals, and many smaller items. The comparison with CPython went from about 400 differing cases per 3150 to a few dozen, all in the categories listed in Chapter 13.

Build 9 added to the C interpreter the match statement (all pattern kinds and guards, with the soft keywords match and case recognised only in statement position), the walrus operator, positional-only parameters (the / marker), variable and function annotations (evaluated eagerly; __annotations__ of modules, classes and functions), ExceptionGroup, BaseExceptionGroup and try/except *(not resumable inside a generator), starred expressions in displays, and the exact random and numpy.random described in Chapters 5 and 6. Build 9 closed language gaps of the CREXX interpreter: the match statement (all pattern kinds, guards, nesting), the*

walrus operator (with the comprehension scoping rule), try/except with ExceptionGroup and BaseExceptionGroup, __annotations__ (module, class and function), and the result of a user __hash__ method (a value that fits 64 bits is kept, -1 becomes -2, a larger int is reduced, a non-int raises TypeError, as in CPython and in the C interpreter since Build 8). Remaining differences: sets of objects with a user __hash__ iterate in insertion order, because the text-keyed dicts of the CREXX interpreter have no numeric hash to place an element in CPython's table (test 34_hash64 is skipped on the CREXX interpreter for that reason; test 34b_hash_values checks the values); BaseExceptionGroup derives from Exception; yield inside an except* block, and a predicate argument of split are not available in the CREXX interpreter. The start-up time did not change (about 0.45 s).

Build 10 closed gaps of the C interpreter that were found while the pure Python library was written. os and time: native listdir, mkdir, rmdir, chdir, getpid, getppid and urandom, the C library calendar behind time.localtime, mktime, strftime (with %Z, %z, tm_zone and tm_gmtoff) and the time zone (timezone, altzone, daylight, tzname), completed by pylib/os.py and pylib/time.py (fspath, fsencode, makedirs, walk, struct_time, strptime, asctime, ctime), and random.SystemRandom and random.getstate through os.urandom; sockets: socket.socketpair() and sock.makefile(); language: a live obj.__dict__, str.splitlines(keepends=True), int(bytes, base), Class.__new__(Class), metaclasses (__new__, __init__, __call__ and methods found through the class), __init_subclass__ with class keywords, super().__init_subclass__(), types.GenericAlias (list[int]), iterable and sized enum classes, binary files ('rb', 'wb', 'ab', 'r+b' with bytes read and write, seek, tell, readinto), open() errors as FileNotFoundError, FileExistsError, PermissionError and the other OSError subclasses with errno, strerror and filename; introspection: function __code__ (co_name, co_filename, co_firstlineno, co_argcount, co_varnames and more), __defaults__, __kwdefaults__, __annotations__, exception __cause__, __context__, __suppress_context__ and __traceback__ (tb_frame, tb_lineno, tb_next), sys.exc_info(), sys._getframe() with f_code, f_lineno, f_back, f_globals and f_locals, so that traceback.format_exc(), inspect.signature() (new Signature, Parameter and BoundArguments in pylib/inspect.py), inspect.stack() and the funcName, lineno and pathname of logging work. Approximations: f_locals is a snapshot, the frame of a traceback has no locals, except* and with do not attach tracebacks, and CPython 3.13 draws marker lines (~~^~~) under the failing expression in tracebacks, which this library does not. The CREXX interpreter received the introspection part in Build 10 (and part of the file and language part): function __code__ (co_name, co_filename, co_firstlineno, co_argcount, co_posonlyargcount, co_kwonlyargcount, co_varnames, co_flags), __defaults__, __kwdefaults__, __annotations__ and __globals__; bound-method attributes forwarded to the function (getattr(obj.method, "x")); exception __cause__, __context__, __suppress_context__ (raise X from Y, and the exception being handled becomes __context__) and __traceback__ (tb_frame, tb_lineno, tb_next, built from a record that every function frame appends while the exception passes, so the chain is complete and tb_frame.f_code.co_name is right); sys.exc_info(); sys._getframe(depth) with f_code, f_lineno, f_back, f_globals and a snapshot f_locals; Class.__new__(Class) without calling __init__; and os.mkdir, rmdir, remove, unlink, rename and listdir (over the rxfs plugin, with FileExistsError and FileNotFoundError carrying errno, strerror and filename), and os.path.normcase (used by logging). pylib/traceback.py, inspect.py and logging therefore find sys._getframe and now work there: traceback.format_exc() and print_exc(), extract_tb, inspect.signature(), inspect.stack() and the funcName and lineno of logging records. Line numbers are those of the statement being executed (the last line of a statement that spans several lines, and the line after a block for a compound statement whose condition fails). Tests 65 and 67 pass on the CREXX interpreter. Tests 64 and 66 stay skipped there: 64 needs the time natives (gmtime, strftime, strptime, mktime) and getpid, and 66 needs a live and assignable obj.__dict__, __init_subclass__ with class keywords, types.GenericAlias (list[int] evaluates to list in the CREXX interpreter), bytes.splitlines(keepends=True), iterable enum classes and the extended binary file modes.

Defects and gaps that are known and not fixed: (1) pandas accepts tz= and ignores it. (2) In the CREXX interpreter the frame of a generator that defines a closure is never freed, and the collector makes allocation-heavy code 15 to 25 per cent slower (Chapter 10). (3) inv and solve follow the operation order of OpenBLAS and so flag a singular matrix where NumPy does (since Build 4: [[1, 2, 3], [4, 5, 6], [7, 8, 9]] raises LinAlgError in both interpreters); for 7 of 613 random integer matrices of order 6 the verdict still differs from NumPy's, and for large matrices (a few dozen rows and more), where NumPy uses blocked algorithms, the rounding differs. (4) Eigenvector, singular-vector and QR signs can differ from NumPy in the last bit (Chapter 6).

Chapter 13. Porting to VM/370

The C interpreter was written to be carried to VM/370, the IBM System/370 virtual-machine operating system with the CMS single-user environment, on which the C compiler available (GCC with the GCCLIB31 library, run under the Hercules emulator) accepts C89 and little else. This chapter describes what has been done for that goal, how to build the card deck, and what has not been done. It is the chapter to read before taking the program to any host that is not a modern POSIX system.

Status. The port has been prepared on a Linux x86-64 host; this manual did not run it on a System/370. The commit history records one run of the test corpus on VM/370plus CMS (GCC 13, -m31), which found the defect described under Sizes that fit below. Everything described as done was done and checked here: the source is strict C89 and passes the conformance matrix; every operating-system dependency is behind one adapter in src/p0_os.c; the whole Python library can be compiled into the program; and the deck step produces card images of at most 71 columns whose compiled code is identical to that of the development source. What remains is the work that needs the real compiler, the real library headers and the real machine, and is listed at the end of the chapter.

The rules

- **Strict C89.** No // comments, no declarations after statements, no long long, no stdint.h, no variable-length arrays, no inline, restrict or designated initialisers, no snprintf or other C99 library function. All functions are static; the only external symbol of the program is main. Development lines are at most 100 columns.
- **Literal character sets.** Character classes are tested against literal strings (the digits, the letters), never with ranges such as c >= 'a', so the same source is correct when the characters are EBCDIC. Embedded library text is stored as ASCII byte values and translated to the host character set when a module is read. Since Build 4 the character tests that used numbers are gone from the interpreter core: ch_print (with the literal set CS_PRINT, built once by init_chars) decides what repr(), ascii() and str.isprintable print, and digit_val, a lookup in the literal digit set, converts hexadecimal and decimal digits in json, re, the lexer, int() and bytes.fromhex.
- **Everything host-specific in one file.** The standard headers and every call that is not ISO C live in src/p0_os.c behind functions named host_*. Calls that exist only on POSIX are compiled only with -DPY_POSIX.
- **Sizes that fit.** A small integer is a long of 18 decimal digits where long has 64 bits and 9 digits where it has 32; the value stack (VSTACK_SIZE, in 16-byte values, default 400,000) and the Python recursion limit (MAX_DEPTH, default 1000) can be set with -D, because the defaults need more memory than a 16 MB CMS machine has. Every test of a small-integer limit uses SI_MAX, never a constant such as 1e17: int_from_double and the hash of a float compared with 1e17 and cast to long, which on a machine with a 32-bit long saturated at 2,147,483,647 (int(1e10) gave 2147483647 and Fraction(0.25) gave 2147483647/18014398509481984). Found running the test corpus on VM/370plus CMS (10_misc and 19_modules failed), the two tests now compare with SI_MAX and larger values become big integers (commit 6e4a7a5); the -DPY_SI32 simulation below did not catch it because it limits the small-int range but not the width of the long. Build 4 went on to look for every other place of that kind, see The 32-bit long hunt below.
- **An independent 64-bit type.** numpy.random and the SHA-512 of the random module use a u64 abstraction (a native type, or two 32-bit halves with -DPY_U64_EMUL) so that they compile where long has 32 bits and give the same streams.

The host adapter

src/p0_os.c is the complete list of what the interpreter needs from the host. A port edits this file only; the other 27,000 lines do not mention the operating system. Without -DPY_POSIX it uses only the ISO C library (time, clock, fopen, getenv).

Function or macro	Purpose	Plain ISO C / POSIX	For CMS
PY_DIRSEP PY_PARDIR PY_PATHLIST_SEP	joining names, parent directory, separator in $PYLIB	"/" ".." ":"	override with -D
PY_RMODE PY_EXE_LIBS	fopen mode for reading; also search <exe dir>/pylib	"rb" 1	"r" 0 (set by PY_HOST_CMS)

Function or macro	Purpose	Plain ISO C / POSIX	For CMS
host_join host_dirname_len	directory plus name; directory part of the script path	slash joining	CMS file naming
host_module_path	file for module leaf in search entry dir (module or package)	dir/leaf.py, dir/leaf/__init__.py	LEAF PY fm and LEAF PYI fm: a starting point, the real mapping is open
host_read_file host_fopen	whole file (or embedded module) into memory; Python open()	fopen, getc	fn ft fm conversion
host_file_exists isdir isfile size remove rename	os.path.* and os.remove, os.rename	stdio	as available
host_getenv host_getcwd host_environ	os.getenv, os.getcwd, os.environ, $PYLIB	getenv; cwd and environ only with PY_POSIX	GLOBALV, CMS query
host_isatty_stdin	prompt in the REPL	isatty(0) with PY_POSIX; otherwise 0 (use py -i)	terminal I/O
host_time host_clock host_sleep host_seed host_urandom	time.time, perf_counter, sleep, seeding of random (host_urandom: /dev/urandom for unseeded generators and SystemRandom)	gettimeofday and nanosleep with PY_POSIX; otherwise time(), clock() and a busy wait	CMS clock
host_listdir, host_mkdir, host_rmdir, host_chdir, host_getpid, host_timeparts, host_mktime, host_tzinfo	os.listdir, mkdir, rmdir, chdir, getpid; the calendar behind time.localtime, mktime and strftime	opendir, mkdir, getpid, localtime, mktime with PY_POSIX; otherwise ENOSYS and UTC	their own directory and clock services
host_sock_* (about 22 functions)	the socket and select modules	BSD sockets with PY_POSIX; otherwise every call fails with "sockets not available"	TCP/IP for VM, or a service machine

Figure 133. The adapter functions of src/p0_os.c

What the interpreter does not use, so that no adapter exists for it: popen, system, threads and signals (SIGPIPE is ignored on the first socket, with PY_POSIX). The os and os.path modules that a Python program sees are POSIX-flavoured emulations built on these functions. The socket block deserves a note: the socket and select modules (src/p7_sock.c) are written only against the host_sock_* functions, which take addresses as four bytes in network order and a port, handles as small integers, and report errors as host error numbers that become OSError.errno together with a function that maps them to the Python exception classes; a CMS port replaces the POSIX branch with the calls of the TCP/IP product it uses, and a stack with no select can poll in a loop with host_sleep.

Embedded libraries

A program that needs the Python library (re is native, but statistics, datetime, pandas and scipy are not) needs the files of pylib at run time, and on a CMS minidisk a tree of 68 files with long names and packages is awkward. Compiled with -DPY_EMBED_LIBS, the interpreter carries the library inside itself. tools/embed_pylib.py walks pylib, writes src/pylib_embed.c (56,600 lines) with one or more arrays of decimal byte values for each file (at most 32,000 bytes per array, because C89 guarantees objects of only 32,767 bytes), and a table of names, sizes and chunk lists. Only the names are string literals; the file contents contain no character literals, so the table is independent of the host character set, and host_from_ascii() maps the bytes of a module to the host character set through a literal set when the module is read. In sys.path the table is the pseudo directory <pylib>, which comes right after the script directory, so a user module can still shadow a library module, and packages, relative imports and the numpy package path work unchanged.

```
$ mkdir -p /tmp/py_empty && cd /tmp/py_empty && cp /home/claude/python/c/py_embed py &&
    cat > t.py <<EOF
import statistics, numpy as np
print(statistics.__file__)
print(statistics.mean([1, 2, 3, 4]), np.fft.fft([1.0, 0.0]).tolist())
print(__import__("sys").path[:2])
EOF
./py t.py
<pylib>/statistics.py
2.5 [(1+0j), (1+0j)]
['', '<pylib>']
```

Figure 134. An embedded build needs no pylib directory

make embed-test builds the interpreter with a bogus compile-time library directory (PY_LIBDIR=/nonexistent/pylib), copies it to an empty temporary directory, unsets PYLIB, and runs the 33 tests there; the same program built without the table fails 13 of them, so the table is really used. The same is done for the deck (deck-embed-test).

The deck step

A card image has 80 columns, and the compiler's source lines must stay within the 71 that the VM/370 build uses for source text. tools/deck.py folds a C file to a given width (default 71) without changing the token stream of the program: lines are broken only between tokens (after ;, {, }, a comma or an opening parenthesis, before && || ? : and binary operators, preferably at the lowest parenthesis depth); a string literal that does not fit is cut into adjacent literals at an escape-sequence boundary and preferably after a blank; comments are re-wrapped word by word, and a trailing comment that does not fit moves to a line of its own; macro lines keep their backslash, and the leading blanks of continuation lines are squeezed so that the spliced logical line stays within the 509 characters C89 guarantees (the longest in the deck is 444); and the output is checked to be pure ASCII without tabs, trailing blanks or ??-trigraphs. A line that cannot be folded (a very long identifier or #include) makes the tool fail instead of guessing.

```
$ cat sample.c
/* a long statement and a long string literal */
static int check(Interp *ip, Value *v, long limit) { if (v->tag == T_INT && v->u.i >
    limit && ip->depth < MAX_DEPTH) { return fail(ip, "value %ld exceeds the limit of
    this Python subset interpreter (see the manual)", v->u.i); } return 0; }
$ python3 /home/claude/python/c/tools/deck.py --limit 71 sample.c
/* a long statement and a long string literal */
static int check(Interp *ip, Value *v, long limit) {
    if (v->tag == T_INT && v->u.i > limit && ip->depth < MAX_DEPTH) {
    return fail(ip,
    "value %ld exceeds the limit of this Python subset interpreter "
    "(see the manual)", v->u.i);
    } return 0;
}
```

Figure 135. The deck step on a small file

tools/deck.py --verify ORIG DECK compares the token streams of the original and the deck (comments dropped, adjacent string literals merged), and gcc -S of py_all.c and deck/py_deck.c gives byte-identical assembly apart from the file name: the deck compiles to exactly the same code. make deck-test compiles the deck with -std=c89 -pedantic-errors and runs the 29 tests against it. The decks are committed to the repository.

File	Lines	Longest line	Use
c/src/*.c (without pylib_embed.c)	27,766	100	development source, 23 files
c/py_all.c	27,766	100	amalgamation, what make compiles
c/deck/py_deck.c	31,708	71	the deck, 995,683 bytes; about 31,700 card images
c/src/pylib_embed.c	57,490	71	pylib as byte arrays (generated)

File	Lines	Longest line	Use
c/deck/py_deck_embed.c	89,198	71	the deck with the library: about 89,200 card images, 5.0 MB

Figure 136. Sizes of the source forms (lines of text)

Building and checking

make posix gives the development build. For VM/370 the build line, as in the sister projects apl and smalltalk (cross compiler targeting the GCCLIB31 headers), is the one in the figure below. It is the documented plan and has not been run: the cross compiler and the headers are not available where this manual was prepared.

```
s390x-linux-gnu-gcc -std=c89 -pedantic -m31 -O2 -I gcclib31/src \
    -D__CMS__ -D__CHAR_UNSIGNED__ -DPY_EMBED_LIBS \
    -DVSTACK_SIZE=40000L -c deck/py_deck_embed.c
```

Figure 137. The planned VM/370 compilation (not run)

-D__CMS__ selects PY_HOST_CMS (and declares setjmp and longjmp for GCCLIB31, whose header declares only jmp_buf), -D__CHAR_UNSIGNED__ matches the S/370 default for char, VSTACK_SIZE=40000L shrinks the value stack to 640 KB, and MAX_DEPTH can be lowered the same way: a Python call needs several hundred kilobytes of C stack at the depth limit of 1000.

Target or command	What it does	Result
make test	33 tests with the posix build (since Build 4 the target builds with -DPY_POSIX; before, test 24_sockets failed there)	33 of 33
make embed-test	embedded build, empty directory, no PYLIB, bogus PY_LIBDIR	33 of 33
make deck make deckcheck	generate the deck; no line over 71 columns, no tab, no non-ASCII, no trigraph	ok
make deck-test	deck compiled with -std=c89 -pedantic-errors	33 of 33
make deck-embed-test	embedded deck, empty directory	33 of 33
make linecheck	development sources at most 100 columns	ok
make strict	gcc and clang, -std=c89 -pedantic-errors -Wall -Wextra -Wconversion -Wshadow -Wdeclaration-after-statement -Wvla -Wstrict-prototypes -Wundef -Wtrigraphs -Wlong-long, on py_all.c, the deck (each with and without -DPY_POSIX, and with -DPY_HOST_CMS) and the embedded amalgamation: no diagnostic; no C99 library function referenced by the object file; the whole suite under gcc -O0 -fsanitize=undefined,address	all ok; 32 of 32 under the sanitizers
-DPY_SI32 -DPY_U64_EMUL	simulate a 32-bit long on a 64-bit host (small ints limited to +-999999999, 64-bit emulation with 32-bit halves)	33 of 33
-funsigned-char	the S/370 default for char	33 of 33
gcc -m32 -msse2 -mfpmath=sse	a real 32-bit long on x86 (multilib); SSE2 because the x87 default gives -0.0 and 0.0 differences in tests 16 and 30	32 of 32, no warnings
tests/tools/si32_diff.py	random expressions against CPython 3 in up to three builds (normal, -DPY_SI32, -m32)	see below

Figure 138. The checks, and what each one showed in this release

The U64 emulation was also checked against native 64-bit arithmetic on 200,000 random operand pairs, and the numpy.random and random streams (MT19937, PCG64, SHA-512 seeding) are identical with and without it, and identical in the -m32 build. A clang sanitizer run, an s390 cross compiler and the GCCLIB31 headers were not available, so none of those was run.

The 32-bit long hunt (Build 4)

A 32-bit long was available only as a simulation (-DPY_SI32 limits the small-integer range, but a long is still 64 bits wide), and one real run on VM/370plus had already found a bug the simulation missed. Build 4 therefore built the interpreter with gcc -m32 as well and wrote tests/tools/si32_diff.py, which generates thousands of random expressions (integers around every boundary, big integers, floats; str, repr, format, % formatting, int(), float(), round, divmod, pow, bit operations, hash, struct, to_bytes, range, slicing, sort, dict and set, re, json, random ranges and more) and compares the output of up to three interpreters with CPython 3:

```
python3 tests/tools/si32_diff.py -n 800 -s SEED \
    /tmp/py64:normal /tmp/pysi:si32 /tmp/py32s:m32
    -c CATS   run only some categories      --self   check the generator against CPython
```

Figure 139. Comparing the builds with CPython (the paths are examples)

Everything it found was wrong with a 32-bit long, and much of it also beyond 10**18 with a 64-bit one. The fixes:

- **mkint** converts any long outside +-SI_MAX to a big integer, so a sum, product, abs, negation, int(float), len or time_ns() can no longer carry a value that is not small.
- **Exact conversions.** int to float is correctly rounded for any size, float to int is exact, int and float compare exactly (2**53 + 1 > 2.0**53), true division of huge ints is correctly rounded, and float ** int, round (also with negative digits), math.log, log2, log10, isqrt and fsum for big ints, float.hex, fromhex, as_integer_ratio and is_integer, int.bit_length and bit_count work.
- **hash()** follows CPython: the value modulo 2**61 - 1 with the sign kept, -1 becomes -2, inf is 314159 and a complex number is hash(re) + 1000003 * hash(im), for int, bool, big int, float and complex. hash(1e10) == hash(10**10) and {10**10: 1}[1e10] hold in every build. pylib/fractions.py (also used by the CREXX interpreter) no longer calls three-argument pow with a negative exponent but its own modular inverse (_modinv), and Fraction equality, ordering and hash are exact for floats.
- **Bit operations.** & | ^ ~ << >> on ints of any size with two's complement semantics (digit arrays of 16 bits). pow(a, b, m) works for big values and negative exponents. int.to_bytes, from_bytes, struct q Q I L, hex, oct, bin, %x %o %d %e %f %g, format (the , and _ groupings, n, #, precision) and str(int) and int(str) in any base up to 36 work for big values.
- **range and counts.** range with bounds, step and length beyond long (len, index, slice, in, count, index, reversed, iteration, enumerate(start=)), list repeat counts, maxsplit, insert and the count, maxsplit and pos arguments of re clamp huge values; above 2**63 they raise OverflowError as CPython does. random.randrange, randint, getrandbits, randbytes, choices, gauss and expovariate accept big ranges and seeds; time.time_ns, monotonic_ns, perf_counter_ns and process_time_ns are exact.
- **Smaller things found on the way**: float('-nan'), the signs of -0.0 // x and %, 1e308 ** 2 (OverflowError), the shortest repr of a float at a power of two (2.0**122), '{:.3}'.format(x), str.split(None, n) and rsplit remainders, count('') past the end, bytes.split(sep, n), bytes.hex(sep, n), reversed(bytes), int.__index__, math.nextafter and ulp, and repr() and isprintable() of non-printable non-ASCII characters.

With -n 800 per seed and all categories, the normal 64-bit build and the -DPY_SI32 build agree with CPython except for the differences listed in Chapter 12 (insertion order for sets with str, bytes or instance keys, UTF-8 byte strings, non-ASCII digits in float(), statistics to one unit in the last place, the repr of a bytearray with a quote; before Build 6 all sets were insertion-ordered and the hash of a Decimal differed). The -m32 build differs from them only by the limits of a 32-bit long: len(range(n)) and sys.maxsize stay below 2**31 (an OverflowError, as with CPython on a 32-bit build), repeat counts of range size for list and str are refused, numpy integer arrays are long (int64 arithmetic wraps at 32 bits), and without SSE2 the x87 gives extended precision and -0.0. The CREXX interpreter was added to the hunt in Build 5 (next section).

The same hunt in the CREXX interpreter (Build 5)

The CREXX interpreter has no C long, but its small integers stop at 18 digits and its floating point arithmetic converts through decimal text with 15 digits, so it failed on the same inputs. Build 5 ran si32_diff.py against ./crexx/py (python3 tests/tools/si32_diff.py -n 150 -s SEED --timeout 900 ./crexx/py:crexx). With -n 150 and seeds 1 to 3, 385, 400 and 422 of 3150 cases differed from CPython before; the last run of seeds 1 to 4 differs in 3 to 5 cases per seed (the order of set union and intersection and the hash of a frozenset, which Build 6 left as they were for the CREXX interpreter, except that the hash of a frozenset of ints now equals CPython's; the repr of a bytearray with a quote, the Unicode version of repr for an unassigned code point, float() of non-ASCII digits). The fixes:

- **Bit operations** & | ^ ~ << >> with two's complement semantics for any size (24-bit limbs), int.to_bytes and from_bytes, hex, oct, bin, format and % for big ints, pow(a, b, m) with big, negative (modular inverse) exponents and big moduli.
- **Exact numbers.** float(big) and big / big are correctly rounded and raise OverflowError where CPython does, int and float compare exactly, int(1e22) and the digits of large floats are exact (the 15-digit conversions of CREXX are never used for them), math.floor, ceil, log, isqrt and fsum work on huge

values, round with negative digits is right. Float parsing is exact for overflow and denormal input, and float repr is shortest also at powers of two and for denormals (5e-324).

- **hash()** is CPython's modular hash for int, big int, float and complex, and dict and set lookups mix 1, 1.0, True and Fraction(1).
- **range** with bounds, steps and lengths of any size (len, index, slice, in, count, reversed, iteration, comparison, repr), enumerate(start=) and slice.indices.
- **Smaller things**: float ** int overflow (OverflowError (34, 'Numerical result out of range')), -0.0 // x, split(None, n) remainders, repr and isprintable of non-printable code points, str.translate and str.maketrans, the format() alignment of a string with a precision, the grouping options of format for b, o, x and X, bytes.hex(sep, n) with a negative group size, UTF-8 decoding with 'replace', the maximal-subpart rule, huge counts (OverflowError or clipped as in CPython), huge repeat counts (MemoryError), and exact statistics (mean, variance, stdev) in pylib/statistics.py. The shared library also gained strict JSON numbers (01, .5 and 1. are errors), struct formats n and N only in native mode, and the OverflowError of re.sub for a huge count; the C interpreter passes the same tests.

Assumptions that remain

The character-set strategy is not settled, and a number of places still assume ASCII or IEEE arithmetic. They are listed in c/PORTING.md; in short:

- **Strings are UTF-8 bytes of Unicode code points.** chr, ord, the \\x and \\u escapes, encode and decode, bytes, json's \\uXXXX, %c and the Latin, Greek and Cyrillic case tables assume that code points below 128 are the bytes 0 to 127. On EBCDIC chr(65) would give the byte 65, not the letter A. The remedy is a translation at the boundaries (source in, output out, chr and ord, encode and decode), or a decision that Python data is ASCII inside and the host_ I/O functions translate.
- **Numeric tests on control characters**: repr() and ascii() of str and bytes and str.isprintable now use the literal set CS_PRINT plus the letters and digits (ch_print), and non-ASCII code points a range table (uni_nonprint, p6_obj.c). Still numeric: the JSON encoder (c >= 32, c < 127, p7_json.c, where the numbers are Unicode code points as in the JSON standard) and its \\b and \\f escapes, which use the C escapes.
- **Collation.** str comparison and sorting are memcmp of the bytes. EBCDIC puts lower case before upper case before digits; Python puts digits before upper case before lower case. Ranges such as [a-z] in re compare code points and on EBCDIC would include characters between i and j and between r and s; the case folding loops of re and hexadecimal and decimal digit values (digit_val) now run over the literal alphabet and are not affected.
- **File names.** The CMS file name is a file name, a file type and a file mode (fn ft fm), at most 8 characters each. The mapping import a_b to A_B PY fm cannot express packages (pandas.core against scipy.sparse.linalg) or names longer than eight characters (interpolate, groupby). Options are an alias table, one library file per package, or the embedded table, which has no such limit and is the likely choice.
- **Floating point** is assumed to be IEEE 754 double. S/370 hexadecimal floating point has a range of about 1e-78 to 7e75, no infinity or NaN and different rounding; repr(), float formatting, numpy printing, struct's f and d formats, inf and nan, overflow and the shortest-repr search will all differ. The apl sister project limits values to +-7e75 and raises domain errors; that approach is not applied here.
- **Integers.** mkint turns every long outside +-SI_MAX into a big integer and hash() follows CPython's modular scheme in every build (it no longer differs on 32-bit hosts). numpy integer arrays use long (32 bits wide on a 32-bit host), and struct and int.to_bytes assume the IEEE and LP64/ILP32 native sizes chosen from LONG_MAX.
- **Fixed buffers.** sprintf and vsprintf (C89 has no snprintf) write into fixed buffers with %.Ns precision limits; do not widen those formats.
- **Time.** Without PY_POSIX time.sleep is a busy wait, time.time has a resolution of one second and the REPL does not detect a terminal (use py -i).

What remains for the real port

1. Compile deck/py_deck_embed.c against the real GCCLIB31 headers with -D__CMS__ -D__CHAR_UNSIGNED__ and fix what the old compiler and library reject. Untested: the headers, clock() and CLOCKS_PER_SEC, qsort, strtod, %g, and vsprintf in GCCLIB31.
2. Produce the TEXT file or load deck, check the use of storage (value stack, heap, the 1.1 MB of embedded library text) in a 16 MB machine, and tune VSTACK_SIZE and MAX_DEPTH.

3. Decide the character-set strategy (above): keep all Python data ASCII inside and translate in the host_ I/O functions and the few C literals used as Python names, or compile the interpreter itself as ASCII-in-EBCDIC with translation of the terminal I/O only.
4. Decide the CMS naming of user programs and modules, and write the fopen string conversion in host_fopen and host_read_file (fn ft fm, or fn.ft, whichever the C library takes).
5. Terminal I/O: line mode with 80-column records, and the prompt behaviour of stdin (host_isatty_stdin is 0 without PY_POSIX).
6. Run the test suite on the target. Expect differences in the float tests (hexadecimal floating point) and in everything that depends on the 32-bit long limits.
7. Replace the host_sock_* block with the calls of the TCP/IP product of the machine, if the socket module is wanted.
8. Repeat the full run of all 33 tests on the target (Hercules or VM/370plus), and repeat the 32-bit comparison with tests/tools/si32_diff.py there if Python 3 is available on a host that can run the binary. The commit history records a run of the corpus on VM/370plus CMS that found the 32-bit long defect above, but not its complete result.

Appendix A. Command Summary

```
C interpreter
  ./c/py [-i] file.py [arg ...]       run a script;  sys.argv = [file.py, arg, ...]
  ./c/py -c "code" [arg ...]          run a string;  sys.argv = ['-c', arg, ...]
  ./c/py [-i]                         REPL on standard input (-i: print the prompts)

CREXX interpreter
  ./crexx/py file.py [arg ...]        run a script
  ./crexx/py -c "code" [arg ...]      run a string
  ./crexx/py                          run the program read from standard input
  ./crexx/py -h                       usage

Without the launcher
  crexx crexx/py.crexx -noexec                compile only
  crexx crexx/py.crexx -nocompile -args --pypath=DIRS file.py [arg ...]

Build and check (in c/)
  make posix | make | make embed | make deck | make deck-embed
  make test | embed-test | deck-test | deck-embed-test | deckcheck | linecheck | strict
      | bench
  (make test builds with -DPY_POSIX, as make posix)
```

Figure 140. Command lines

Variable	Used by	Meaning
PYLIB	both	colon-separated list of directories searched for modules, after the script directory
CREXX_VM	crexx/py	the crexx driver of a CREXX built with crexx/vm-startup.patch (start-up 0.4 s instead of 2 to 3 s); wins over CREXX
CREXX	crexx/py	the crexx driver (default: crexx on the PATH)
VERBOSE, VERBOSE_LINES	tests/run_tests.sh	VERBOSE=1 shows a diff for each failing test (VERBOSE_LINES lines, default 10)
SKIP	tests/run_tests.sh	space-separated test names to skip

Figure 141. Environment variables

Option	Meaning
-DPY_POSIX	clock, sleep, getcwd, environment, isatty, and the BSD sockets of the socket and select modules (make posix)
-DPY_LIBDIR=\"dir\"	compile-time library directory, the last entry of sys.path (make LIBDIR=dir)
-DPY_EMBED_LIBS	compile pylib into the program (make embed); sys.path gets the entry <pylib> after the script directory
-DPY_HOST_CMS -D__CMS__ -D__CHAR_UNSIGNED__	VM/370 CMS host variant (Chapter 13); __CMS__ sets PY_HOST_CMS
-DVSTACK_SIZE=n -DMAX_DEPTH=n	size of the value stack in 16-byte values (default 400000L) and the Python recursion limit (default 1000)
-DPY_SI32 -DPY_U64_EMUL	simulate a 32-bit long (small ints up to 999999999) and emulate 64-bit arithmetic, for testing the port on a 64-bit host
-DPY_GC_STRESS	collect garbage at every safe point (for testing)
-std=c89 -pedantic -Wall -Wextra	the flags with which the source is clean

Figure 142. Build options of the C interpreter

Exit status: 0 on success, 1 for an uncaught exception (traceback on standard error), the argument of sys.exit() when it is an integer, 2 when the script file cannot be opened. The tests are run with tests/run_tests.sh interpreter [args].

Appendix B. Contents of the Repository

Path	Contents
README.md SPEC.md	overview and the specification of the language subset shared by the two interpreters
c/Makefile c/README.md	build rules (posix, embed, deck, deck-embed, the test and check targets, bench, clean) and notes on the C interpreter
c/PORTING.md	the VM/370 port: adapter, deck step, embedded library, conformance checks, remaining work (Chapter 13)
c/py_all.c c/py_embed_all.c	the amalgamated source, generated by tools/amalg.sh and checked in; py_all.c is what make compiles; py_embed_all.c has the library tables inlined
c/deck/py_deck.c py_deck_embed.c	the same folded to 71 columns for the VM/370 deck step (generated, checked in)
c/src/py.c	includes the parts below in order
c/src/p0_os.c	standard headers and every dependency on the host operating system (the host_* adapter functions, Chapter 13)
c/src/p0_base.c	types, tagged values, mark-and-sweep collector, strings, lists, dictionaries, sets
c/src/p1_num.c	big integers (base 10000), float formatting and parsing
c/src/p2_lex.c	lexer with INDENT and DEDENT, f-strings, escapes
c/src/p3_parse.c	recursive-descent parser, AST, comprehension rewriting
c/src/p4_res.c	scope resolution: locals, cells, free variables, globals
c/src/p6_obj.c	attributes, equality, hashing, comparison, slicing, operators
c/src/p5_eval.c	evaluator, calls, generators, classes, exceptions
c/src/p7_fmt.c	format mini-language, % formatting, str.format, f-strings
c/src/p7_bi.c	builtin functions and the methods of str, list, dict, set, tuple, int, float
c/src/p7_bytes.c p7_cplx.c	bytes, bytearray, int.to_bytes, struct; complex, cmath, replacements for the C99 libm functions
c/src/p7_np.c p7_np2.c	numpy ndarray (bool, int64, float64, complex128), ufuncs, reductions, indexing, printing; more numpy functions
c/src/p7_la.c p7_rnd.c	numpy.linalg (LU, Jacobi, Householder QR, SVD, lstsq); numpy.random (MT19937, PCG64, SeedSequence, ziggurat: NumPy's exact sequences)
c/src/p7_imp.c	import system, eval, exec, globals, os
c/src/p7_pyrand.c	random (MT19937, SHA-512 seeding and the algorithms of CPython 3.13: exact sequences)
c/src/p7_mod.c	math, sys, time; the exception hierarchy; initialisation
c/src/p7_re.c p7_json.c	native re (parser, compiler, backtracking VM) and json
c/src/p7_sock.c	socket and select on the host_sock_* adapter
c/src/p8_main.c	command line and REPL
c/src/pylib_embed.c	generated: pylib as byte arrays for -DPY_EMBED_LIBS
c/tools/	amalg.sh (builds py_all.c from src/), deck.py (the 71-column deck step), embed_pylib.py, strict.sh (conformance matrix)
c/bench/	fib.py loop.py collections.py strings.py numpy_bench.py re_bench.py json_bench.py and run.sh
crexx/README.md STARTUP.md vm-startup.patch	notes on the CREXX interpreter; the profile and the fix for the start-up time; the patch for the CREXX virtual machine (rxvmintp.c)
crexx/py	POSIX shell launcher: compiles py.crexx on first use, then runs it; honours CREXX_VM and CREXX
crexx/py.crexx	the interpreter, generated by build.py and checked in so that the repository runs at once
crexx/build.py	concatenates src/*.crexx and expands the symbolic constants, class and builtin tables
crexx/src/a_head b_lex c_parse	start-up, init and boot; lexer; nodes, parser and scope resolution
crexx/src/d_obj e_big f_ops	objects, classes, dictionaries; big integers and float printing; arithmetic, repr and str
crexx/src/g_call g_item g_eval g_gen	calls and frames; subscripts and slices; the evaluator; generators
crexx/src/h_builtins h_meth h_mod h_imp h_ns	builtin functions; methods; native modules; imports; globals, eval, exec
crexx/src/i_fmt j_np k_np2	formatting; numpy ndarray; more numpy

Path	Contents
crexx/src/l_bytes l_cplx l_sub l_sock	bytes and bytearray; complex numbers and the bridge to the complex arrays of pylib/numpy/_cplx.py; subclasses of built-in types and __new__; the socket primitive
crexx/pylib/	socket.py and select.py (CREXX only, on the rxsocket library)
crexx/src/consts.txt classes.txt builtins.txt	tables that build.py turns into constants
crexx/py.rxas py.rxbin	generated assembler and bytecode (not checked in)
pylib/*.py	the standard library modules (Chapter 5): abc bisect cmath collections contextlib copy csv dataclasses datetime decimal enum fractions functools heapq io itertools json math operator os pprint random re statistics string struct sys textwrap types typing warnings, and _clsinfo.py (annotation scanner); the modules that are native in the C interpreter (re json cmath struct math os random sys) are used by the CREXX interpreter
pylib/numpy/	__init__.py (functions not native), _cplx.py (complex arrays of the CREXX interpreter), _dt.py (dtype objects of the CREXX interpreter), fft.py, linalg.py, random.py
pylib/pandas/	__init__ base core dtindex fmt groupby npcompat pio resample reshape strs tslib window
pylib/scipy/	__init__ _la _util constants fft integrate interpolate linalg optimize special stats; signal/, sparse/ and spatial/ packages
tests/	33 test programs NN_name.py with expected output NN_name.out, run_tests.sh, the package pkg/ used by 15_import.py, and tools/fuzz_re_json.py (differential fuzzer for re and json, not part of the suite)
docs/	this manual (Python-CREXX-Guide.docx and .pdf)

Figure 143. Repository

Appendix C. Index of Modules

The table lists every module that can be imported, with the interpreter in which it is native code (N), Python from pylib (P), or native with a Python fall-back for missing names (N+P). A dash means that the module does not exist. Chapter 5 describes the modules.

Module	C interpreter	CREXX interpreter	See
abc	P	N+P	5
bisect	P	P	5
cmath	N	P	5
collections	P	P	5
contextlib	P	P	5
copy	P	P	5
csv	P	P	5
dataclasses	P	P	5
datetime	P	P	5
decimal	P	P	5
enum	P	P	5, 12
fractions	P	P	5
functools	P	P	5
heapq	P	P	5
io (StringIO only)	P	P	5
itertools	P	P	5
json	N	P	5
math	N	N+P	5
operator	P	P	5
os, os.path	N	N+P	5
pprint	P	P	5
random	N	N+P	5
re	N	P	5
statistics	P	P	5
string	P	P	5
select	N (POSIX)	P (crexx/pylib)	5
socket	N (POSIX)	P (crexx/pylib)	5
struct	N	P	5
sys	N	N+P	5
textwrap	P	P	5
time	N	N	5
types	P	P	5
typing	P	N+P	5
warnings	P	N+P	5
__future__	-	N	12
numpy	N+P (overlay)	N+P (overlay)	6
numpy.fft	P	P	6
numpy.linalg	N	P	6
numpy.random	N	P	6
pandas (and its submodules)	P	P	7
scipy.stats special optimize integrate interpolate linalg spatial signal sparse fft constants	P	P	8
collections.abc, builtins, array, glob, shutil, pathlib, argparse, hashlib, base64, uuid, threading, logging, unittest, traceback, inspect, gc, numbers, locale, platform, secrets, tempfile, zlib, sqlite3, calendar, weakref, asyncio, selectors, subprocess, ctypes, signal	-	-	12

Figure 144. Modules

Appendix D. Sample Programs and Tests

The directory tests holds the 32 regression tests of the figure below. Each test is a program, with the output that CPython 3 printed for it; both interpreters print exactly that output.

File	Lines	Covers
01_basics.py	23	arithmetic, floats, bit operations, truthiness
02_strings.py	19	indexing, slicing, methods, formatting
03_control.py	55	if, while, for, break, continue, else on loops, comprehensions
04_functions.py	60	defaults, *args, **kwargs, keyword-only, closures, lambda, recursion, decorators
05_collections.py	41	list, tuple, dict, set operations and methods, sorting
06_classes.py	73	classes, inheritance, super, properties, special methods
07_exceptions.py	81	try, except, else, finally, raise, user exceptions, messages
08_iter_gen.py	50	iterators, generators, generator expressions, yield from
09_algos.py	54	sieve, sorting, searching, dynamic programming, big integers
10_misc.py	24	math, random, sys, time
11_numpy.py	30	numpy: creation, indexing, broadcasting, reductions, printing
12_stdlib.py	38	json, re, datetime, collections, itertools, functools, statistics, string, copy, operator, textwrap
13_pandas.py	36	Series and DataFrame basics, groupby, merge, printing
14_scipy.py	28	stats, optimize, integrate, interpolate, linalg, special, spatial
15_import.py	69	import forms, packages (tests/pkg), relative imports, caching, star-import, circular imports
16_linalg_random.py	96	numpy.linalg and numpy.random (random values checked for shape, range and determinism)
17_language2.py	222	multiple inheritance and MRO, descriptors and special methods, walrus, send/throw, eval/exec, many corner cases
18_re.py	154	regular-expression corner cases
19_modules.py	173	heapq, bisect, dataclasses, enum, contextlib, fractions, decimal, csv, pprint, collections, itertools, functools
20_pandas2.py	193	more pandas: indexing, missing data, groupby, merge, pivot, strings, dates, IO, printing
21_scipy2.py	180	more SciPy: distributions, tests, optimisation, ODEs, splines, sparse, signal, constants
22_complex.py	22	complex numbers, cmath, numpy complex128 arrays, complex eigenvalues and roots
23_new_subclass.py	67	__new__, subclasses of tuple, list, dict, set, str, int, float and Exception, __missing__, namedtuple
24_sockets.py	33	socket and select over a loopback TCP connection, error classes, address functions
25_fft.py	14	numpy.fft and scipy.fft basics
26_pandas_dt.py	17	date_range, Timestamp, the .dt accessor, to_datetime, partial-string indexing
27_pandas_dt2.py	457	pandas datetimes in depth: Timestamp, Timedelta, DatetimeIndex, offsets, resample, time-based rolling, formatting
28_fft_more.py	74	FFT lengths, norms and axes, n-dimensional transforms, dct and dst, freqz, hilbert, spectrogram, welch
29_np_dtype_pkg.py	20	dtype objects, scalar types, numpy.fft and numpy.linalg imported as submodules
30_eig.py	115	eigvals, eig, eigh, svd and qr on random, symmetric, triangular, integer and sparse matrices up to 8 by 8: eigen-equation residuals, orthogonality, LAPACK eigenvalue order
31_gc.py	165	allocation-heavy code whose results must not depend on when objects are reclaimed: linked lists, trees, generators, closures, classes, dictionaries, numpy arrays in loops
32_frozenset.py	29	frozenset: construction, operators and their result types, comparison with set, hash, use as dict key and set element, immutability, subclass
33_hashing.py	134	unhashable set, list, dict and bytearray (set element, dict key, add, get, setdefault, pop, fromkeys, tuple item), set probes converted to frozenset, tuple and frozenset hash values of CPython 3.13

File	Lines	Covers
34_hash64.py	48	results of a user __hash__ of any size, dict lookups and the iteration order of sets of such objects (skipped by the CREXX interpreter for the set order)
34b_hash_values.py	36	the value part of 34_hash64: hash() of 2**63 - 1, -2**63, 2**64 + 5, hash((1, 2)), -1 becoming -2, TypeError for a non-int result, dict lookups with big hashes
35_match_walrus.py	38	match with every pattern kind, the walrus operator in conditions and comprehensions, / in parameters, annotations, ExceptionGroup and except*, star expressions in displays

Figure 145. Regression tests (tests/)

File	Shows
c/bench/fib.py	recursive function calls: fib(27)
c/bench/loop.py	a three-million-iteration integer loop and a one-million-iteration loop
c/bench/collections.py	dict, list, set and comprehension work
c/bench/strings.py	string building, split, upper, replace, formatting
c/bench/numpy_bench.py	numpy arithmetic, reductions, matrix products, masks and sorting on 1,000,000 elements, each operation timed
c/bench/re_bench.py	regular expressions: findall, sub, split and finditer on a 50 KB text (argument pure: the Python re of pylib)
c/bench/json_bench.py	json dumps and loads of 300 records (argument pure: the Python json of pylib)
c/bench/run.sh	runs the seven programs with ./py and python3 and prints the times, and the native re and json against the pylib versions
c/bench/bigint.py fib25.py small_loop.py small_coll.py small_str.py small_np.py	the smaller programs and the big-integer program of Chapter 11
c/bench/fft_bench.py fft_small.py complex_bench.py	FFT (5 rounds of 4096 points; one round of 1024 points) and Mandelbrot points with complex numbers, Chapter 11

Figure 146. Benchmark and example programs

Every example program printed in this manual was run on the C interpreter, and its output is the output shown; the programs were also run on CPython 3.13 (with NumPy 2.5, pandas 3.0 and SciPy 1.18 for Chapters 6 to 8) and on the CREXX interpreter, and the places where the output differs are noted in the text or in Appendix E.

Appendix E. Differences between the C and CREXX Interpreters

Both interpreters implement the same language and pass the same tests. They differ where one of them has a feature the other lacks, where the platform imposes a difference, and in speed. The figure below collects the differences that were found; Chapter 12 explains each one.

Item	C interpreter	CREXX interpreter
Implementation	strict C89, mark-and-sweep GC	CREXX Level B, parallel integer arrays, no GC
Start-up	2 ms	about 0.4 s with the patched CREXX VM (CREXX_VM), 2 to 3.4 s with a stock VM; several minutes once to compile
Speed	about CPython	10 to 100 times slower than CPython
str	bytes (len, index and slice count bytes)	Unicode code points
bytes, bytearray, complex, struct, __new__, subclasses of built-in types	supported	supported (since this release both interpreters have them)
memoryview, array module	absent	absent
Walrus :=	supported (Build 9)	supported (Build 9)
lambda with yield	syntax error	supported
async def	syntax error	accepted and run as ordinary def
match, except*, ExceptionGroup	supported (Build 9)	supported (Build 9)
super() in classmethod	works	fails
__eq__ without __hash__	unhashable (as CPython)	still hashable
Enum: Color(1)	works	fails
typing.NamedTuple("P", [...])	works	fails
Generic[T] as a base class	TypeError ('type' object is not subscriptable)	works
sys.maxsize	999999999999999999	9223372036854775807
sys.platform	c89	linux
sys.version_info	(3, 12, 0, 'final', 0)	(3, 13, 0, 'final', 0), although sys.version says 3.12.0
Recursion limit	1000	about 900
Non-ASCII text	byte semantics; isalpha, title wrong	code points; ß.upper() is ß
str.translate str.maketrans	absent	present
int.is_integer	absent	present
Set iteration order	CPython 3.13 table emulated: the order of sets of int, bool, None, float, complex, tuples and frozensets of those equals CPython's; sets with str or bytes keys, or instances without __hash__, use insertion order	insertion order always (set union and intersection too)
__hash__ returning 2**62 or more	as CPython since Build 8: a value that fits 64 bits is kept (-1 becomes -2), a larger int is reduced modulo 2**61 - 1; sets of such instances follow the CPython table order	as CPython since Build 9 (hash(), TypeError for a non-int result); sets of such instances keep insertion order
iter(callable, sentinel)	present	absent
ascii()	absent	present
range.start .stop .step	absent (C); count, index also absent	start, stop, step present; count and index absent
math: native functions	almost all, but fma nextafter sumprod ulp absent	native sqrt, trigonometry, ...; erf, gamma, fsum and others in Python (fsum not exactly rounded)
random extras (gauss, choices, ...)	absent	present (Python)
os.remove, rename, unlink	present	absent
file.closed, file.name	absent	present

Item	C interpreter	CREXX interpreter
sys.stdout.buffer	absent	present as an attribute, write(bytes) fails
os.getpid os.path.getsize	absent present	present absent
time_ns, perf_counter_ns	absent	present
sys.exc_info sys.getsizeof	present absent	absent present
__future__ module	absent (ModuleNotFoundError)	present
Inconsistent dedent	SyntaxError	accepted
Traceback line numbers	line of the error	always line 1
REPL	yes	no (runs a program from stdin)
__file__ of the main script	not defined	defined
numpy: np.sort axis, ndim	last axis only, up to 8 dimensions	any axis
numpy: slices of the first axis	copies	views
numpy: integer overflow	wraps at 64 bits	becomes float
numpy: argwhere, tri, ediff1d, sinc, nanargmax, nanprod, true_divide, copysign, arcsinh, arccosh, arctanh, Inf, NaN, ...	present	absent
numpy.linalg and numpy.random	native	Python (pylib/numpy)
numpy.linalg eigvals, eig, eigh, svd, qr	native; ports of the LAPACK routines (dgeev, dsyevd, dgesdd, dgeqr2)	Python (pylib/numpy/linalg.py), the same algorithms; same results as the C interpreter
numpy complex arrays	native complex128; sort, argsort, unique, diag, trace, allclose, where and linalg (except eig/eigvals) raise TypeError	class _CArr in Python; sort, unique, diag, trace, allclose, where work; concatenate with real arrays and complex linspace fail
numpy sized dtype names (int8 ... uint64, float16, float128, complex64 ...)	aliases of the four real types	absent (float32, int32 and a few more exist; int_ and float_ do not)
numpy.fft, scipy.fft	Python on native complex arrays, fast enough for thousands of points	Python on _CArr; 1024 points take 4 s, 5 x 4096 take 31 s and 450 MB
numpy.linalg.multi_dot, trace	absent	present
re	native engine; byte offsets; ASCII-only \w and case folding; conditionals, possessive and atomic groups; fast	Python engine; code point offsets; no conditionals; possessive quantifiers act as greedy; about 100 times slower
json, cmath, struct	native	Python from pylib, same results
socket, select	native, only with -DPY_POSIX; inet_pton and inet_ntop; no makefile, socketpair, create_server	Python on rxsocket; makefile, socketpair, create_server; no inet_pton; select polls pending bytes
pandas, SciPy, the remaining standard library modules	identical Python source	identical Python source
Memory	garbage collected	garbage collected since Build 3 (--memstat, --nogc, --gcfix, --gcstress); generator frames that define closures leak; 72 MB at start

Figure 147. Differences between the interpreters